MStruct
Technical Note

Polymath9

4 November 2026

Version 1.0.0 / 版本 1.0.0
https://doi.org/10.5281/zenodo.23637500
© 2026 Polymath9. CC BY 4.0

This note defines MStruct and describes its syntax, rule tables and rendering conventions. The syntax entries include source examples. The English text is followed by the complete Chinese text.

本文定义 MStruct，并逐条说明其语法、规则表与渲染约定。语法条目配有源码示例。英文全文在前，中文全文在后。

Contents

Part I English **3**

1 Definition of MStruct **4**

2 Basic syntax **11**

3 Additional event syntax **17**

4 Chord symbols **29**

5 Roman numerals **37**

6 Tag events **46**

7 Timeline syntax **71**

8 Search syntax **76**

9 Rendering rules **102**

Part II 中文 **107**

1 MStruct 定义 **108**

2 基本语法 **114**

3 更多事件语法 **120**

4 和弦记号 **132**

5 罗马数字 **140**

6 标签事件 **148**

7 时间轴语法 **173**

8 搜索语法 **178**

9 渲染规则 **202**

Part I
English

Abstract

MStruct is a text-based system for describing and presenting musical structure, developed by Polymath9 and inspired by simai notation and LaTeX.
It records harmonic and structural analysis in time order.
It combines chord symbols, Roman numerals, analytical tags and musical settings in measures and timed slots.
This technical note defines the document model and gives an entry-by-entry reference for basic events, chord notation, Roman notation, tags, recording timelines and search queries.
Source examples accompany the syntax descriptions.
The search reference pairs queries with concrete matching and nonmatching passages and explains each result.
The note also describes rule-bundle identity, compatible updates and historical interpretation.
A chapter on rendering records the vertical grid, alignment, range continuity and horizontal layout of the score.
The complete English text is followed by the complete Chinese text.

Keywords: MStruct; music analysis; harmonic analysis; text notation; query language; digital musicology

Authorship and AI assistance

Generative AI tools, including OpenAI Codex, assisted with document structuring, drafting, translation, editorial revision, consistency checking and the preparation of validation scripts.
Polymath9 directs the MStruct language design and musical definitions and makes the final editorial decisions.
The technical descriptions are based on the implementation and rule tables.
Polymath9 is the author of this note and assumes responsibility for its contents.

1 Definition of MStruct

1.1 Definition and origins

MStruct is a text-based system for describing and presenting musical structure, developed by Polymath9 and inspired by simai notation [1] and LaTeX [2].
It supports authoring, rendering and searching musical analyses.
Its core is a musical annotation language that records harmonic and structural interpretations in time order.
The analysis reduces a work to selected harmonic relationships and formal organisation.

1.2 Uses and limits

MStruct uses harmonic events as the framework for musical annotations.
Its main uses and representational limits are as follows.

1. **Presenting analyses**

   The MStruct editor renders written analyses as formatted, readable analysis scores.
   These scores and their playback presentations can be used to produce demonstration videos.

2. **Finding musical examples**

   The components of MStruct follow explicit syntax rules, particularly for chord symbols and Roman numerals.
   As analyses accumulate, learners and researchers can search the encoded techniques and chord progressions to find examples for study and research.

3. **Organising interpretations**

   MStruct is fundamentally a musical annotation language.
   Together with COMMENT, it organises an analyst' s interpretations and prose comments in a formatted text file.
   These files could support future statistical processing and workflows involving large language models.

4. **Representational limits**

   The harmonic-event framework does not fully represent independent contrapuntal voices, serial organisation or the evolution of sound spectra.
   It also does not express the hierarchical voice-leading structures required for Schenkerian analysis.
   It can record the harmonic aspects of a passage where those aspects are meaningful, while these other organising relationships remain outside its representation.

1.3 Application scope

MStruct was designed primarily to record chords and harmonic or Roman-numeral analyses, using harmonic events as the temporal framework for tonal, scale, modal and formal annotations. The amount of useful information it can record depends on the musical style and its organising principles.
The following groups are ordered by the design priorities of MStruct, from highest to lowest.

1. **Music based on common-practice harmony**

   This group includes most classical music of the Common Practice Period and much harmony-driven modern popular music, jazz and electronic music based on related harmonic structures.
   It has the highest design priority.
   MStruct can usually record substantial analytical information for this music and include it in the archive' s search index.

2. **Weakly functional or tonally ambiguous harmony**

   The chords in this group remain mainly tertian or quartal, but their functions may be weak or their tonal centres ambiguous.
   Examples include late-Romantic and Impressionist classical music, modal jazz, and other music using comparable harmonic structures.
   Chord symbols, formal structure and comments can usually preserve useful information.
   Roman-numeral analysis plays a reduced role and may be inapplicable in some passages.

3. **Popular and electronic music driven by rhythm and timbre**

   This music may remain within a tonal harmonic framework while using simple chord progressions and placing much of its musical information in rhythmic or timbral change.
   MStruct can record the overall harmonic structure as a searchable harmonic index and a source of simple examples.
   It provides limited quantitative information about the non-harmonic features that drive this music.

4. **Tonal polyphony with elaborate counterpoint**

   Examples include some works by Bach and Beethoven.
   MStruct can record their harmonic changes.
   Because its harmonic reduction does not encode the individual melodic lines, information needed for contrapuntal analysis is lost.

5. **Music organised primarily through melodic lines or series**

   This group includes Gregorian chant, some modern polyphony and serial music whose vertical structures lie largely outside traditional tertian or quartal harmony.
   The ordering and horizontal development of a series are central to serial organisation and are not preserved by a sequence of chord symbols alone.
   Atonality by itself does not exclude every use of MStruct.
   Some freely atonal music still uses triadic sonorities while seeking an approximately equal weighting of the twelve pitches; those harmonic aspects may permit a limited quantitative analysis.

6. **Music organised primarily through timbre and sound spectra**

   This group includes music in which timbral or spectral evolution is the principal organising process, as in relevant spectral-music practices.
   Harmonic-event annotations do not capture the principal information needed for a quantitative analysis of those processes.

7. **Music outside twelve-tone equal temperament**

   This group includes xenharmonic music, such as some works by Jacob Collier.
   MStruct' s twelve-pitch framework does not preserve the tuning and microtonal relationships that distinguish this music.
   For music in the last three groups, its contribution is generally limited to organising descriptive metadata and marking such local modes or scales as can be represented.

1.4 Document envelope

Syntax

```
METADATA: ... END_METADATA / MAIN: ... END_MAIN / COMMENT: ... END_COMMENT
```

The metadata section (`METADATA:` / `END_METADATA`) stores the work name, document options and other descriptive information.
The analysis section (`MAIN:` / `END_MAIN`) records the music analysis in time order.
The comment section (`COMMENT:` / `END_COMMENT`) defines numbered prose comments that the analysis can reference.
A complete file contains METADATA and MAIN, followed by an optional COMMENT section.
Section markers occupy their own lines.
The following chapters mainly discuss MAIN; other scopes are identified where needed.
The example below shows the section boundaries, and Chapter 2 explains its analysis settings.

Examples

```
METADATA:
{SONG_NAME}Example
{IS_STABLE_TEMPO}False
END_METADATA

MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;
END_MAIN
```

1.5 Elements of the analysis

MAIN organizes musical time into measures (ending in `;`).
Each measure contains a sequence of slots (separated by `,`).
A slot is a span of time that holds musical content (its duration is set by `{D:}`).
A musical event describes a harmony (`C|I`), a rest (`0`) or a continuation (`-`) in a slot.
A setting (`{TYPE:value}`) specifies conditions such as measure number (`{BN:}`), meter (`{TS:}`), duration (`{D:}`), tempo (`{BPM:}`) or tonal reference (`{TC:}`).
The tonal reference (`{TC:}`) supplies the key used to interpret harmonic scale degrees.
A control (for example `{||}` or `{||:}` / `{:||}`) marks a boundary or arranges the performance order, as in a repeat.
An annotation (for example `[F:A||]` or `[NR:P]`) adds information to a position.
An analytical tag (for example `</PAC>` or the pair `<Tonality>` / `</Tonality>`) names a musical feature at a position or over a span.
A harmonic event (for example `C|I`) can place two interpretations in separate fields.

The C field (for example `[C:C]` or `C|`) names a chord using its root and chord type.
The R field (for example `[R:I]` or `|I`) uses Roman numerals to describe harmonic scale degrees and qualities relative to the tonal reference.

1.6 Characters and nesting

Delimiters

```
{} [] <> , ; |
```

Structural punctuation uses ASCII characters.
Whitespace and line breaks can separate complete tokens.
A semicolon closes an ordinary measure.
Commas inside a matched bracketed list separate items within that field.
End tags use `/`.
Two backslashes inside a harmonic field introduce an alternative analysis, as described in Chapter 3.
Setting type names accept either letter case.
Note spelling, tag names and many setting values require the letter case shown in their definitions.

1.7 Rule bundles

A rule bundle is a set of configuration tables used by MStruct.
These tables define the available musical symbols, their meanings and the input toolbox.
The rule descriptions in Chapters 1 to 8 reflect the rule tables as of 4 November 2026.

1.8 Format, rules and application versions

Fields

```
MSTRUCT_FORMAT_VERSION / MSTRUCT_PROFILE_ID / MSTRUCT_PROFILE_HASH /
↪  AUTHORING_APP_VERSION
```

A document records four system fields.
The format version ({MSTRUCT_FORMAT_VERSION}) identifies the document grammar.
The profile ID ({MSTRUCT_PROFILE_ID}) identifies a rule set and its version series.
The hash ({MSTRUCT_PROFILE_HASH}) identifies the exact rule content.
The authoring application version ({AUTHORING_APP_VERSION}) records the software that wrote the file.
Grammar, rules and applications have separate versions.

Examples

```
{MSTRUCT_FORMAT_VERSION}1
{MSTRUCT_PROFILE_ID}org.mstruct.official/rulesets/1.0.0
{MSTRUCT_PROFILE_HASH}sha256:<64 lowercase hexadecimal digits>
{AUTHORING_APP_VERSION}<application version>
```

1.9 A complete rule bundle

Manifest field

```
manifestVersion = 2
```

A rule bundle contains a manifest and configuration files.
The manifest lists twelve CSV files in a fixed order.
Ten tables define musical semantics: special-chords; roman-basic, roman-advanced, roman-figured-bass and roman-special; tags, cadence-tags, tonal-tags, sequence-tags and form-tags.
The two authoring tables, toolbox-menus and toolbox-events, define the input toolbox.
Every filename has the .csv extension.
Validation checks the complete file set, each table and the references between tables.
Older manifest-v1 bundles retain their original eleven-file framing.

1.10 Content hash and integrity

Value format

```
sha256: + 64 lowercase hexadecimal digits
```

A bundle hash uses SHA-256, written as sha256: followed by 64 lowercase hexadecimal digits.
CSV normalization removes an initial BOM, converts line endings to LF and retains exactly one final newline.
Row order is preserved.
The hash is calculated from the canonical framing of the manifest and file contents.
Loading recomputes the hash and checks it against the declared value.
It identifies content for integrity checks, caching and exact historical reading.
Profile IDs and versions are separate identity information: identical content can have different identities, and a compatible version series can contain several content hashes.
A different hash alone does not establish incompatibility.

1.11 Same-version compatibility

A rule version identifies a backward-compatible series.
Additions and display or ordering changes can remain in that series while preserving existing definitions and behavior.
Compatibility considers musical semantics and authoring behavior separately.
Deleting or redefining an existing rule requires a new major version.
Under this contract, a verified complete bundle with the same document format and complete profile ID, including its version, can be used for editing.

1.12 Obtaining rule bundles

Different providers can distribute their own rule bundles and versions.
MStruct Archive supplies its rule catalog at https://mstruct-archive.com/api/mstruct-profiles.
Its synchronization index is https://mstruct-archive.com/api/mstruct-profiles?index=1.
An exact build can be requested with the profileId and hash query parameters.
The editor can synchronize the catalog and download bundles from this service, subject to its current access policy.
Downloaded and imported bundles pass the same content and integrity checks before use.
A request specifying profileId and hash returns the complete bundle as JSON.

1.13 Incompatible rule versions

An incompatible change to an existing definition or behavior requires a new major rule version.
Adopting it in an existing document requires explicit migration.
For searching, compatible musical meanings can be grouped into a semantic family.
Semantic incompatibility starts a new family; a difference confined to authoring behavior can retain the same musical family.

1.14 Compatible updates when editing

An editor can adopt an available verified same-version rule bundle in its working copy.
This updates the working rule identity while preserving the analysis source.
Saving records the selected hash.
A local standalone editor can use an installed or embedded complete bundle without downloading the original exact build.
The selected bundle remains in use for the editing session.

1.15 Exact historical interpretation

A saved profile ID and hash identify the rules used for a particular document revision.
Retaining the corresponding bundle makes that interpretation reproducible.
Selecting compatible rules for editing and reading a saved revision with its exact bundle serve different purposes.

1.16 Offline rules and application capability

A rule bundle supplies configuration data; application updates supply new compiler syntax.
A newly introduced syntax feature also requires an application that implements it.
Standalone packages include a complete verified rule bundle for offline operation.
Other verified bundles can be installed or imported.
If no local bundle matches the document format and complete profile ID, obtain a matching bundle before editing.

1.17 Rules and derived search indexes

A search index is derived from an analysis revision and its exact rule identity.
Its data format has an implementation version separate from the rule version.
Changing the index format can require rebuilding the index while retaining the analysis and its rule identity.
Semantic queries can be interpreted within compatible rule families and matched against each analysis's own index.

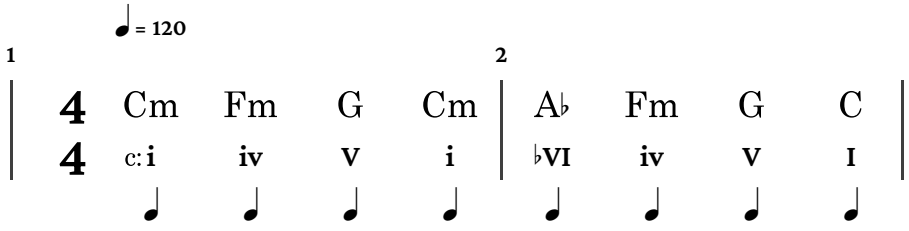
2 Basic syntax

2.1 A complete MAIN example

BN sets the measure number, and TS sets the time signature.
D specifies each slot duration as a fraction of a whole note.
TC sets the tonal reference, and BPM sets the numeric playback tempo.
The example begins at measure 1 in 4/4, uses C minor as its reference and plays 120 quarter notes per minute.
The shorthand setting `{1/4}` gives each slot a quarter-note duration.
A comma advances one slot.
A semicolon closes the last slot and the measure.
The vertical bar separates the C and R fields, so Cm|i gives the chord Cm and the Roman analysis i.
The following entries explain each setting and the available field forms.

Examples

```
{BN:1}{TS:4/4}{BPM:1/4=120}{TC:c}
{1/4}
Cm|i,Fm|iv,G|V,Cm|i;
Ab|bVI,Fm|iv,G|V,C|I;
```



2.2 BN: bar number

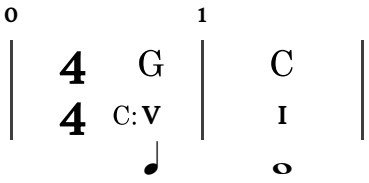
Syntax

```
{BN:n}
```

BN stands for Bar Number.
BN must be defined before the first event.
BN takes a nonnegative decimal integer and sets the current source measure number.
Subsequent ordinary measures advance the number automatically.
A later BN setting belongs at a measure boundary.
BN 0 denotes a pickup measure.
On its initial traversal, the pickup must accumulate strictly less duration than the numeric TS.

Examples

```
{BN:0}{TS:4/4}{D:1/4}{TC:C}G|V;
{D:1}C|I;
```



2.3 TS: time signature

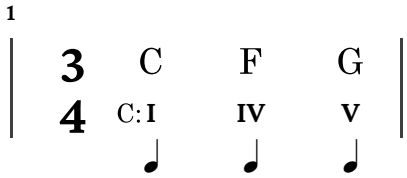
Syntax

```
{TS:p/q} / {TS:adlib}
```

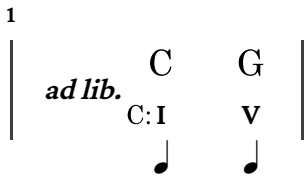
TS stands for Time Signature.
TS must be defined at the beginning, including for BN 0.
The numerator and denominator accept any positive integers.
An ordinary metered measure must total exactly p/q whole notes.
TS changes belong at measure boundaries.
The exact lowercase value `adlib` allows measures of variable total duration.
An adlib pickup also follows this variable-duration rule.
D remains mandatory and determines each slot duration in an adlib passage.

Examples

```
{BN:1}{TS:3/4}{D:1/4}{TC:C}C|I,F|IV,G|V;
```




```
{BN:1}{TS:adlib}{D:1/4}{TC:C}C|I,G|V;
```



2.4 BPM: metronome setting

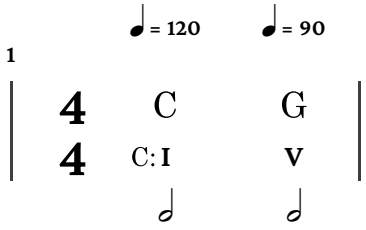
Syntax

```
{BPM:p/q=v}
```

BPM stands for Beats Per Minute.
The beat unit p/q is a positive rational duration measured in whole notes.
v is a finite positive decimal number giving the number of beats per minute.
For example, 1/4=120 means 120 quarter notes per minute.
The tempo setting persists until a later tempo change.
A BPM setting applies to its entire slot and can be written before or after the harmonic event in that slot.
A slot lasting d whole notes takes `60000*d/((p/q)*v)` milliseconds.
The stable-tempo option (`{IS_STABLE_TEMPO}True` in metadata) calculates playback time from slot durations and numeric tempo settings.
Each event slot included in this playback must therefore have a defined numeric tempo.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}{BPM:1/4=120}C|I,{BPM:1/4=90}G|V;
```



2.5 BPMhidden: hidden metronome setting

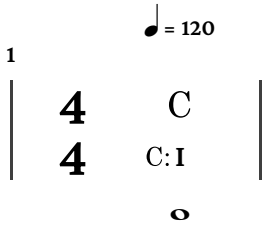
Syntax

```
{BPMhidden:p/q=v}
```

BPMhidden sets the playback tempo using the same numeric syntax as BPM.
The score displays a metronome label for BPM and hides it for BPMhidden.
Both settings change the current playback tempo.
Settings within one slot apply from left to right.
The final value drives playback, while an earlier BPM label can remain visible.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}{BPM:1/4=120}{BPMhidden:1/4=119.5}C|I;
```



The example above plays at 119.5 quarter notes per minute in preview, while the visible label remains 120.

2.6 TEMPO: verbal tempo marking

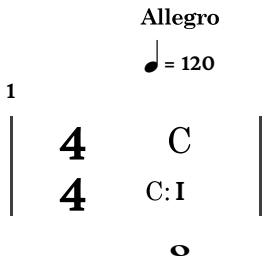
Syntax

```
{TEMPO:text}
```

TEMPO attaches a nonempty text marking above the BPM row.
A marking such as Allegro supplies the verbal indication.
A stable-tempo file requires a separate numeric BPM or BPMhidden setting.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}{TEMPO:Allegro}{BPM:1/4=120}C|I;
```



2.7 TC: tonal reference

Syntax

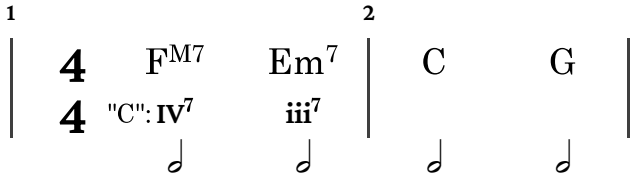
```
{TC:C} / {TC:c} / {TC:"C"} / {TC:None}
```

TC stands for Tonal Center.
An ordinary TC uses A-G for major or a-g for minor.
The note letter can be followed by a single b or #.
Matched ASCII double quotes mark an ambiguous center or a center used only as a reference.
The quoted center retains its major or minor mode for Roman interpretation.

The reference is initially undefined.
Once established, it persists across subsequent events until a setting changes it.
A later TC changes the reference from that event boundary onward.
The exact value `None` explicitly clears the reference.
Roman fields can resume after a valid tonal reference has been established again.
The modulation entries in Chapter 3 explain their reference requirements.
A slot using a Roman field must have a defined tonal reference other than None.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:"C"}FM7|IV7,Em7|iii7;
{TC:None}C|,G|;
```



2.8 D: slot duration

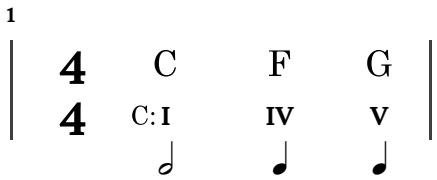
Syntax

```
{D:p/q} / {p/q} / {n}
```

D stands for Division.
D expresses slot duration as a positive integer or positive fraction of a whole note.
`{1/4}` abbreviates `{D:1/4}`.
`{1}` gives a duration of one whole note.
D must be defined before the first slot.
Its value persists across commas and ordinary barlines.
A later D changes the current and following slots.
For example, D 1/8 gives each slot an eighth-note duration under any TS.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,{1/4}F|IV,G|V;
```



2.9 Comma, semicolon and empty slots

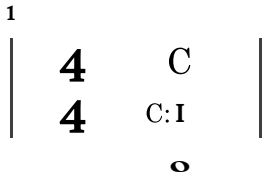
Syntax

```
, / ;
```

A comma closes the current slot using its D.
A semicolon closes the current slot and the measure.
Newlines act as whitespace between tokens.
An ordinary final measure also requires a semicolon.
An empty comma-separated slot continues the preceding event for another D.
Separators inside a matched field list divide the items in that field.
A comma immediately before a semicolon creates an additional slot.

Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I,,,;
```



2.10 Harmonic event fields

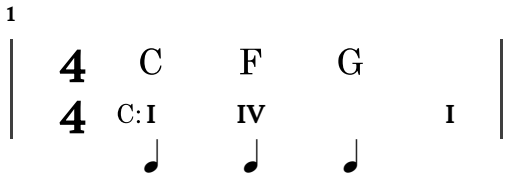
Syntax

```
[C:chord|R:roman] / chord|roman / chord| / |roman
```

The explicit form contains a C field, an R field, or both.
Each included field requires a nonempty value.
Every shorthand event requires a vertical bar.
C| supplies only the chord field, while |I supplies only the Roman field.
`[C:C]` and `[R:I]` are valid explicit forms for those single fields.
Roman requires a tonal reference.
When both fields are present, each gives an independently interpreted analysis of the same position.

Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}[C:C|R:I],F|IV,[C:G],[R:I];
```



3 Additional event syntax

3.1 X: unspecified event

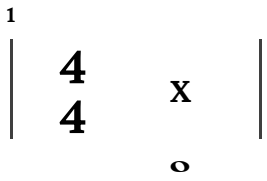
Syntax

```
X
```

Bare uppercase X is a complete placeholder event.
It reserves a slot where the analyst leaves the harmony unspecified.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
X;
```



3.2 0: rest

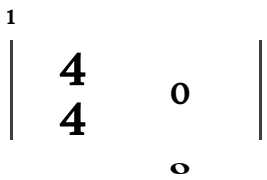
Syntax

```
0
```

Bare zero represents a rest.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
0;
```



3.3 -: continuation

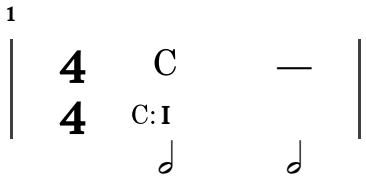
Syntax

```
-
```

A bare ASCII hyphen continues the preceding musical content for the current slot.
The bare form continues the whole event, while C:- and R:- mark individual fields.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,-;
```



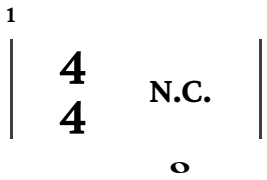
3.4 N.C.: no chord

Syntax

```
N.C.
```

N.C. records a no-chord event.
Its exact spelling includes both full stops.
The current tonal reference remains in effect.
Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
N.C.;
```



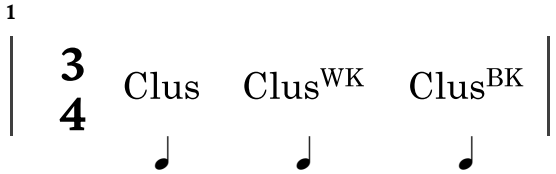
3.5 Clus, ClusWK and ClusBK

Syntax

```
Clus / ClusWK / ClusBK
```

Clus is the general event for a tone cluster of any type.
In particular, ClusWK represents a white-key cluster, and ClusBK represents a black-key cluster.
These events classify the cluster without specifying a root or complete pitch list.
Their spellings require the letter case shown here.
Examples

```
{BN:1}{TS:3/4}{D:1/4}{TC:None}Clus,ClusWK,ClusBK;
```



3.6 Field-level X and -

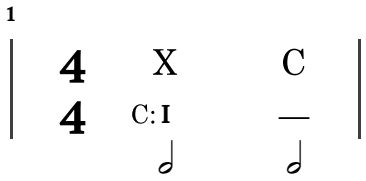
Syntax

```
[C:X|R:I] / [C:C|R:-]
```

X or - as the complete value of C or R marks that individual field.
C:- continues the preceding chord field, and R:- continues the preceding Roman field.
In particular, R:- also preserves an active figured-bass chain, defined in Chapter 5.
R:X leaves the Roman field unspecified; any active figured-bass chain must have ended before it.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}[C:X|R:I],[C:C|R:-];
```



3.7 Secondary harmonic event

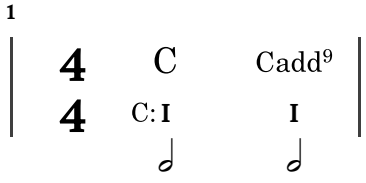
Syntax

```
'C|I' / '[C:C|R:I]'
```

A matched pair of ASCII single quotes marks a harmonic event as secondary.
The quotes must enclose the complete event.
Every included field must contain a chord or Roman symbol.
Secondary events usually record small changes in the momentary vertical harmony caused by melodic or inner-voice motion while the root or basic chord type remains the same.
Search skips them by default; `--notignore2nd` includes them.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,'Cadd9|I';
```



3.8 Alternative analysis in the same domain

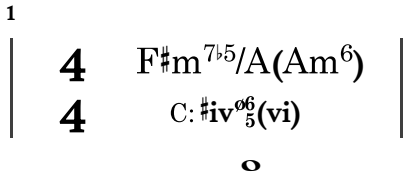
Syntax

```
C:C\\Am / R:I\\vi
```

Exactly two consecutive backslashes add one complete alternative interpretation to a field.
Both sides require nonempty valid symbols in the same domain.
Field markers cannot carry an alternative interpretation.
Each symbol receives an independent interpretation.
The figured-bass chain follows the main Roman symbol.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F#m7b5/A\\Am6|#ivho65\\vi;
```



3.9 NR: transformation annotation

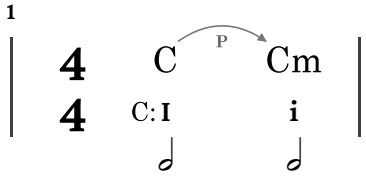
Syntax

```
[NR:P] / [NR:PLR] / [NR:N]
```

NR abbreviates Neo-Riemannian Transformation.
NR records an analytical claim about a transformation between adjacent chords.
Its name requires uppercase letters.
Its value is a string of 1-6 characters chosen from P, L and R, or a single N, S or H.
NR belongs before the C chord symbol in the same slot.
Each slot allows at most one NR.
The preceding musical event must contain a C chord symbol or be a bare continuation.
A field marker alone cannot supply a chord endpoint.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,[NR:P]Cm|i;
```



3.10 M: pivot modulation

Syntax

```
{M:target|relation} ... {/M}
```

M stands for Modulation.
M requires an established tonal reference and opens a modulation from it to the target.
The relation describes the target relative to the source key using the basic Roman notation defined in Chapter 5.
The tonic pitch and major or minor quality represented by relation must agree with target.
A pivot is a chord interpreted in both the source key and the target key.
Its R field gives two layers of analysis, one for each key.
Each pivot event requires a C chord symbol and a two-layer R analysis.
{/M} closes the pivot range.
The range must close before another modulation, a conflicting TC change or an omitted passage.
Chapter 5 gives a dual-analysis example.

3.11 MH: direct modulation

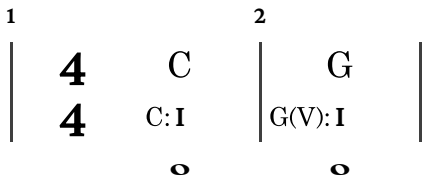
Syntax

```
{MH:target|relation}
```

MH abbreviates Modulation (Hard).
MH uses the same target and relation syntax as M.
It requires a valid source tonal reference.
It changes the reference directly at the next eligible harmonic event.
MH is a single setting and requires no closing marker.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}C|I;
{MH:G|V}G|I;
```



3.12 MM: metric modulation

Syntax

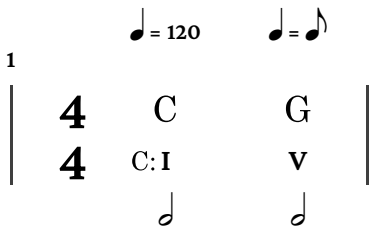
```
{MM:old=new}
```

MM stands for Metric Modulation.
Both sides specify positive fractional beat durations.
The relation states that the duration of the old note value equals that of the new note value

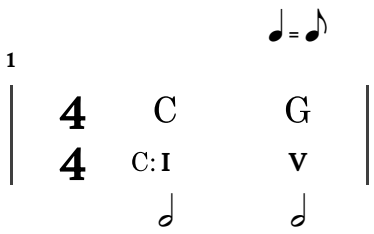
across the change.
MM can record this relationship without a preceding BPM or BPMhidden setting.
When a numeric tempo exists, its beat duration is multiplied by new/old while the numeric beat count is retained.
For example, 1/4=120 followed by MM:1/4=1/8 gives 1/8=120, equivalent to 60 quarter notes per minute.
The left-hand duration may differ from the current BPM beat unit; the same ratio still applies.
Without a numeric tempo, MM records the relation without assigning a numeric playback speed.
A later BPM or BPMhidden establishes a new explicit tempo.
Stable-tempo playback still requires a numeric tempo for every event slot.

Examples

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}{BPM:1/4=120}C|I,{MM:1/4=1/8}G|V;
```




```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,{MM:1/4=1/8}G|V;
```



3.13 T: timestamp label

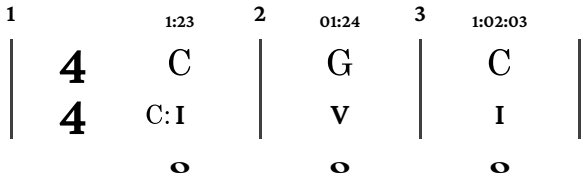
Syntax

```
{T=m:ss} / {T=mm:ss} / {T=h:mm:ss} / {T:m:ss}
```

T stands for Timestamp and attaches a time label for reading.
It accepts m:ss, mm:ss and h:mm:ss, with any number of digits in the first component.
In a two-component value, the first component gives nonnegative minutes and the second gives seconds from 00 to 59.
In a three-component value, hours are nonnegative and both minutes and seconds use two digits from 00 to 59.
T accepts either = or : before the value and preserves the entered text.
Labels may increase or decrease in source order.
T changes neither playback timing nor the duration of an omitted passage.
Recording synchronization uses the MStructTime timestamps described in Chapter 7.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}{T=1:23}C|I;{T=01:24}G|V;{T=1:02:03}C|I;
```



3.14 Ellipsis and resumed analysis

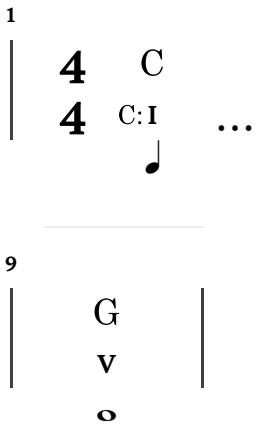
Syntax

```
... / {BN:n}
```

Three full stops mark an omitted passage.
An ellipsis can truncate an incomplete measure.
The next resumed measure must explicitly define BN.
D and TS retain their existing values until changed.
The omitted passage has no inferred duration in the stable-tempo playback plan.
A T label at the resumed passage remains a reading aid.

Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I...
{BN:9}{D:1}G|V;
```



3.15 LB: system break

Syntax

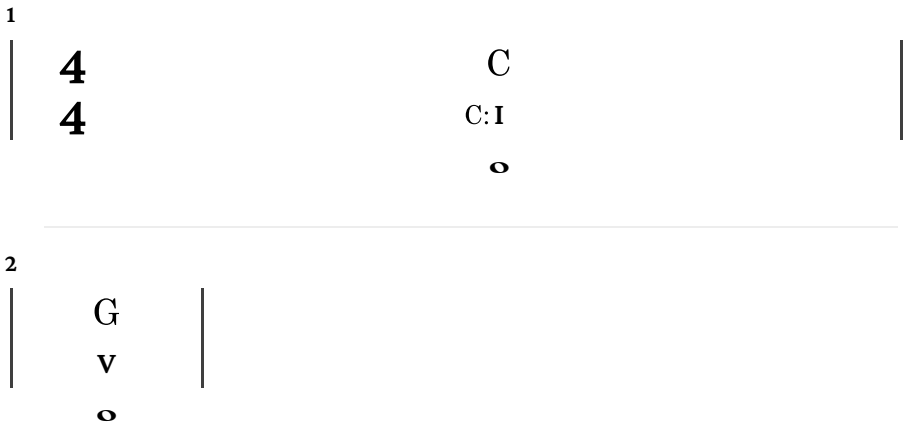
```
{LB}
```

LB stands for Line Break.
LB requests a new system after a complete measure.

It belongs after the semicolon that closes that measure.
D, TC and playback time continue across the system break.
The score also wraps automatically when it fills the available width.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;{LB}G|V;
```



3.16 Double barline

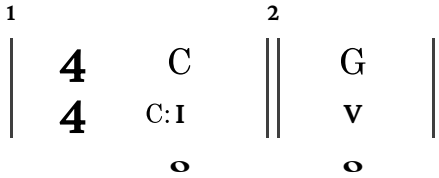
Syntax

```
{||}
```

{||} displays a double barline at a slot boundary.
The semicolon still closes the measure after its required duration.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I{||};G|V;
```



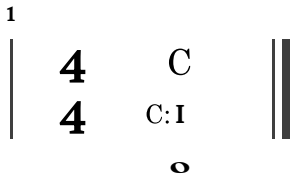
3.17 Final barline

Syntax

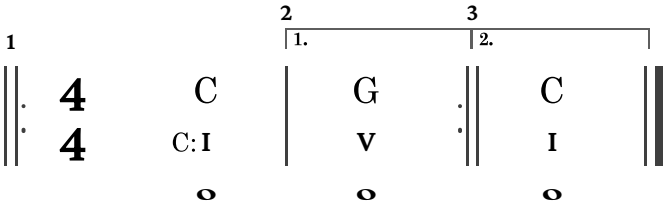
```
{||END}
```

{||END} is a zero-duration final barline that ends MAIN.
It occurs in the final measure before its semicolon, with only whitespace between the marker and semicolon.
After the semicolon, MAIN contains only whitespace.
The barline is thin on the left and thick on the right.
The last slot may be empty, extending the preceding event by its D.
At the same boundary, {||END} is mutually exclusive with a double barline or either repeat sign.
A final ending may close with {/ENDING} immediately before {||END}.
The measure must still pass the duration checks for the current meter and actual performance route.
Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I,,,{||END};
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;
{ENDING:1}G|V{/ENDING:1}{:||};
{ENDING:2}C|I{/ENDING}{||END};
```



3.18 Repeat signs and repeat count

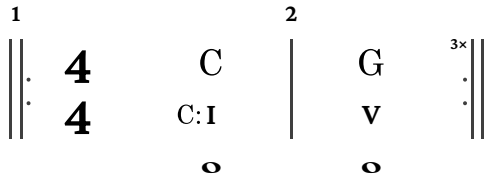
Syntax

```
{||:} / {:||} / {:||Nx}
```

The left sign opens a repeated region, and the right sign closes it.
An ordinary right repeat gives two total traversals.
An explicit Nx gives N total traversals, with N from 2 to 1000.
Omitting the left sign uses the beginning of MAIN as the implicit repeat start.
Each repeated region must close before another begins.
A jump restores the duration, meter and tempo state saved at its target.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;G|V{:||3x};
```



3.19 Different endings

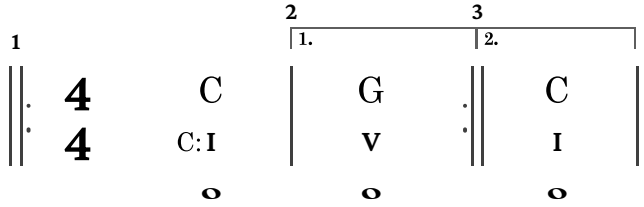
Syntax

```
{ENDING:1,2} ... {/ENDING}
```

An ending selector lists the passes that use that ending.
The list uses distinct increasing integers from 1 to 1000, separated by commas.
These commas separate pass numbers within the control marker.
The endings must cover the required passes and use valid repeat-sign positions.
A numbered closing marker, such as {/ENDING:1}, closes a nonfinal ending and must repeat its opening list exactly.
The unnumbered {/ENDING} closes the final ending and completes the ending group.
For example, pass 1 uses {ENDING:1} ... {/ENDING:1} before the repeat sign; pass 2 uses {ENDING:2} ... {/ENDING} to finish the group.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;
{ENDING:1}G|V{/ENDING:1}{:||};
{ENDING:2}C|I{/ENDING};
```



3.20 Continuous displayed measure numbering

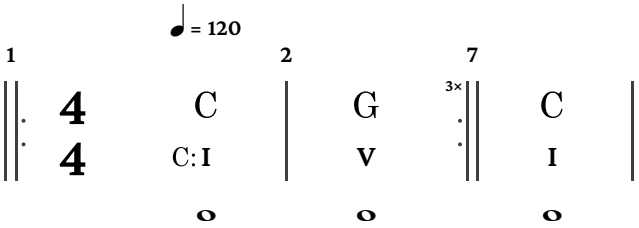
Syntax

```
{:||>>} / {:||Nx>>}
```

The >> flag requires a stable-tempo file.
It displays cumulative measure numbers across repeat traversals.
The source retains one copy of each measure.
In the example, two measures are repeated three times, and the following measure is displayed as measure 7.

Examples

```
METADATA:
{IS_STABLE_TEMPO}True
{OFFSET}0
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}{BPM:1/4=120}{||:}C|I;G|V{:||3x>>};C|I;
END_MAIN
```



3.21 F: a local form symbol

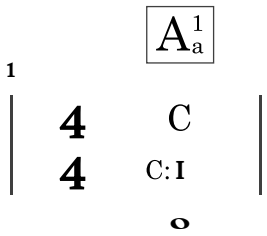
Syntax

```
[F:body|sup|sub]
```

F attaches a printed form symbol to the musical event in its slot.
The body requires a value, while the superscript and subscript may be empty.
All three parts accept printable ASCII characters except [,] and |.
Each slot allows at most one F.
F requires a musical event in the same slot.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
[F:A|1|a]C|I;
```



3.22 COMMENT references and ranges

Syntax

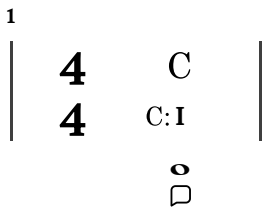
```
[COMMENT:n] ... [/COMMENT] / [/COMMENT:n]
```

The document COMMENT section must define the referenced number.
[COMMENT:n] begins a comment range.
[/COMMENT] ends the active range.
[/COMMENT:n] attaches a comment reference to one position.
Only one range can be active at a time.
A change within one slot requires the old range to close before the new range opens.
A definition can have Simplified Chinese (zh_cn) and English (en) versions.
Each identifier must be unique within its language.
A referenced position displays a comment icon; the interactive score opens the comment from that icon.

Examples

```
COMMENT:
[COMMENT:1<en>]
A tonic arrival.
[/COMMENT:1<en>]
[COMMENT:1<zh_cn>]
主和弦落点。
[/COMMENT:1<zh_cn>]
END_COMMENT


METADATA:
{IS_STABLE_TEMPO}False
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}[/COMMENT:1]C|I;
END_MAIN
COMMENT:
[COMMENT:1<en>]
A tonic arrival.
[/COMMENT:1<en>]
[COMMENT:1<zh_cn>]
主和弦落点。
[/COMMENT:1<zh_cn>]
END_COMMENT
```



4 Chord symbols

4.1 Suffix order and structure validity

Syntax

```
root quality M extension altered sus add no /bass on lower
```

Standard chord suffixes follow the order shown above.
Each optional group occurs at most once and follows its own quality restrictions.
The final interval structure must contain root interval 1.
A chord without on must contain between 1 and 12 pitches with distinct pitch classes.
Duplicate intervals and enharmonically equivalent intervals, including octave equivalents, cause structure errors in that chord.
The dedicated forms below define their own permitted suffixes.
The following entries define these components and their restrictions.

Examples

`C7(b9)sus4add#11no5` $C^{7\flat 9}\text{sus}^{4}\text{add}^{\sharp 11}\text{no}^{5}$

4.2 Root and accidental

Syntax

```
A-G + bb / b / # / x
```

A chord root (A-G) uses one uppercase note letter.
The letter can be followed by one accidental token: bb for a double flat, b for a flat, # for a sharp or x for a double sharp.
A natural root consists of the letter alone.
A pitch class groups pitches separated by whole octaves.
The parser preserves the root spelling when different spellings have the same pitch class.
Lowercase roots, leading accidentals and Unicode accidental signs are invalid input.

Examples

`C#` $C^{\sharp}$

`Bbb` $B^{\flat\flat}$

`Fx` $F^{\times}$

4.3 Triad quality

Syntax

```
root / rootm / rootaug / rootdim
```

Chord intervals (such as 1,b3,5) are measured from the root and use major-scale interval names.
For example, 1 is the root, 3 is a major third, b3 is a minor third and 5 is a perfect fifth.
The default major triad contains 1,3,5.

The quality suffix m gives 1,b3,5.
The suffix aug gives 1,3,#5.
The suffix dim gives 1,b3,b5.
Quality tokens are case sensitive.
The aliases maj/min are invalid input.
Symbol display preferences select how these qualities appear while preserving the source and interval structure.

Examples

`C` C

`Cm` Cm

`Caug` Caug

`Cdim` Cdim

4.4 Sevenths and major sevenths

Syntax

```
7 / M7 / m7 / mM7 / dim7 / dimM7
```

The suffix 7 adds b7 to a major, minor or augmented triad.
For diminished quality, 7 adds bb7.
M selects natural 7 and must precede 7,9,11 or 13.
The numbers 9,11,13 specify the highest extension, as defined in the next entry.
C7, CM7, Cm7, CmM7 and Cdim7 therefore specify different structures.
A bare CM is invalid.
The spelling m7b5 combines a minor seventh chord with an altered fifth, whose syntax is defined under altered tensions.

Examples

`C7` C^{7}

`CM7` C^{M7}

`CmM7` Cm^{M7}

`Cdim7` $Cdim^{7}$

`Cm7b5` $Cm^{7\flat5}$

4.5 Highest extension

Syntax

```
9 / 11 / 13
```

The highest-extension suffix 9 adds the seventh and ninth.
The suffix 11 adds the seventh, ninth and eleventh.
The suffix 13 adds the seventh, ninth, eleventh and thirteenth.
The seventh follows the quality and M rules above.
The generated structure includes every listed interval, including the eleventh in a thirteenth chord.
The altered-tension syntax below can replace permitted members of this structure.

Examples

`C9` C^{9}

`CM11` $\mathrm{C}^{\mathrm{M11}}$

`Cm13` Cm^{13}

4.6 Sixth-chord series

Syntax

```
6 / (b6) / 6/9 / 6/b9 / (b6/9) / (b6/b9)
```

A sixth-series suffix occupies the same position as the highest extension.
Choosing it replaces the seventh/extension group, so it cannot be combined with a separate highest extension.
The sixth-series suffixes add the indicated sixth and, where written, ninth.
These suffixes generate no seventh.
They accept major, minor and augmented triad qualities.
Flattened-sixth forms require the parentheses shown here.
The M modifier and the sus suffix defined below are invalid in the sixth series.

Examples

`C6` C^{6}

`Cm6/9` $\mathrm{Cm}^{6/9}$

`C(b6/b9)` $\mathrm{C}^{\flat 6/\flat 9}$

4.7 Altered tensions

Syntax

```
(b5,#5,b9,#9,b11,#11,b13,#13,b15,#15)
```

Altered tensions use a comma-separated parenthesized list after 7,9,11 or 13.
The available degrees are 5,9,11,13,15, each with b or #.
Items follow ascending degree order, with b before # when both occur at one degree.
An alteration replaces an existing unaltered member only when that member was supplied implicitly.
It adds the specified altered member when no unaltered member is present.
The explicitly written highest extension retains its manual-origin status, as defined under no below.
Thus C9(b9), C11(#11) and C13(b13) are invalid replacements of an explicit highest extension.
Augmented and diminished qualities reject altered fifths in this list.
The parser also accepts the legacy adjacent form and displays it in the same way as the parenthesized form.

Examples

`C13(b5,b9,#9)` $C^{13\flat5\flat9\sharp9}$

`C13b5b9#9` $C^{13\flat5\flat9\sharp9}$

4.8 sus: remove the third and add suspensions

Syntax

```
sus / sus2 / sus4 / sus(b2,2,4,#4)
```

The sus suffix removes the third supplied by the chord quality.
A bare sus adds no replacement interval.
An optional interval or parenthesized list adds members selected from b2,2,4,#4 in ascending order.
The suffix accepts major and augmented qualities outside the sixth series.
Cmsus4 is invalid.
The suffix defines the current chord structure; a later resolution is a separate analytical choice.

Examples

`Csus` Csus

`Csus4` Csus^{4}

`C7sus(2,4)` $\mathrm{C}^{7}\mathrm{sus}^{(2,4)}$

4.9 add: explicit added intervals

Syntax

```
add9 / add(2,#4)
```

The add suffix adds one explicit interval or a parenthesized ascending list.
Its permitted intervals are b2,2,b3,3,4,#4,b5,5,#5,b6,6,b7,7,b8,b9,9,#9,b10,10,11,#11,b12,12, b13,13,#13,b15,#15.
For example, add9 adds the ninth alone, with no seventh.
Intervals containing x are invalid in add.
The resulting structure must pass the duplicate-pitch and enharmonic-conflict checks described under structure validity below.

Examples

`Cadd9` Cadd^{9}

`Cadd(2,#4)` Cadd$^{(2,\sharp 4)}$

`Edim7addb8onE` Edim^{7}add$^{\flat 8}$*on*E

4.10 no: remove an exact naturally supplied interval

Syntax

```
no3 / no(b3,5)
```

The no suffix removes an interval recorded with natural-origin status in the generated structure.
This status records how the parser supplied an interval, including an altered interval such as b3.
The permitted items are b3,3,b5,5, in ascending degree order with flats first.
A single item is written directly after no, while multiple items require parentheses.
Removal requires an exact match of accidental, degree and natural-origin status.
The parser marks the b3 in Cm as manual, so Cmnob3 is invalid.
For a plain diminished triad, b3 has natural-origin status and b5 has manual-origin status.
For a diminished seventh chord or higher extension, both have natural-origin status.
Therefore Cdimnob5 is invalid and Cdim7nob5 is valid.

Examples

`Cno3` Cno^{3}

`Cno(3,5)` Cno$^{(3,5)}$

`Cdim7nob5` Cdim^{7}no$^{\flat 5}$

4.11 Slash bass

Syntax

```
upper/bass
```

A slash bass (/bass) specifies an absolute bass note after the chord suffixes.
The bass follows the root-spelling rules above.
The parser recalculates the interval structure relative to that bass.
For example, C/E has the intervals 1,b6,b10 relative to E.
Each chord accepts at most one slash bass.

Examples

`C/E` C/E

`Baug/F` Baug/F

4.12 on: polychord

Syntax

```
upper on lower
```

The on operator combines an upper chord and a lower chord into a polychord.
Source joins the two complete chord symbols with on and no spaces, as in CM7onDm.
Each side can have its own slash bass.
The lower chord must contain exactly three notes in its interval structure.
A second on inside the lower chord is invalid.
The final structure merges both sides relative to the lower root or bass and preserves compound-interval spelling.

Examples

`CM7onDm` $\mathrm{C^{M7}}\mathit{on}\mathrm{Dm}$

`C/GonF/A` C/G*on*F/A

4.13 1 and 5: terminal pitched forms

Syntax

```
C1 / C5
```

A root followed by 1 gives only interval 1.
A root followed by 5 gives intervals 1 and 5.
Both forms end the chord symbol and reject further suffixes, including sevenths, add, slash bass and on.
The root can carry an accidental.

Examples

`C1` C^{1}

`C5` C^{5}

`F#5` $F^{\sharp 5}$

4.14 q4: stacked fourths

Syntax

```
C[q4:(4,b7,b10)]
```

The q4 form supplies root interval 1 and takes one to three intervals measured from that root.
The list selects one member from each consecutive tier: 4/#4, then 7/b7, then 10/b10.
For example, 4,b7,b10 describes three successive fourths by their cumulative intervals above the root.
The form accepts add, slash bass and on.
It rejects sus and no.

Examples

`C[q4:(4,b7,b10)]` $Cq4^{(4,\flat 7,\flat 10)}$

4.15 q5: stacked fifths

Syntax

```
C[q5:(5,9,13)]
```

The q5 form supplies root interval 1 and takes one to three intervals measured from that root.
The list selects one member from each consecutive tier: 5/b5, then 9/b9, then 13/b13.
The tiers must begin with the first tier and follow this order.
The form accepts add, slash bass and on, and rejects sus and no.

Examples

`C[q5:(5,9,13)]` $Cq5^{(5,9,13)}$

4.16 nf: explicit semitone offsets

Syntax

```
C[nf:(0,4,7)]
```

nf stands for Normal Form.
The nf form specifies semitone offsets above the named root.
Its list contains at least two distinct integers from 0 to 36 in strictly ascending order.
The offsets generate the interval structure directly.
The resulting structure must pass the structure checks above, including the presence of root interval 1.
This form ends the chord symbol and rejects further suffixes.

Examples

`C[nf:(0,4,7)]` $C^{[0,4,7]}$

4.17 sp: configured named structure

Syntax

```
C[sp:sowhat]
```

The sp form selects a named structure from special-chords.csv.
The name after sp: must match a compiler name in that table.
The table supplies its exact interval structure and display name.
The current built-in names are sowhat (1,4,b7,b10,12), blk (1,#4,#6,9) and mys (1,#4,b7,10, 13,16).
The form accepts slash bass and on, and rejects sus, add and no.

Examples

`C[sp:sowhat]` C^{sowhat}

`C[sp:blk]` C^{blk}

`C[sp:mys]` C^{mys}

4.18 7alt: retained source notation

Syntax

```
C7alt / C7alt/G
```

The exact token 7alt preserves an original symbol whose altered pitches remain unspecified.
It constructs the skeleton 1,3,b7.
Only an optional slash bass can follow 7alt.
The form rejects add, sus, no and on, and cannot appear as the lower chord after on.
A new analysis should write the altered pitches explicitly when they are known.

Examples

`C7alt` $C^{7}\text{alt}$

`C7alt/G` $C^{7}\text{alt/G}$

5 Roman numerals

5.1 Reference and separate grammar

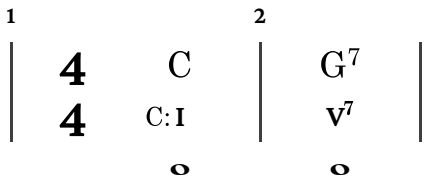
Syntax

```
{TC:C} ... |I
```

A Roman symbol specifies scale-degree content relative to the current tonal reference.
It inherits the major or minor mode of that reference, including a quoted reference.
Four Roman configuration tables define the available basic symbols, quality modifiers, figured bass and special symbols.
The following entries define each component and its permitted combinations.
Roman fields and their annotations require a tonal reference, as defined in Chapter 2.
In MStruct, the chord symbol and Roman numeral describe the same musical event and complement each other.
The Roman field therefore usually records harmonic degree, seventh-chord quality and inversion; the chord field supplies tensions such as 9, 11 and 13.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;G7|V7;
```



5.2 Bases, case and accidentals

Syntax

```
I / i / bIII / nIII
```

A basic Roman symbol must match a token in roman-basic.csv.
Its scale degree uses major-scale interval names relative to the tonal reference.
Lowercase symbols have minor-triad quality.
Uppercase symbols have major-triad quality, except V, which has the configured dominant quality.
III/iii and VI/vi are available only in major mode.
nIII/niii and nVI/nvi are available only in minor mode.
In either mode, b denotes a flattened major-scale degree and n denotes the corresponding unaltered degree.
Thus bIII in minor names b3, while nIII names natural 3.

Examples

`bIII` ♭**III**

`nIII` ♮**III**

`III` **III**

5.3 Compact figured bass

Syntax

```
6 / 64 / 7 / 65 / 43 / 42
```

Compact figured bass is a suffix that selects an inversion and, where specified, adds a seventh.
The suffixes 6 and 64 represent the first and second inversion of a triad.
The suffix 7 adds a seventh in root position.
The suffixes 65,43,42 add a seventh and select the first, second and third inversion respectively.
The configured quality and tonal reference determine the seventh unless a quality modifier or special symbol supplies an override.
For example, I7 in major contains natural 7, while IDM7 uses b7.
The next entry defines DM and the other quality modifiers, including their quality restrictions and recommendations.

Examples

`I6` $\mathbf{I^6}$

`I64` $\mathbf{I^6_4}$

`I7` $\mathbf{I^7}$

`IDM7` $\mathbf{I^{DM7}}$

`V65` $\mathbf{V^6_5}$

5.4 Advanced quality modifiers

Syntax

```
+ / o / ho / DM / oM / M
```

A quality modifier (+,o,ho,DM,oM,M) appears after the basic symbol and before figured bass.
The modifier + replaces the fifth with #5.
The modifier o replaces the fifth with b5 and selects bb7 when the symbol contains a seventh.
The modifier ho supplies b5 and b7 and requires a seventh.
DM selects b7.
oM selects b5 and natural 7.
M selects natural 7.
These last three modifiers require a figured-bass suffix that adds a seventh.

The modifiers + and DM require a major or dominant basic quality.
The modifiers o, ho and oM require a minor basic quality.
M accepts a minor or dominant base; the special leading-tone symbols listed below can also provide a major seventh over a diminished triad.
An uppercase major base other than V rejects M because its seventh is already major, so IM7 and IVM65 are invalid.
VM7 remains valid because M changes the dominant seventh to a major seventh.
Other quality conflicts, such as Io7 or i+, are also errors.
The basic table lists recommended modifiers for each base.
A combination outside that list remains valid, with an advanced-not-recommended warning, when it satisfies all quality, seventh, nonredundancy and figured-bass requirements.
For example, iM7 is recommended, while bIIDM7 is valid with that warning.
The tables at the end of this chapter list recommendations and figured-bass requirements.

Examples

`I+` **I^{+}**

`io7` **i^{o7}**

`iiho7` **ii^{ø7}**

`iM7` **i^{M7}**

`ioM7` **i^{oM7}**

5.5 sus and altered-fifth figures

Syntax

```
sus / 7sus / 7b5 / 7b5sus
```

The figured-bass suffix sus removes the third and adds no replacement fourth.
The suffix 7sus adds a seventh and removes the third.
These sus forms require major or dominant base quality and either no quality modifier or one of +,DM,M.
Each combination must also satisfy the modifier's figured-bass whitelist.
The suffix 7b5 adds a seventh and flattens the fifth.
The suffix 7b5sus also removes the third and follows the sus restrictions.
The spellings isus and i7b5 are invalid Roman input.

Examples

`Isus` **I^{sus}**

`V7sus` **V^{7sus}**

`IDM7b5sus` **I$^{DM7sus}_{♭5}$**

5.6 Applied Roman suffixes

Syntax

```
V7/V / V7/V/V
```

An applied suffix (/target) shifts the preceding Roman structure by the configured degree of a basic Roman target.
Each target must be a basic symbol with no quality modifier or figured-bass suffix.
At most two successive targets are allowed.
The degree shifts accumulate for successive targets.
The current tonal reference supplies the major or minor mode used to check the main symbol and its targets.
Targets with the same configured degree apply the same shift, as in V7/II and V7/ii.
Eligible special symbols and general figured-bass lists, defined below, also accept applied suffixes.
The next entries explain how an applied target list is retained in a figured-bass chain.

Examples

`V7/V` **V^7/V**

`V7/V/V` **$V^7/V/V$**

`viio42/V` **vii^{o4}_{2}/V**

5.7 General figured-bass list

Syntax

```
V[9,#7,5,3-]
```

A general figured-bass list uses brackets to specify intervals above a retained bass or root reference.
It replaces the compact figured-bass suffix.
The list contains two to four items, each with a figure from 2 through 14.
Figures follow descending numeric order, allow equal numbers and can carry n,b,bb,#,x.
An unaltered figure denotes a major or perfect interval; an accidental modifies that interval.
Every repeated figure remains visible, while identical structural pitches merge.
A figured-bass chain connects successive lists under one retained reference.
A newly named general-figure symbol must end its list with - to continue the chain or -x to terminate it at that event.
Thus an independent I[5,3] is invalid; I[5,3-x] supplies an explicit termination.
The next entry defines continuation lists and chain boundaries.

Examples

`V[9,#7,5,3-x]` **$V^{9}_{\sharp 7\,5\,3}$**

`V[n6,6,6-x]` **$V^{\flat 6}_{6\,6}$**

5.8 Continuation and termination of figure chains

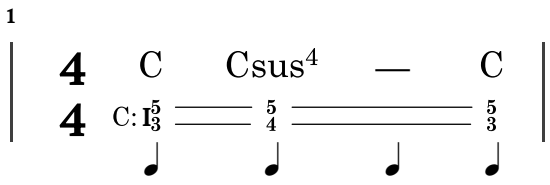
Syntax

```
I[5,3-] ... [5,4] / I[5,3-x]
```

A - after the last figure keeps the chain active.
A continuation list omits the initial Roman name, as in [5,4].
It retains the initial root reference and must use the same figure count and applied target list.
A continuation list with no final marker ends the chain.
The marker -x explicitly ends the chain at the current event.
Chord-only events, bare - and R:- can occur between lists while preserving the chain.
A new named Roman, R:X or another special event produces an interruption error while the chain is active.
A tonal-reference or modulation change also requires the active chain to have ended.
The ending -x can express an interrupted expectation of continuation.

Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}
C|I[5,3-],Csus4|[5,4-],-,C|[5,3];
```



5.9 Configured augmented-sixth symbols

Syntax

```
Ger+6 / Gero3 / Fr+6 / It+6 / Tr+6 / Blk+6
```

The special table defines each augmented-sixth symbol as a complete token with its own structure.
For example, Ger+6 has b6,1,b3,#4, and Fr+6 has b6,1,2,#4.
Gero3 also selects its corresponding note-spelling convention.
The ending +6 belongs to the listed special tokens and is invalid as a general suffix on a basic Roman symbol.

Examples

`Ger+6` **Ger^{+6}**

`Gero3` **Ger^{o3}**

`Fr+6` **Fr^{+6}**

`It+6` **It^{+6}**

`Tr+6` **Tr^{+6}**

`Blk+6` **Blk^{+6}**

5.10 VS, VSBlk and leading-tone symbols

Syntax

```
VS7 / VSBlk / viio7 / viiho7
```

S stands for Substitution.
The VS series describes a dominant-seventh harmony on flat degree II, generally used as a tritone substitute for the dominant.
VS requires one of the configured suffixes 7,65,43,42,7b5 or a general figured-bass list.
Bare VS is invalid.
VSBlk is a separate complete special token.
The special table also defines exact structures for viio, viio6, viio64, viio7, viio65, viio43, viio42, viioM7 and viiho7, viiho65, viiho43, viiho42.
Each listed leading-tone symbol is a complete token from that table.

Examples

`VS7` **V(S)^{7}**

`VSBlk` **V(S)^{Blk}**

`viio7` **vii^{o7}**

`viiho65` **vii$^{ø6}_{5}$**

5.11 Common-tone symbols

Syntax

```
cto7 / ct+6
```

The common-tone diminished-seventh symbol cto7 preserves the notation with no fixed Roman pitch structure.
It accepts an applied suffix.
The current default table contains cto7 and omits the inversion tokens cto65, cto43 and cto42.
The common-tone symbol ct+6 has the explicit structure 1,#2,#4,b6.

Examples

`cto7/V` **CT^{o7}/V**

`ct+6` **CT^{+6}**

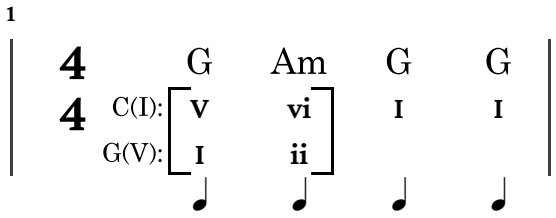
5.12 Dual Roman layers in M

Syntax

```
old&new
```

Within the M pivot range defined in Chapter 3, & separates two Roman layers for the same event.
The left layer uses the original tonal reference, and the right layer uses the target reference.
Each layer maintains its own figured-bass chain context.
In the example from C to G, G is V in C and I in G.
Am is vi in C and ii in G.
Examples

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}
{M:G|V}G|V&I,Am|vi&ii{/M},G|I,G|I;
```



5.13 Current bases and recommended modifiers

The table lists the recommended modifiers for each base.
Third quality and figured-bass rules determine validity; a valid combination outside the recommendations produces a warning.

Base	Degree / quality	Mode gate	Recommended
I	1 / M	*	+,DM
i	1 / m	*	o,ho,oM,M
bII	b2 / M	*	-
bii	b2 / m	*	M
II	2 / M	*	DM
ii	2 / m	*	o,ho,M
bIII	b3 / M	*	+,DM
biii	b3 / m	*	M
III	3 / M	M	+
iii	3 / m	M	M,ho
nIII	3 / M	m	-
niii	3 / m	m	ho
IV	4 / M	*	DM
iv	4 / m	*	ho,M
#IV	#4 / M	*	-
#iv	#4 / m	*	M,ho
bV	b5 / M	*	-
bv	b5 / m	*	M

Base	Degree / quality	Mode gate	Recommended
V	5 / D	*	+,M
v	5 / m	*	M,ho
bVI	b6 / M	*	-
bvi	b6 / m	*	M
VI	6 / M	M	-
vi	6 / m	M	M,ho
nVI	6 / M	m	-
nvi	6 / m	m	M,ho
bVII	b7 / M	*	DM
bvii	b7 / m	*	M
nVII	7 / M	*	-
nvii	7 / m	*	M

5.14 Current figures allowed for each modifier

Modifier	Needs 7th	7th override	Figures
(empty)	-	none	6,64,7,65,43,42,7b5,sus,7sus,7b5sus, []
+	-	7	6,64,7,65,43,42,sus,7sus
o	-	bb7	6,64,7,65,43,42
ho	True	b7	6,64,7,65,43,42
DM	True	b7	6,64,7,65,43,42,7b5,sus,7sus,7b5sus
oM	True	7	7,65,43,42
M	True	7	7,sus,7sus,7b5sus,65,43,42

5.15 Exact structures of special Romans

Token	Degree structure	Additional figure rule
Ger+6	b6 1 b3 #4	-
Gero3	#4 b6 1 b3	-
Fr+6	b6 1 2 #4	-
It+6	b6 1 #4	-
Tr+6	b6 7 2 #4	-
Blk+6	b6 b7 2 #4	-
VS	b2 4 b6	7,65,43,42,7b5,[] (required)
VSBlk	b2 b3 5 7	-
viio	7 2 4	-
viio6	2 4 7	-
viio64	4 7 2	-
viio7	7 2 4 b6	-
viio65	2 4 b6 7	-
viio43	4 b6 7 2	-
viio42	b6 7 2 4	-
viioM7	7 2 4 #6	-

Token	Degree structure	Additional figure rule
`viiho7`	`7 2 4 6`	-
`viiho65`	`2 4 6 7`	-
`viiho43`	`4 6 7 2`	-
`viiho42`	`6 7 2 4`	-
`cto7`	`(none)`	-
`ct+6`	`1 #2 #4 b6`	-

6 Tag events

6.1 Tag spelling and duration modes

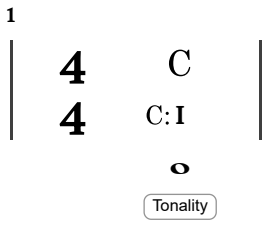
Syntax

```
<name> ... </name> / </name>
```

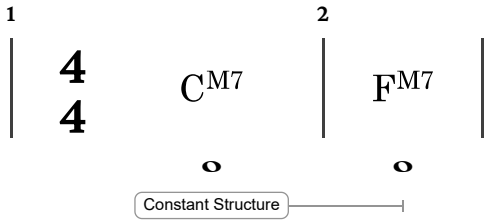
An analytical tag (`<name>` or `</name>`) names a musical feature at a slot.
The slot must also contain a musical event.
A tag range begins with an opening tag (`<name>`) and ends with its matching closing tag (`</name>`).
The range is active from its opening through its closing slot.
The ordinary-tag configuration assigns each tag a duration mode: 1, 2 or 3.
Mode 1 uses only a one-shot tag (`</name>`) that applies at one position.
Mode 2 allows a paired range, and its closing form acts as a one-shot tag when no range of that name is active.
Mode 3 requires a paired range.
Each active range must close before the same tag opens again and before the document ends.
Tag names and parameters are case sensitive.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Tonality>C|I;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Constant_Structure>CM7|;FM7|</Constant_Structure>;
```



6.2 Detached visual ending

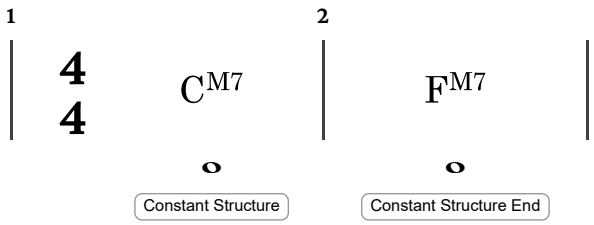
Syntax

```
<*/name>
```

A detached ending (<*/name>) closes an active tag range in duration mode 2 or 3.
The score displays separate opening and closing labels in place of a continuous line.
The range remains active for search and playback through the closing slot.
Ordinary tags use their configured name, as in <*/Constant_Structure>.
Mode and Scale use their family name without the tonic parameter, as in <*/Mode_Dorian> and <*/Scale_WholeTone>.
pedal and Centricity use <*/pedal> and <*/Centricity>, also without parameters.
A detached ending requires a matching active range and cannot act as a one-shot tag.
Mode-1 tags, including cadences and implicit forms, have no paired range to close this way.
DTC uses its own clearing command (</DTC>).
The entries for sequences and explicit forms below describe their support for detached endings.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Constant_Structure>CM7|;FM7|<*/Constant_Structure>;
```



6.3 Tonality, modality and related classifications

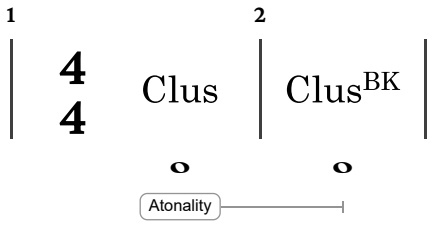
Values

```
Tonality / Modality / Atonality / Polytonality / Pantriadic
```

Tonality, Modality, Atonality, Polytonality and Pantriadic classify the tonal organization of the analyzed music.
The current ordinary-tag table assigns duration mode 2 to all five tags.
Each can label one position or a paired range.
The tonal reference ({TC:}) remains independent of these classifications and continues to determine Roman interpretation.
Use {TC:None} when the analysis has no single tonal reference for Roman interpretation.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:None}<Atonality>Clus;ClusBK</Atonality>;
```



6.4 Centricity

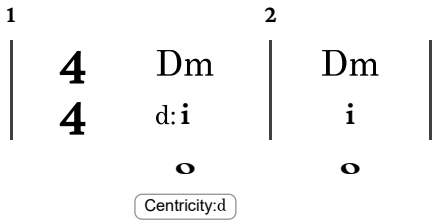
Syntax

```
<Centricity:C> ... </Centricity> / </Centricity:"C">
```

Centricity is generally used for music with a clear pitch center but without tonal harmony.
Centricity identifies a tonal center and immediately sets TC to its parameter.
The parameter accepts a concrete or quoted tonal reference with one optional b or #.
Centricity uses duration mode 2.
Its one-shot form also changes TC.
TC retains this value after a Centricity range closes.
Only one Centricity range may be active at a time.
The active range rejects TC, M and MH settings, including settings that repeat the same center.
Close the range before applying one of those settings.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}</Centricity:d>Dm|i;Dm|i;
```



6.5 Mode

Syntax

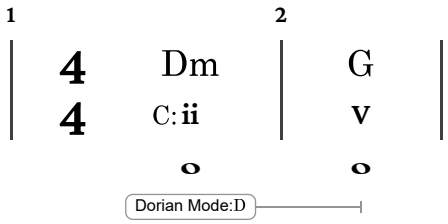
```
<Mode_Dorian:D> ... </Mode_Dorian>
```

Mode generally describes a larger-scale harmonic organization, such as a whole work or passage whose bass notes or chords remain within one mode.
Scale, defined below, usually describes a smaller melodic span.
A Mode tag (`<Mode_Dorian:D>`) identifies a configured mode and its tonic.
An available family must have a nonempty mode name in tonal-tags.csv.
The tonic parameter uses A-G or a-g, followed by an optional bb, b, # or x.

The parameter retains its letter case.
Mode uses duration mode 2, with an unparameterized closing tag (`</Mode_Dorian>`).
Only one Mode range may be active at a time.
TC remains independent of Mode, and Roman interpretation follows TC.
TC and the modulation settings M and MH may change within a Mode range.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}<Mode_Dorian:D>Dm|ii;G|V</Mode_Dorian>;
```



6.6 Scale

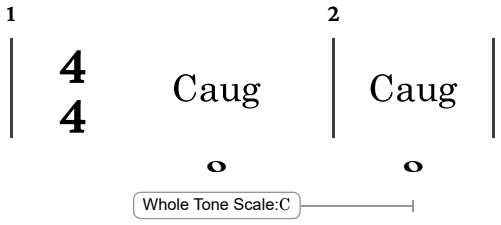
Syntax

```
<Scale_WholeTone:C> ... </Scale_WholeTone>
```

Scale generally describes a scale used in the melody over the current harmony within a short passage.
Its typical scope is smaller than the harmonic organization described by Mode.
A Scale tag (`<Scale_WholeTone:C>`) identifies a configured scale and its tonic.
Available families use the scale-name columns in tonal-tags.csv.
The tonic parameter uses the same note spelling as Mode, including both letter cases and the optional bb, b, # or x accidental.
Scale uses duration mode 2.
TC retains its value when Scale opens or closes.
AlteredScale, MixedBluesScale, MinorBluesScale, MajorBluesScale and BebopScale are available only as Scale families.
The inventory below lists all configured families.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Scale_WholeTone:C>Caug|;Caug|</Scale_WholeTone>;
```



6.7 DTC: two named tonalities

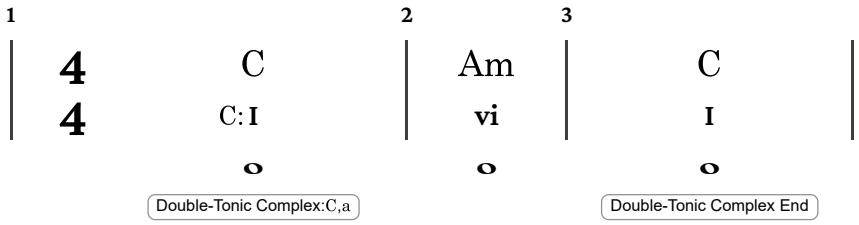
Syntax

```
<DTC:C,a> / </DTC>
```

DTC (<DTC:C,a>) labels a double-tonic complex with two named tonalities.
Both parameters are unquoted concrete major or minor references with one optional b or #.
The two references must differ in tonic pitch class or in major/minor quality.
Same-tonic major and minor references are allowed.
Enharmonic references with the same major/minor quality count as duplicates.
The displayed pair preserves input order.
Only one pair can be active at a time, and each new DTC replaces the previous pair.
The pair remains active until another DTC replaces it or </DTC> clears it.
Clearing an inactive DTC leaves it inactive.
DTC requires no closing tag at the end of the document.
TC remains independent of DTC and retains its value when DTC changes or clears.
The global-tag metadata field ({GLOBAL_TAGS}) accepts one complete pair, for example </DTC:C,a>.
A global DTC applies throughout the document and leaves TC unchanged.
When a global DTC is present, MAIN must contain no DTC command, including clearing or closing commands.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<DTC:C,a>C|I;Am|vi;</DTC>C|I;
```



6.8 Implicit form classifications and sections

Syntax

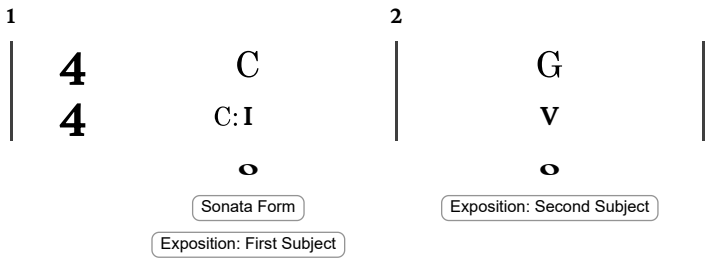
```
</Sonata_Form> / </Sonata_Form:E_Sub1>
```

A form tag classifies a form (</Sonata_Form>) or identifies a section within it (</Sonata_Form:E_Sub1>).
Forms use a separate seven-column configuration table.
The allow_global column admits an item to GLOBAL_TAGS when its value is 1.
These descriptive classifications coexist with their sections; a later section does not replace the classification.
A mode-1 form tag creates an implicit state: its label applies from that slot onward until a form command replaces or clears it.
The parent column groups sections into a family.
An entry with a blank parent defines its own family.

The replace_previous value determines how a new implicit item affects earlier items in the same family.
With a value of 1, a new item replaces an earlier item in the same family only if both have allow_global=0; a value of 0 allows them to coexist.
Replacing an item discards its implicit state; clearing the replacement leaves the earlier item inactive.
Active implicit form labels participate in search and playback.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Sonata_Form></Sonata_Form:E_Sub1>C|I;</Sonata_Form:E_Sub2>G|V;
```



6.9 FORM_END

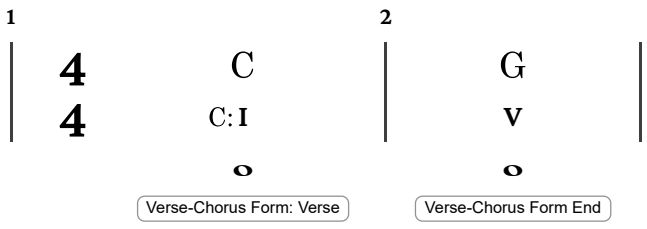
Syntax

```
</FORM_END> / </FORM_END:family> / </FORM_END:family:section>
```

FORM_END clears implicit form states.
The bare form (`</FORM_END>`) clears all implicit form items.
A family target (`</FORM_END:family>`) clears every implicit item in that family.
A complete target (`</FORM_END:family:section>`) clears only the named item.
Each target must exist in the configuration.
Clearing a configured inactive target leaves it inactive.
FORM_END acts as a command, so searches cannot use it as an active tag.
Explicit paired form ranges require their own closing tags.
On repeats and timeline jumps (`LOOP`, described in Chapter 7), form commands follow actual performance order.
A backward jump restores the implicit form state saved before its target.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Pop_Form:Verse>C|I;</FORM_END:Pop_Form>G|V;
```



6.10 Explicit form ranges

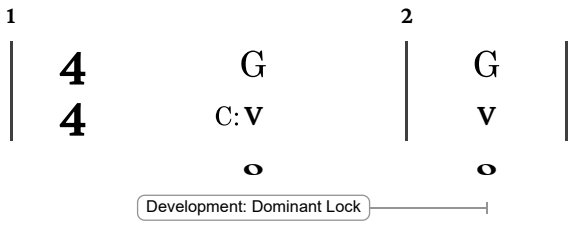
Syntax

```
<Sonata_Form:D_DL> ... </Sonata_Form:D_DL>
```

A form configured with duration mode 2 or 3 can open an explicit paired range.
The range continues until its matching closing tag.
A detached closing tag (`<*/Sonata_Form:D_DL>`) is also accepted.
Implicit replacement and FORM_END affect only implicit states and leave the explicit range active.
The current D_DL, 12_Bar_Blues and 16_Bar_Blues entries use mode 2.
Most other configured forms use implicit mode 1.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Sonata_Form:D_DL>G|V;G|V</Sonata_Form:D_DL>;
```



6.11 Cadence labels

Syntax

```
</PAC>
```

A cadence label (`</PAC>`) classifies a harmonic arrival or a connection between phrases.
All configured cadence labels are one-shot tags.
Place the label on the event identified by the definition, usually the arrival chord.
The analyst determines the cadence from the source music, including its phrase structure, soprano and voice leading.
The compiler checks the tag spelling and its allowed parameter.
The formulae below give scale degrees relative to the stated tonic.
The examples use the MStruct Roman spellings defined in Chapter 5.

6.12 PAC

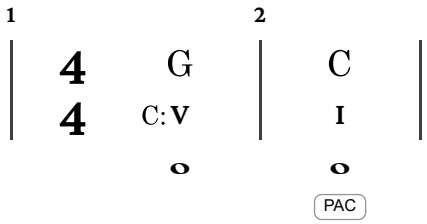
Syntax

```
</PAC>
```

PAC stands for Perfect Authentic Cadence.
A perfect authentic cadence moves from dominant to tonic.
Both chords are in root position, and the soprano arrives on the tonic scale degree.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;C|I</PAC>;
```



6.13 IAC:not_tonic

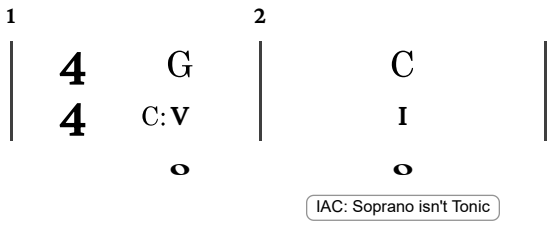
Syntax

```
</IAC:not_tonic>
```

IAC stands for Imperfect Authentic Cadence.
This imperfect authentic cadence moves from dominant to tonic with a soprano arrival on a scale degree other than the tonic.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;C|I</IAC:not_tonic>;
```



6.14 IAC:inverted

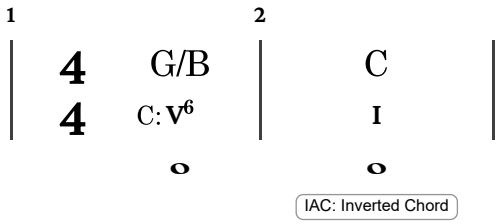
Syntax

```
</IAC:inverted>
```

IAC stands for Imperfect Authentic Cadence.
This imperfect authentic cadence uses an inverted chord in its dominant-to-tonic progression. The harmonic events specify which chord is inverted and its inversion.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G/B|V6;C|I</IAC:inverted>;
```



6.15 IAC:viio

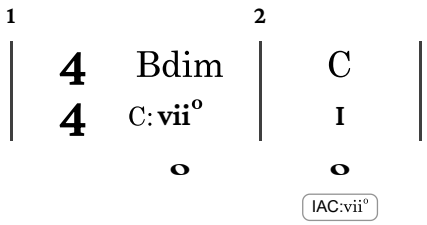
Syntax

```
</IAC:viio>
```

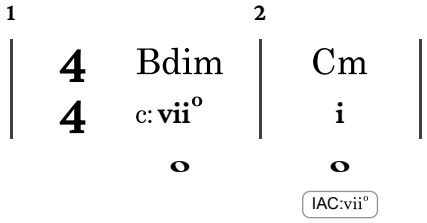
IAC stands for Imperfect Authentic Cadence.
This imperfect authentic cadence moves from the leading-tone diminished triad to a major or minor tonic: viio to I or i.
The tag belongs on the tonic arrival.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Bdim|viio;C|I</IAC:viio>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}Bdim|viio;Cm|i</IAC:viio>;
```



6.16 HC

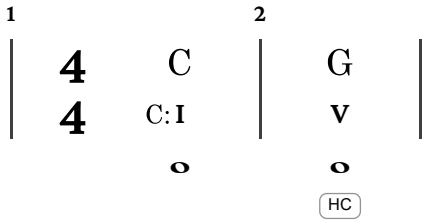
Syntax

```
</HC>
```

HC stands for Half Cadence.
A half cadence ends a phrase on dominant harmony.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;G|V</HC>;
```



6.17 HC:Phrygian

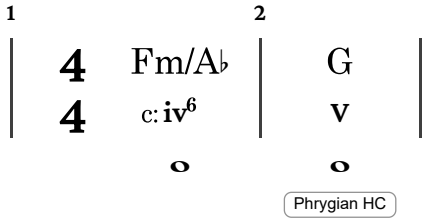
Syntax

```
</HC:Phrygian>
```

HC stands for Half Cadence.
A Phrygian half cadence moves from iv6 to V in a minor reference.
Its bass descends by semitone.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}Fm/Ab|iv6;G|V</HC:Phrygian>;
```



6.18 HC-PAC

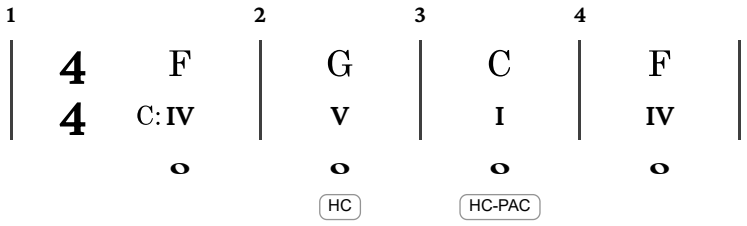
Syntax

```
</HC-PAC>
```

HC-PAC connects two parallel phrases.
The first phrase ends with a half cadence.
The second phrase begins by immediately resolving that dominant to a tonic chord satisfying PAC conditions.
The dominant and tonic are both in root position, and the soprano arrives on the tonic scale degree.
Place HC on the first phrase's final dominant and HC-PAC on the tonic at the opening of the second phrase.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;G|V</HC>;C|I</HC-PAC>;F|IV;
```



6.19 HC-IAC

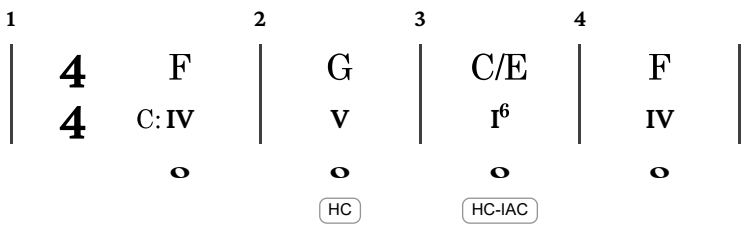
Syntax

```
</HC-IAC>
```

HC-IAC connects two parallel phrases.
The first phrase ends with a half cadence.
The second phrase begins by immediately resolving that dominant to a tonic chord satisfying IAC conditions.
Place HC on the first phrase's final dominant and HC-IAC on the tonic at the opening of the second phrase.
The example uses a first-inversion tonic to satisfy the inversion condition for IAC.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;G|V</HC>;C/E|I6</HC-IAC>;F|IV;
```



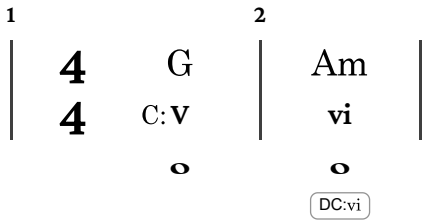
6.20 DC:vi

Syntax

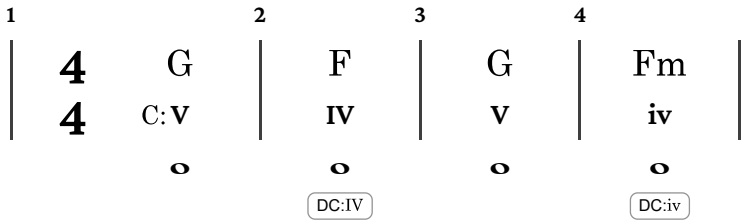
```
</DC:vi>
```

DC stands for Deceptive Cadence.
A deceptive cadence redirects the expected dominant-to-tonic resolution to a different harmony.
The parameter names the destination after the dominant.
It must use one of the destinations in the cadence-parameter inventory below.
IV and iv are available when a move from V to a major or minor subdominant deflects an expected cadential tonic arrival.
If a phrase closes on V and IV or iv begins the next phrase without that expected resolution, label the close as HC.
Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;Am|vi</DC:vi>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;F|IV</DC:IV>;G|V;Fm|iv</DC:iv>;
```



6.21 PC:IV

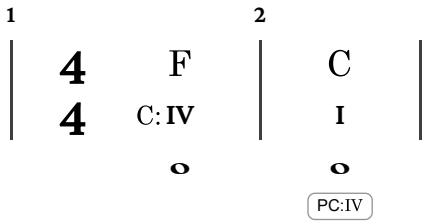
Syntax

```
</PC:IV>
```

PC stands for Plagal Cadence.
A plagal cadence approaches tonic from the harmony named by its parameter.
The allowed parameters are IV, iv, ii, iio and iiho7.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;C|I</PC:IV>;
```



6.22 EC

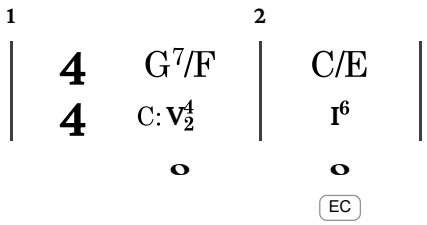
Syntax

```
</EC>
```

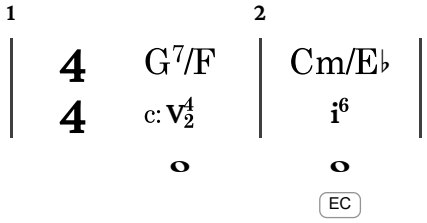
EC stands for Evaded Cadence.
An evaded cadence moves from V42 to I6 or i6.
The third-inversion dominant seventh resolves to the first-inversion major or minor tonic.
In the major example, the bass descends by semitone from F to E.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G7/F|V42;C/E|I6</EC>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}G7/F|V42;Cm/Eb|i6</EC>;
```



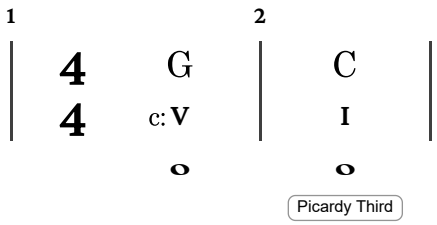
6.23 Picardy_Third

Syntax

```
</Picardy_Third>
```

A Picardy third raises the tonic third at a minor-mode close, producing a major tonic chord.
Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}G|V;C|I</Picardy_Third>;
```



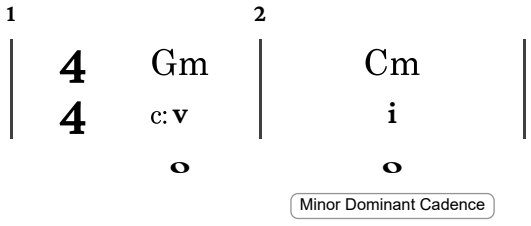
6.24 Minor_Dominant_Cadence

Syntax

```
</Minor_Dominant_Cadence>
```

A minor dominant cadence resolves a minor dominant triad to a major or minor tonic: v to I or i.
The dominant triad contains a minor third above its root.
Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Gm|v;Cm|i</Minor_Dominant_Cadence>;
```



6.25 Backdoor_Cadence

Syntax

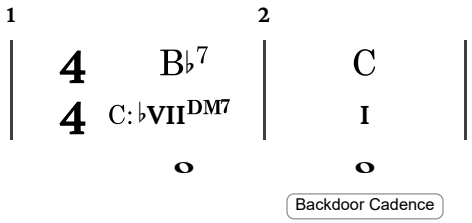
```
</Backdoor_Cadence>
```

A backdoor cadence resolves a dominant-seventh chord rooted on flat degree VII to a major or minor tonic.
With C as tonic, Bb7 resolves to C or Cm.

The example uses bVIIDM7 to specify the dominant-seventh quality.
In MStruct, bare bVII7 gives this base item its default major seventh.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Bb7|bVIIDM7;C|I</Backdoor_Cadence>;
```



6.26 Aeolian_Cadence

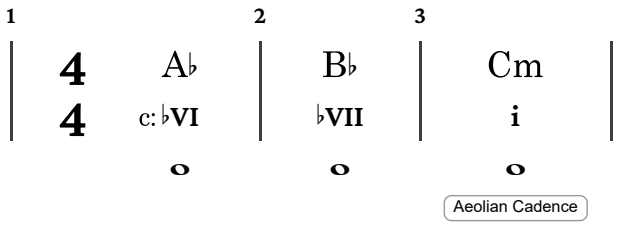
Syntax

```
</Aeolian_Cadence>
```

This Aeolian cadence moves through bVI and bVII to i.
The two approach chords are major triads, and the tonic is a minor triad.
With C as tonic, the progression is Ab to Bb to Cm.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Ab|bVI;Bb|bVII;Cm|i</Aeolian_Cadence>;
```



6.27 Mario_Cadence

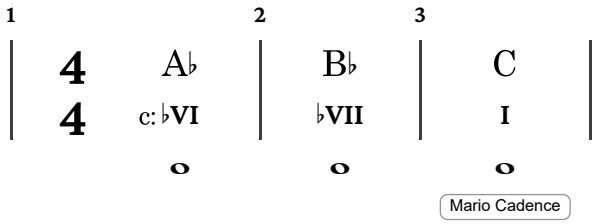
Syntax

```
</Mario_Cadence>
```

A Mario cadence moves through bVI and bVII to I.
All three chords are major triads.
With C as tonic, the progression is Ab to Bb to C.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Ab|bVI;Bb|bVII;C|I</Mario_Cadence>;
```



6.28 Aeolian_Cadence:minor

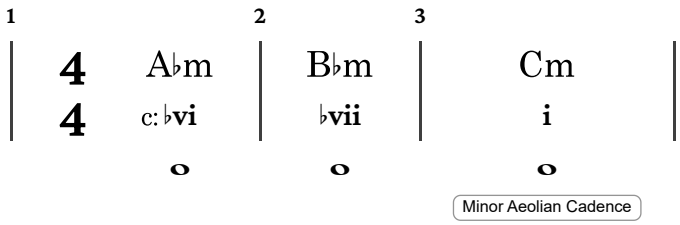
Syntax

```
</Aeolian_Cadence:minor>
```

The minor-triad Aeolian cadence moves through bvi and bvii to i.
All three chords are minor triads.
With C as tonic, the progression is Abm to Bbm to Cm.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Abm|bvi;Bbm|bvii;Cm|i</Aeolian_Cadence:minor>;
```



6.29 Phrygian_Cadence_to_Tonic:bII

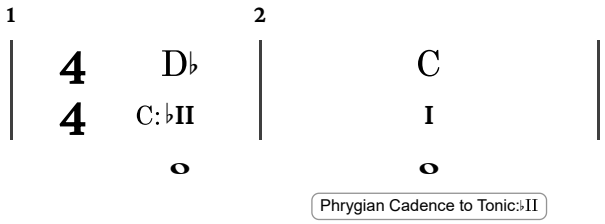
Syntax

```
</Phrygian_Cadence_to_Tonic:bII>
```

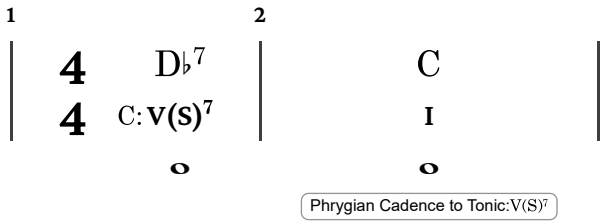
This Phrygian cadence approaches a major or minor tonic from a harmony rooted on flat degree II.
The parameter names the approach harmony.
It must be exactly bII, bII7, VS7, VS7b5 or VSBlk.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Db|bII;C|I</Phrygian_Cadence_to_Tonic:bII>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
Db7|VS7;C|I</Phrygian_Cadence_to_Tonic:VS7>;
```



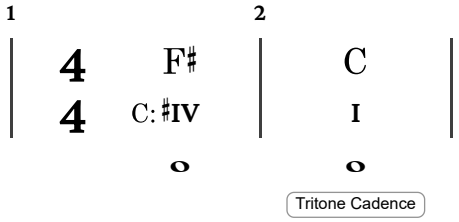
6.30 Tritone_Cadence

Syntax

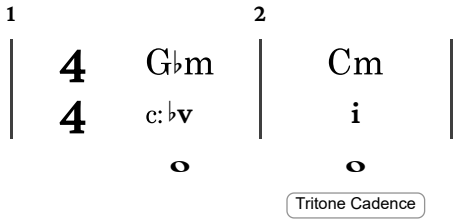
```
</Tritone_Cadence>
```

A tritone cadence resolves a harmony rooted a tritone from the tonic to I or i.
The approach triad may be major or minor, with Roman spelling #IV, #iv, bV or bv.
With C as tonic, F#, F#m, Gb or Gbm may approach the tonic.
The Roman spelling preserves the written distinction between sharp degree IV and flat degree V.
Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F#|#IV;C|I</Tritone_Cadence>;
```



```
{BN:1}{TS:4/4}{D:1}{TC:c}Gbm|bv;Cm|i</Tritone_Cadence>;
```



6.31 Mediant_Cadence:III

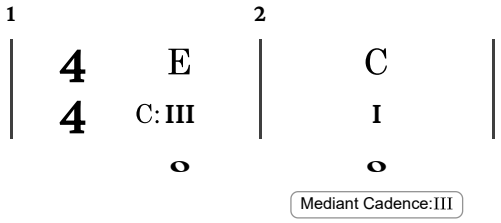
Syntax

```
</Mediant_Cadence:III>
```

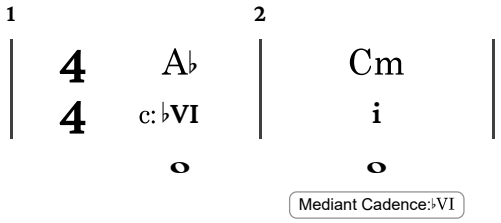
A mediant cadence resolves a degree-III or degree-VI harmony to tonic.
The parameter names the harmony before the tonic.
The allowed parameters are III, iii, bIII, biii, nIII, niii, VI, vi, bVI, bvi, nVI and nvi.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
E|III;C|I</Mediant_Cadence:III>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}Ab|bVI;Cm|i</Mediant_Cadence:bVI>;
```



6.32 pedal: sustained pitch-degree label

Syntax

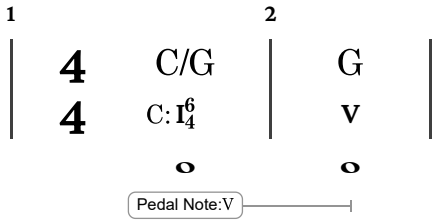
```
<pedal:V> ... </pedal>
```

pedal (`<pedal:V>`) labels a sustained pitch by scale degree.
Its parameter must be a basic Roman token from the current base table.
The tag uses duration mode 3 and requires a paired range in MAIN.

Its closing tag is `</pedal>`, without a Roman parameter.
The harmonic events specify their own bass notes.
Applying pedal leaves TC unchanged.
The parameterized ending form (`</pedal:V>`) is also available in GLOBAL_TAGS.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<pedal:V>C/G|I64;G|V</pedal>;
```



6.33 Seq and SeqInv

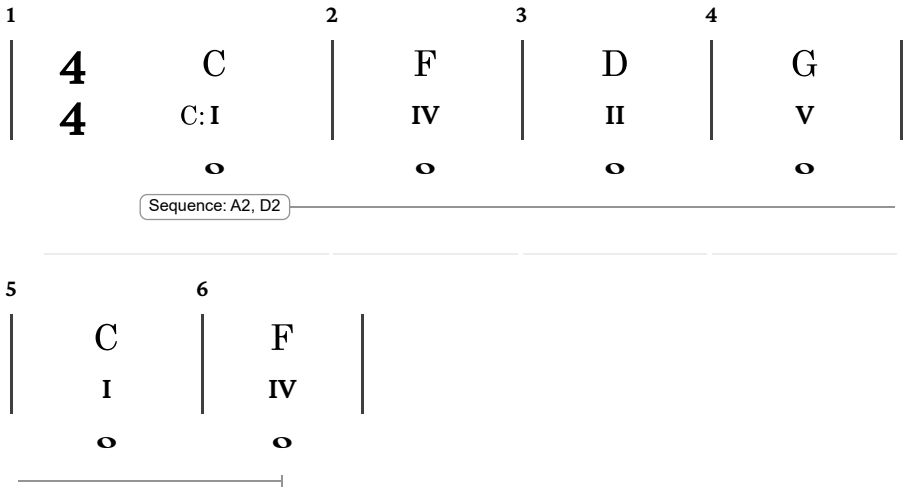
Syntax

```
<Seq:A2,D2> ... </Seq>
```

Seq (`<Seq:A2,D2>`) and SeqInv (`<SeqInv:A2,D2>`) label configured sequence types.
Each parameter list contains one or more distinct exact types from sequence-tags.csv.
The list uses commas between items and forbids spaces and empty items.
Each type identifies only its exact spelling; derived spellings require their own entries.
Seq and SeqInv have independent active ranges and may overlap.
Both require paired ranges and close without parameters: `</Seq>` or `</SeqInv>`.
They also accept detached endings: `<*/Seq>` or `<*/SeqInv>`.
GLOBAL_TAGS accepts neither Seq nor SeqInv.
The inventory below lists the configured classification codes.
The six-measure example treats C-F as a two-chord unit: it moves up a second to D-G, then down a second to C-F.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Seq:A2,D2>C|I;F|IV;D|II;G|V;C|I;F|IV</Seq>;
```



6.34 Configured progression tags

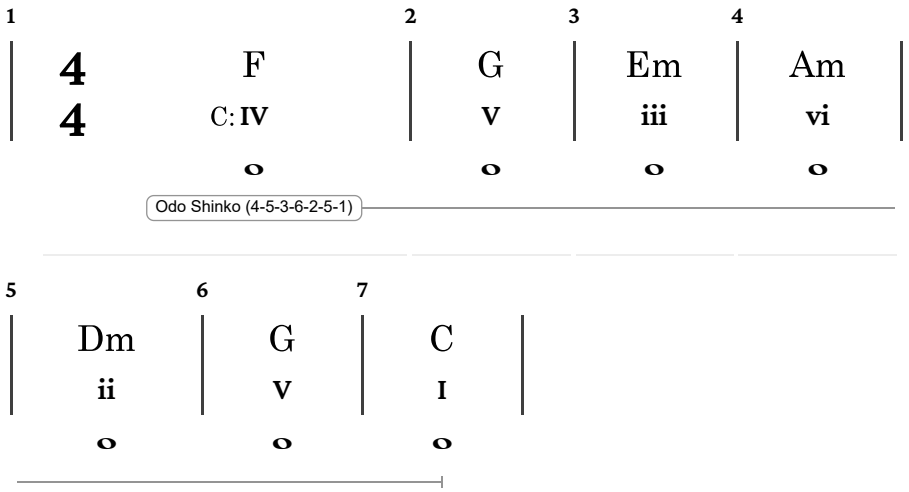
Syntax

```
<Descending_Line_Cliche> ... </Descending_Line_Cliche>
```

The progression tags classify a range of harmonic events by a named progression.
The current tags are Descending_Line_Cliche, Ascending_Line_Cliche, Omnibus_Progression, Coltrane_Changes, Constant_Structure, 4536251 and 4536251V.
Each uses duration mode 3 and requires an explicit paired range.
The analyst selects the classification for the enclosed harmonies.

Examples

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<4536251>F|IV;G|V;Em|iii;Am|vi;Dm|ii;G|V;C|I</4536251>;
```



6.35 GLOBAL_TAGS

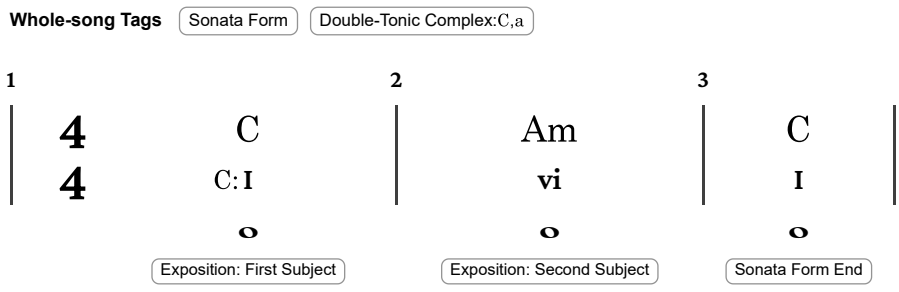
Syntax

```
{GLOBAL_TAGS}</Tonality></Sonata_Form>
```

GLOBAL_TAGS (`{GLOBAL_TAGS}`) is a metadata field containing analytical labels for the entire document.
Its value concatenates complete ending-form tags.
Each tag identity must be unique after normalization.
The field accepts ordinary tags and the supported parameterized tags described above.
A form tag is admitted when its configuration has allow_global=1, whether or not it has a parent.
The current table admits Binary_Form, Ternary_Form, Rondo_Form, Arch_Form, Variation_Form, Sonata_Form, 12_Bar_Blues, 16_Bar_Blues, Pop_Form, Canon, Fugue, StropheBridge_Form and EDM_Form.
These global classifications remain active when local sections change or FORM_END clears local form state.
One DTC pair is also allowed, for example </DTC:C,a>; it excludes all local DTC commands in MAIN.
FORM_END, Seq, SeqInv and forms with allow_global=0 are unavailable in this field.
Each global label applies to every search and playback event in the document.
TC retains its value independently of global labels.
A global label continues to apply when a local range of the same tag closes.
The catalog metadata field (`{TAG}`) stores a separate list of catalog keywords.

Examples

```
METADATA:
{IS_STABLE_TEMPO}False
{GLOBAL_TAGS}</Sonata_Form></DTC:C,a>
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}</Sonata_Form:E_Sub1>C|I;
</Sonata_Form:E_Sub2>Am|vi;</FORM_END:Sonata_Form>C|I;
END_MAIN
```



6.36 Complete ordinary tag inventory

Token	Name	Mode
<Coltrane_Changes>	Coltrane Changes	3
<Constant_Structure>	Constant Structure	3

Token	Name	Mode
<Omnibus_Progression>	Omnibus Progression	3
<Atonality>	Atonality	2
<Polytonality>	Polytonality	2
<Pantriadic>	Pantriadic	2
<Modality>	Modality	2
<Tonality>	Tonality	2
<4536251>	Odo Shinko (4-5-3-6-2-5-1)	3
<4536251V>	Odo Shinko (4-5-3-6-2-5-1) Variant	3

6.37 Complete Mode/Scale family inventory

Family	Mode name	Scale name
WholeTone	Whole Tone Mode	Whole Tone Scale
Hexatonic	Hexatonic Mode	Hexatonic Scale
Octatonic	Octatonic Mode	Octatonic Scale
MoLT3	Mode of Limited Transpositions 3	Mode of Limited Transpositions Scale 3
MoLT4	Mode of Limited Transpositions 4	Mode of Limited Transpositions Scale 4
MoLT5	Mode of Limited Transpositions 5	Mode of Limited Transpositions Scale 5
MoLT6	Mode of Limited Transpositions 6	Mode of Limited Transpositions Scale 6
MoLT7	Mode of Limited Transpositions 7	Mode of Limited Transpositions Scale 7
GongPentatonic	Gong Pentatonic Mode	Gong Pentatonic Scale
ShangPentatonic	Shang Pentatonic Mode	Shang Pentatonic Scale
JuePentatonic	Jue Pentatonic Mode	Jue Pentatonic Scale
ZhiPentatonic	Zhi Pentatonic Mode	Zhi Pentatonic Scale
YuPentatonic	Yu Pentatonic Mode	Yu Pentatonic Scale
Miyakobushi	Miyakobushi Mode	Miyakobushi Scale
Ryukyu	Ryukyu Mode	Ryukyu Scale
Ionian	Ionian Mode	Ionian Scale
Melodic_Minor	Melodic Minor Mode	Melodic Minor Scale
Dorian	Dorian Mode	Dorian Scale
Dorianb2	Dorianb2 Mode	Dorianb2 Scale
Phrygian	Phrygian Mode	Phrygian Scale
PhrygianDominant	Phrygian Dominant Mode	Phrygian Dominant Scale
Lydian	Lydian Mode	Lydian Scale
Lydian#5	Lydian#5 Mode	Lydian#5 Scale
Lydianb7	Lydianb7 Mode	Lydianb7 Scale
Mixolydian	Mixolydian Mode	Mixolydian Scale
Aeolian	Aeolian Mode	Aeolian Scale
Melodic_Major	Melodic Major Mode	Melodic Major Scale
Locrian	Locrian Mode	Locrian Scale
Locriann2	Locriann2 Mode	Locriann2 Scale
SuperLocrian	Super Locrian Mode	Super Locrian Scale
AlteredScale	-	Altered Scale

Family	Mode name	Scale name
HarmonicMinor	Harmonic Minor Mode	Harmonic Minor Scale
HarmonicMajor	Harmonic Major Mode	Harmonic Major Scale
DoubleHarmonicMajor	Double Harmonic Major Mode	Double Harmonic Major Scale
MixedBluesScale	-	Mixed Blues Scale
MinorBluesScale	-	Minor Blues Scale
MajorBluesScale	-	Major Blues Scale
BebopScale	-	Bebop Scale

6.38 Cadence parameter whitelists

Family	Allowed complete Romans
<PC:>	IV,iv,ii,iio,iiho7
<Phrygian_Cadence_to_Tonic:>	bII,bII7,VS7,VS7b5,VSBlk
<Mediant_Cadence:>	III,iii,bIII,biii,nIII,niii,VI,vi,bVI,bvi,nVI,nvi
<DC:>	III,iii,bIII,biii,nIII,niii,IV,iv,VI,vi,bVI,bvi,nVI,nvi

6.39 Complete sequence-type inventory

Base code	Admitted derived codes
A1	-
A2	A2(+5/-4),A2(+4/-3)
A3	A3(+4/-2)
A4	-
A5	-
D1	D1(+4/-A4)
D2	D2(-5/+4)
D3	D3(-5/+3)
D4	-
D5	-

6.40 Complete form inventory

The Global column is allow_global: 1 admits the item to GLOBAL_TAGS, and 0 restricts it to local use.

Token	Name	Mode / replace	Global
<Binary_Form>	Binary Form	1 / 1	1
<Ternary_Form>	Ternary Form	1 / 1	1
<Rondo_Form>	Rondo Form	1 / 1	1
<Arch_Form>	Arch Form	1 / 1	1
<Variation_Form>	Theme and Variation Form	1 / 1	1

Token	Name	Mode / replace	Global
<Sonata_Form>	Sonata Form	1 / 1	1
<Sonata_Form:E_Intro>	Sonata Form: Intro	1 / 1	0
<Sonata_Form:E_Sub1>	Exposition: First Subject	1 / 1	0
<Sonata_Form:E_Transition>	Exposition: Transition	1 / 1	0
<Sonata_Form:E_MC>	Exposition: Medial Caesura	1 / 1	0
<Sonata_Form:E_Sub2>	Exposition: Second Subject	1 / 1	0
<Sonata_Form:E_EEC>	Exposition: EEC	1 / 0	0
<Sonata_Form:E_Closing>	Exposition: Closing Section	1 / 1	0
<Sonata_Form:E_Codetta>	Exposition: Codetta	1 / 1	0
<Sonata_Form:D>	Sonata Form: Development	1 / 1	0
<Sonata_Form:D_Retransition>	Development: Retransition	1 / 1	0
<Sonata_Form:D_DL>	Development: Dominant Lock	2 / 0	0
<Sonata_Form:R_Sub1>	Recapitulation: First Subject	1 / 1	0
<Sonata_Form:R_Transition>	Recapitulation: Transition	1 / 1	0
<Sonata_Form:R_Sub2>	Recapitulation: Second Subject	1 / 1	0
<Sonata_Form:R_ESC>	Recapitulation: ESC	1 / 0	0
<Sonata_Form:R_Closing>	Recapitulation: Closing Section	1 / 1	0
<Sonata_Form:Coda>	Recapitulation: Coda	1 / 1	0
<12_Bar_Blues>	12 Bar Blues	2 / 1	1
<16_Bar_Blues>	16 Bar Blues	2 / 1	1
<Pop_Form>	Verse-Chorus Form	1 / 1	1
<Pop_Form:Intro>	Verse-Chorus Form: Intro	1 / 1	0
<Pop_Form:UpfrontChorus>	Verse-Chorus Form: Upfront Chorus	1 / 1	0
<Pop_Form:Verse>	Verse-Chorus Form: Verse	1 / 1	0
<Pop_Form:Prechorus>	Verse-Chorus Form: Pre-Chorus	1 / 1	0
<Pop_Form:Chorus>	Verse-Chorus Form: Chorus	1 / 1	0
<Pop_Form:PostChorus>	Verse-Chorus Form: Post-Chorus	1 / 1	0
<Pop_Form:InstBreak>	Verse-Chorus Form: Instrumental Break	1 / 1	0
<Pop_Form:Verse2>	Verse-Chorus Form: Verse 2	1 / 1	0
<Pop_Form:Prechorus2>	Verse-Chorus Form: Pre-Chorus 2	1 / 1	0
<Pop_Form:Chorus2>	Verse-Chorus Form: Chorus 2	1 / 1	0
<Pop_Form:ChorusExtension>	Verse-Chorus Form: Chorus Extension	1 / 1	0
<Pop_Form:Bridge>	Verse-Chorus Form: Bridge	1 / 1	0
<Pop_Form:Bridge_Solo>	Verse-Chorus Form: Bridge Instrumental Solo	1 / 1	0
<Pop_Form:PreChorus3>	Verse-Chorus Form: Pre-Chorus 3	1 / 1	0
<Pop_Form:BreakdownChorus>	Verse-Chorus Form: Breakdown Chorus	1 / 1	0
<Pop_Form:Chorus3+>	Verse-Chorus Form: Chorus 3+	1 / 1	0
<Pop_Form:Outro>	Verse-Chorus Form: Outro	1 / 1	0
<Canon>	Canon	1 / 1	1
<Fugue>	Fugue	1 / 1	1

Token	Name	Mode / replace	Global
<StropheBridge_Form>	Strophe-Bridge Form	1 / 1	1
<StropheBridge_Form:Intro>	Strophe-Bridge Form: Intro	1 / 1	0
<StropheBridge_Form:Strophe>	Strophe-Bridge Form: Strophe	1 / 1	0
<StropheBridge_Form:Bridge>	Strophe-Bridge Form: Bridge	1 / 1	0
<StropheBridge_Form:InstBridge>	Strophe-Bridge Form: Instrumental Bridge	1 / 1	0
<StropheBridge_Form:Refrain>	Strophe-Bridge Form: Refrain	1 / 0	0
<StropheBridge_Form:Outro>	Strophe-Bridge Form: Outro	1 / 1	0
<Strophic_Form>	Strophic Form	1 / 1	0
<Strophic_Form:Intro>	Strophic Form: Intro	1 / 1	0
<Strophic_Form:Strophe>	Strophic Form: Strophe	1 / 1	0
<Strophic_Form:Refrain>	Strophic Form: Refrain	1 / 0	0
<Strophic_Form:InstStrophe>	Strophic Form: Instrumental Strophe	1 / 1	0
<Strophic_Form:InstBreak>	Strophic Form: Instrumental Break	1 / 1	0
<Strophic_Form:Outro>	Strophic Form: Outro	1 / 1	0
<EDM_Form>	EDM Form	1 / 1	1
<EDM_Form:Intro>	EDM Form: Intro	1 / 1	0
<EDM_Form:Groove>	EDM Form: Groove	1 / 1	0
<EDM_Form:Breakdown>	EDM Form: Breakdown	1 / 1	0
<EDM_Form:BuildUp>	EDM Form: Build-Up	1 / 1	0
<EDM_Form:Drop>	EDM Form: Drop	1 / 1	0
<EDM_Form:Outro>	EDM Form: Outro	1 / 1	0

7 Timeline syntax

7.1 One recording, one timeline

A timeline (.mstructtime) maps the events of an analysis file to times in one recording.
The analysis file supplies the chord and Roman content.
The timeline supplies recording metadata, an explicit traversal route (LOOP) and timestamps (MAIN).
Timeline associations require a non-stable-tempo analysis.
An analysis with associated timelines must retain its non-stable-tempo setting.

7.2 METADATA, LOOP and MAIN

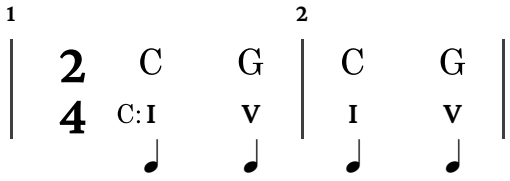
Syntax

```
METADATA: ... LOOP: ... MAIN:
```

A timeline contains METADATA, LOOP and MAIN in that order.
Each section closes with its corresponding END_ marker.
METADATA describes the recording.
LOOP defines jumps between positions in the parent analysis.
An empty LOOP follows the structural source positions in order.
MAIN records timestamps at those positions.
The following example traverses measure 1 twice and measure 2 once.
The parent analysis appears before its timeline.

Examples

```
{BN:1}{TS:2/4}{D:1/4}{TC:C}
C|I,G|V;C|I,G|V;
```




```
METADATA:
{MSTRUCTTIME_NAME}Example recording
{PERFORMER_1}Example performer
{RECORD_YEAR}2026
{MUSIC_LENGTH}0:06
{URL_APPLE_MUSIC}https://music.apple.com/example
{SCORE_URL_1}https://example.org/score
{URL_ADDITIONAL_1:Recording details}https://example.org/recording
END_METADATA

LOOP:
1,2/4,1,1,1/4;
1,2/4,2,2,1/4;
END_LOOP
```

```
MAIN:
{BN:1}{TS:2/4}{D:1/4}
[0,1][2000,2],[1000,1][3000,2];
{BN:2}{D:1/4}[4000,1],[5000,1];
END_MAIN
```

7.3 Recording metadata

Fields

```
{MSTRUCTTIME_NAME} / {PERFORMER_n} / {MUSIC_LENGTH}
```

The recording name ({MSTRUCTTIME_NAME}) is required.
At least one named performer ({PERFORMER_n}) is required.
The file retains up to 100 performers.
The recording duration ({MUSIC_LENGTH}) is required and uses m:ss or h:mm:ss.
A name, performer or album value containing non-ASCII characters requires a corresponding nonempty ASCII romanization field.
The name uses {MSTRUCTTIME_NAME_ROMANIZED}.
Each performer uses {PERFORMER_n_ROMANIZED}.
The album uses {RECORD_ALBUM_ROMANIZED}.
The optional recording year ({RECORD_YEAR}) uses a nonnegative integer string.
The recording album ({RECORD_ALBUM}) is optional.
Each analysis (.mstruct) and each recording timeline (.mstructtime) stores its own independent set of links in METADATA.
Platform links use {URL_NETEASE}, {URL_TME}, {URL_YOUTUBE}, {URL_BILIBILI}, {URL_SPOTIFY} and {URL_APPLE_MUSIC}, followed by the corresponding URL; these fields are optional.
Score links use {SCORE_URL_n} with n from 1 to 9.
Additional links use {URL_ADDITIONAL_n:label} followed by a URL, with n from 1 to 9; the editor requires both the website label and URL.
The older unnumbered input {URL_ADDITIONAL:label} remains readable; saving writes numbered fields.
A timeline without link fields has an empty link set.
On the website, analysis links and the selected timeline' s recording links are displayed separately.

7.4 LOOP jump instruction

Syntax

```
sourceMeasure,sourceBeat,pass,targetMeasure,targetBeat;
```

Each nonempty LOOP line contains one jump instruction.
The source and target each specify a measure number and a beat coordinate.
The pass is the positive count of visits to that source position.
After the specified visit, traversal jumps to the target position.
A source position with no applicable instruction proceeds to the next structural source position.
Each source-measure, source-beat and pass combination must be unique.
Both coordinates must resolve to positions in the parent analysis.
The route must finish within the traversal step limit.

Examples

```
1,2/4,1,1,1/4;
1,2/4,2,2,1/4;
```

7.5 Source endpoint versus target start

A source beat p/q identifies a slot end at p/q whole notes after the measure begins.
A target beat p/q identifies a slot start at (p-1)/q whole notes after the measure begins.
Displayed event beats use the same one-based start notation.
Their denominator is a common denominator of the meter (TS) and slot durations (D).
A 4/4 measure can contain two half-note slots.
The second slot starts halfway through the measure, so its target coordinate is 3/4.
It ends at the measure boundary, so its source coordinate is 4/4.
A jump after this slot back to its own start therefore uses source 4/4 and target 3/4.

7.6 LOOP end instruction

Syntax

```
sourceMeasure,sourceBeat,pass,end;
```

The four-field end instruction terminates traversal after the specified source visit.
That visit remains the final position in the route.
The last recorded event remains selected until the recording duration (MUSIC_LENGTH) ends.
The word end is case insensitive.

Examples

```
1,2/4,1,1,1/4;
1,2/4,2,end;
```

7.7 Timestamp record

Syntax

```
[milliseconds,pass] / [,pass]
```

Recording timestamps are normally entered with the timeline editor.
A timestamp [milliseconds,pass] assigns a recording time to one visit of a source position.
The time uses integer milliseconds from the recording start.
The pass uses a positive integer.
An empty time field, as in [,2], marks an unrecorded visit.
An explicit 0 marks the recording start.
Records for several visits to the same position appear in adjacent brackets.
BN selects the source measure.
Each comma advances the position by D, and each semicolon advances to the next measure.

Examples

```
{BN:1}{TS:2/4}{D:1/4}
[0,1][2000,2],[1000,1][,2];
```

7.8 Which positions receive rows

Event row types

```
START / event / ...
```

The timeline table begins with an implicit START row at 0 ms.
Musical events and visible ellipses each receive a row.
Empty comma slots extend the preceding event.
A slot containing only settings produces no event row.
An ellipsis has its own measure, start position and pass.
Its timestamp can appear as `...[5000,1]` in timeline MAIN.
Recorded times must be nondecreasing along the actual LOOP route.
Several consecutive rows may have equal times.

7.9 Unrecorded and residual timestamps

Syntax

```
[,1]
```

Playback keeps the existing highlight while passing an unrecorded event.
The next recorded event receives the highlight at its timestamp.
Residual timestamps are records for visits outside the current route.
The file preserves these records for recovery.
Resetting from a measure clears timestamps in that measure and all later measures, including residual records.

7.10 Importing analysis repeats

Repeat import converts a validated analysis route, including its ending choices, into explicit LOOP instructions.
The conversion uses slot ends for sources and slot starts for targets.
The resulting timeline follows the LOOP route.
Repeats with continuous measure numbering (>>) cannot be imported.
Changes to the parent analysis structure require the route coordinates and existing timestamps to be checked again.

7.11 Offsets and file association

Syntax

```
{MSTRUCTTIME_n}filename.mstructtime
```

Parent metadata associates timelines through MSTRUCTTIME_n fields.
It retains up to 100 distinct associated filenames.
A generated filename uses the form parent-time-eightcharacters.mstructtime, with a fresh identifier for each new file.
Non-stable playback follows the associated timeline timestamps and adds the user global delay and temporary session offset.
These offsets leave the timestamps stored in the timeline unchanged.
Stable analyses use the BPM route without an associated timeline, adding the user global delay, temporary session offset and analysis file's OFFSET.
The analysis file's OFFSET applies only to stable playback.

8 Search syntax

Each query below has concrete matching sources, counterexamples and reasons. A match means that the whole query finds at least one source window. Unless a row supplies other settings, its source begins with `{BN:1}{TS:4/4}{D:1}{TC:C}`; semicolons separate source measures. A `{GLOBAL_TAGS}` line belongs in metadata.

8.1 One event and an ordered sequence

Syntax

```
C|I / C|I,G|V
```

A query event describes one source event to be matched.
A harmonic query event supplies both a chord field and a Roman field, separated by |.
A whole-field * leaves that side unconstrained.
The equivalent explicit form uses [C:chord|R:roman].
C| is valid in MAIN but requires C|* in a search.
Top-level commas request a sequence of adjacent events in the stated order.
Queries use these commas across source measure boundaries and do not accept measure-ending semicolons.
All conditions attached to one query event must hold together.
A result window identifies the source events that satisfy the sequence.
Repeated visits to those same events can share one result window.

Query 1

```
C|I,G|V
```

Result	Concrete source	Reason
Match	`C|I;G|V;`	C|I is followed by G|V across the barline, so the two events satisfy the query in order.
No match	`C|I;F|IV;G|V;`	The ordinary F|IV event intervenes between C|I and G|V and breaks the requested adjacency.

Query 2

```
[C:C|R:I]
```

Result	Concrete source	Reason
Match	`C|I;`	The event C|I has exact chord C and exact Roman I, so it satisfies both query fields.

8.2 Complete fields and equivalent chord notation

Syntax

```
Cm6|* / *|V7
```

Complete concrete chord queries match the same final bass pitch class and complete pitch-class set by default.
The final bass is the root of the structure after slash and on operations.
Cm6 and Am7b5/C both have bass C and notes C, Eb, G, A.
G/C and CM9no3 both have bass C and notes C, D, G, B.
A different final bass gives a different match even if the absolute pitch-class set is unchanged.
Matching ignores enharmonic spelling, octave doubling and note order.
Add –notallowdifferentnotation after the query to compare chord notation; each query permits this option at most once.
Notation comparison still normalizes formal and legacy altered-note lists.
Configured special chords use their interpreted sets; unknown historical exact special tokens remain searchable by source identity.
Exact Roman queries retain their written identities.
The main symbol or a same-domain alternative can satisfy a positive query.
Either eligible layer of a dual Roman analysis can match, and a Roman sequence follows a continuous tonal path.

Query 1

```
C7|*
```

Result	Concrete source	Reason
Match	`C7|IDM7;`	C7 matches the exact chord field, and * accepts the Roman IDM7.
No match	`CM7|I7;`	The chord CM7 has a major seventh, so its complete symbol differs from the required C7.

Query 2

```
*|V7
```

Result	Concrete source	Reason
Match	`G7|V7;`	The Roman field is exactly V7, and * accepts the chord G7.
No match	`G7/B|V65;`	The Roman field V65 specifies a different inversion from the exact query V7.

Query 3

```
Cm6|*
```

Result	Concrete source	Reason
Match	`Am7b5/C|;`	Am7b5/C has final bass C and the same complete set C, Eb, G, A as Cm6.
No match	`Am7b5|;Cm6/Eb|;`	Both chords have a different final bass from C, so neither is equivalent to Cm6.

Query 4

```
Cm6|* --notallowdifferentnotation
```

Result	Concrete source	Reason
Match	`Cm6|;`	The option requests notation matching, and the source is written Cm6.
No match	`Am7b5/C|;`	Am7b5/C has an equivalent final set and bass but uses a different notation.

Query 5

```
G/C|*
```

Result	Concrete source	Reason
Match	`CM9no3|;`	Both chords have final bass C and complete set C, D, G, B.
No match	`CM9no3/G|;`	The complete absolute set is unchanged, but the final bass is G.

8.3 Roman queries for figured-bass chains

Syntax

```
V[6,4-] / *[6,4] / [6,4-x]
```

For a continuation event in a figured-bass chain, search reads the complete original Roman head, including its case, quality modifier, special token and applied targets, together with that event's own figure list.
The match is located at the continuation event; the returned source and score keep their original text and notation.
A query list is validated independently and does not need a new-chain marker.
Query figure lists still require two to four descending figures from 2 through 14, with the permitted accidentals.
For example, V[6,4] can match the middle event of V[5,3-],[6,4-],[5,3], while vi[6,4] does not.
In queries, an omitted ending marker or - accepts a source list that continues, ends naturally or ends with -x.
An explicit query -x accepts only a source list that explicitly ends with -x.

Query 1

```
*|V[6,4-]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	The middle event matches by inheriting the original V head and its own 6,4 list. The result points to C/G and retains the source notation.

Query 2

```
*|V[6,4]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	The query may omit a new-chain marker. It matches the middle event and accepts the source continuation marker.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5-],G|[5,3];`	The source figure list differs from the required 6,4 list.

Query 3

```
*|*[6,4-]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	The head wildcard accepts the inherited V and the figure-list wildcard accepts 6,4.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];`	The source figure list differs from the required 6,4 list.

Query 4

```
*|[6,4-]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	A bare figure-list query accepts any inherited Roman head while retaining list validation.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];`	The source figure list differs from the required 6,4 list.

Query 5

```
*|[6,4]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	An omitted query ending accepts a source continuation marker.
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4];`	An omitted query ending also accepts a natural end of the chain.
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-x];`	An omitted query ending also accepts an explicit -x ending.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];`	The source figure list differs from the required 6,4 list.

8.4 Roman figure lists in OR and NOT

Syntax

```
(V[6,4]||*[6,4-]) / V[6,4]!(V[6,4-x])
```

Roman figure-list queries use the same inherited-head and ending-marker rules inside finite OR and complete-field NOT.
Each OR branch is one complete Roman pattern; a NOT exclusion is compared with the actual source event, including its explicit ending marker.

Query 1

```
*|(V[6,4]||*[6,4-])
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];`	The Roman OR accepts the inherited V branch at the middle event.

Query 2

```
*|V[6,4]!(V[6,4-x])
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];`	The positive query matches the middle event, and its continuation marker does not match the explicit -x exclusion.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-x];`	The event matches the positive query and the explicit -x exclusion, so NOT rejects it.

8.5 Exact Roman identity and explicit chain ending

Syntax

```
vi[6,4] / V[6,4-x]
```

Roman head case remains significant for exact queries.
An explicit -x query ending selects only a source list that explicitly ends the chain at that event.

Query 1

```
*|vi[6,4]
```

Result	Concrete source	Reason
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];`	The inherited head is V, not the explicitly requested lowercase vi.

Query 2

```
*|V[6,4-x]
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-x];`	An explicit -x query matches the source event that explicitly terminates the chain.
No match	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	The source middle list ends with - and does not explicitly terminate the chain with -x.

8.6 Compiler-atom wildcards

Syntax

```
* / ?
```

A wildcard stands for part of a valid chord or Roman symbol.
Chord fields containing * or ? match notation, including when equivalent chord notation is enabled by default.
Outside lists, ? matches exactly one compiler atom and * matches zero or more atoms.
A compiler atom is one meaningful unit produced by symbol parsing.
A root and its accidental form one note atom, so ? can match F# as a unit.
Fixed keywords and figure groups have their own atom boundaries.
Every pattern must allow at least one valid concrete symbol.
A whole-field * accepts an omitted field and excludes an explicit field placeholder X or continuation -.
Query 1

```
C*|*
```

Result	Concrete source	Reason
Match	`C|I;Cm7|i7;C7/E|IDM65;`	Each chord has root C. The suffix * matches the empty suffix, m7 and 7/E, producing three single-event windows.
No match	`C#|;`	C# is one complete root atom, so the fixed root C fails before the suffix wildcard is applied.

Query 2

```
?m7|*
```

Result	Concrete source	Reason
Match	`Cm7|i7;F#m7|#iv7;Bbm7|bvii7;`	The ? matches the complete root atom C, F# or Bb. All three chords then match the fixed m7 suffix.
No match	`Cm|i;`	Cm has suffix m, while the query requires the complete suffix m7.

Query 3

```
*|V*
```

Result	Concrete source	Reason
Match	`G|V;G/B|V6;G7|V7;`	All three Roman symbols begin with V. The suffix * matches the empty suffix, 6 and 7.
No match	`Gm|v;`	The source Roman v is lowercase, while the query requires uppercase V.

8.7 List-item wildcards

Syntax

```
C13(b5,*)|* / Cadd(?)|*
```

In altered, add, sus, no, q4, q5 and nf lists, wildcards match complete list items.
A ? matches exactly one item, and a * matches one or more items.
The complete list must match in its written order.
For example, C13(b5,*) requires b5 followed by at least one further item.
A[q4:*], D[q5:*] and C[nf:*] abbreviate wildcard queries for those complete nonempty lists.

Query 1

```
C13(b5,*)|*
```

Result	Concrete source	Reason
Match	`C13(b5,b9)|;C13(b5,b9,#9)|;`	Both lists begin with the required b5. The * matches b9 in the first chord and b9,#9 in the second.
No match	`C13(b5)|;`	The list ends after b5, leaving no item for the required nonempty *.

Query 2

```
Cadd(?)|*
```

Result	Concrete source	Reason
Match	`Cadd(9)|;Cadd(#11)|;`	Each add list contains exactly one item. The ? matches the whole item 9 or #11.
No match	`Cadd(9,11)|;`	The source add list contains two items, while ? requires exactly one.

Query 3

```
A[q4:*]|*
```

Result	Concrete source	Reason
Match	`A[q4:(4,b7)]|;A[q4:(#4,7,10)]|;`	Both chords have root A and a nonempty q4 list. The * matches the complete list 4,b7 or #4,7,10.

Query 4

```
C[nf:*]|*
```

Result	Concrete source	Reason
Match	`C[nf:(0,4,7)]|;C[nf:(0,3,7,10)]|;`	Both chords have root C and a nonempty nf list. The * matches the complete list 0,4,7 or 0,3,7,10.

8.8 Actual adjacency and transparent events

Syntax

```
C|*,G|*
```

Adjacent query positions follow the actual playback route, including repeat jumps and selected endings.
Events in mutually exclusive endings therefore cannot form one adjacent sequence.
A transparent event is skipped while the matcher searches for the next requested event.
Bare continuations (-), unrequested rests (0) and, by default, secondary events are transparent.
An explicitly requested rest occupies one query position.
An intervening ordinary harmonic event that fails the next condition breaks the window.
Repeated traversals through the same source event IDs produce one visual result window.

Query 1

```
C|*,G|*
```

Result	Concrete source	Reason
Match	`C|I;0;G|V;`	The query skips the unrequested rest 0, so C and G satisfy its two consecutive positions.
Match	`C|I;'F|IV';G|V;`	The query skips the single-quoted secondary F|IV by default, so C and G satisfy its two positions.
No match	`C|I;F|IV;G|V;`	The ordinary F|IV event participates in matching and interrupts the requested C-to-G sequence.

Query 2

```
C|*,0,G|*
```

Result	Concrete source	Reason
Match	`C|I;0;G|V;`	The three source events satisfy the requested chord C, rest 0 and chord G in order.
No match	`C|I;G|V;`	The source has no rest between C and G, so it fails the middle query position 0.

8.9 Including secondary events

Syntax

```
--notignore2nd
```

The –notignore2nd option includes secondary events marked by single quotes in sequence matching.
They can satisfy a requested position or interrupt adjacency as ordinary events do.
The option follows the event query with intervening whitespace.
Each query permits this option at most once.

Query 1

```
C|*,G|* --notignore2nd
```

Result	Concrete source	Reason
Match	`C|I;G|V;`	C and G are adjacent source events and satisfy both query positions.
No match	`C|I;'F|IV';G|V;`	The –notignore2nd option makes the secondary F|IV participate in matching, so it interrupts the C-to-G sequence.

8.10 TC constraints: ordinary, Ambi and None

Syntax

```
{TC:C} / {TC:M} / {TC:"M"} / {TC:None}
```

A TC condition tests the tonal reference in effect at an event, including an inherited reference.
An unquoted concrete center such as C selects an ordinary reference.
Unquoted M, m and * select any ordinary major reference, any ordinary minor reference and any ordinary reference, respectively.
Quoted values ”C”, ”M”, ”m” and ”*” apply the corresponding tests to ambiguous references (Ambi).
None selects an explicitly cleared reference and excludes an initially undefined reference.
A query without a TC condition allows Roman matches under ordinary or ambiguous references.
Consecutive Roman matches must follow one underlying note-and-mode path.
Changing only the certainty from C to ”C” preserves that path.

Query 1

```
{TC:M}*|V
```

Result	Concrete source	Reason
Match	`G|V;{TC:D}A|V;`	G|V inherits ordinary TC C, and A|V uses ordinary TC D. Both events have Roman V under a major reference, as required by M.
No match	`{BN:1}{TS:4/4}{D:1}{TC:"C"} G|V;`	The event has Roman V, but its quoted TC ”C” is an ambiguous reference. The unquoted M condition requires an ordinary major reference.

Query 2

```
{TC:"M"}*|IV7
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:4/4}{D:1}{TC:"C"} FM7|IV7;`	The event has exact Roman IV7 under ambiguous major TC ”C”, satisfying the quoted ”M” condition.
No match	`FM7|IV7;`	Roman IV7 matches, but ordinary TC C fails the condition for an ambiguous major reference.

Query 3

```
{TC:None}C|*
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:4/4}{D:1}{TC:None} C|;`	The event has exact chord C under explicitly cleared TC None. The whole-field * accepts the omitted Roman field.
No match	`C|I;`	The chord C matches, but the active TC C fails the required None state.

8.11 TS constraints

Syntax

```
{TS:3/4} / {TS:*/4} / {TS:adlib}
```

A TS condition tests the meter in effect at an event, including an inherited meter.
The complete value, numerator or denominator can use *.
For example, */4 accepts both 3/4 and 4/4.
The value adlib matches free meter exactly.
Each numeric component must be a positive integer no greater than 2147483647.
Event searches do not accept D or BN conditions.

Query 1

```
{TS:3/4}*|*
```

Result	Concrete source	Reason
Match	`{BN:1}{TS:3/4}{D:3/4}{TC:C} C|I;G|V;`	Both C|I and G|V inherit TS 3/4, so each satisfies one single-event query window.
No match	`C|I;`	C|I uses TS 4/4, while the query requires exact TS 3/4.

Query 2

```
{TS:*/4}*|*
```

Result	Concrete source	Reason
Match	`C|I;{TS:3/4}{D:3/4}G|V;`	C|I uses TS 4/4, and G|V uses TS 3/4. Both denominators equal the required 4.
No match	`{BN:1}{TS:6/8}{D:6/8}{TC:C} C|I;`	C|I uses TS 6/8, whose denominator 8 fails the required denominator 4.

8.12 Local and global active tags

Syntax

```
<PAC> / <Mode_Dorian:*>
```

A tag condition selects events carrying the requested active tag.
It uses the tag name in angle brackets and omits the MAIN closing slash.
The active set combines local ranges, applicable one-shot or end tags and global tags (GLOBAL_TAGS).
A complete exact tag identity can remain searchable independently of the current default vocabulary.
Parameterized cadence, pedal, Centricity and configured Mode or Scale tags accept * or ? in the complete parameter position.
Each wildcard selects any one permitted parameter.
A fixed name containing a colon, such as IAC:not_tonic, is matched as a name rather than a parameterized family.

Query 1

```
<PAC>
```

Result	Concrete source	Reason
Match	`G|V;C|I</PAC>;`	The end tag </PAC> labels the C|I arrival, so only that event satisfies <PAC>.

Query 2

```
<Mode_Dorian:*>
```

Result	Concrete source	Reason
Match	`<Mode_Dorian:D>Dm|ii;G|V</Mode_ Dorian>;`	Both events lie in the active Mode_ Dorian:D range. The * accepts its complete parameter D.
No match	`<Mode_Phrygian:E>Em|iii</Mode_ Phrygian>;`	The source tag belongs to Mode_ Phrygian, while the query fixes the family to Mode_Dorian.

Query 3

```
<PC:?>
```

Result	Concrete source	Reason
Match	`F|IV;C|I</PC:IV>;`	The C|I event carries the end tag PC:IV. The ? matches the complete permitted parameter IV.
Match	`Fm|iv;C|I</PC:iv>;`	The C|I event carries the end tag PC:iv. The ? also accepts the complete permitted parameter iv.

8.13 Metadata-only global-tag condition

Syntax

```
--globaltag:<Tonality>
```

The –globaltag option filters documents by items in metadata GLOBAL_TAGS.
A tag written only in MAIN fails this metadata condition.
Every distinct –globaltag condition in a query must be satisfied.
The option can be used alone or combined with an event query.
The option accepts DTC conditions, including unordered pairs and major/minor wildcards, but excludes Seq and SeqInv.
Form admission follows the allow_global column of the bound configuration.

Query 1

```
--globaltag:<Tonality>
```

Result	Concrete source	Reason
Match	`{GLOBAL_TAGS}</Tonality> C|I;`	The metadata GLOBAL_TAGS contains </Tonality>, satisfying the document-level condition.
No match	`</Tonality>C|I;`	Tonality appears only in MAIN, while –globaltag requires it in metadata GLOBAL_TAGS.

Query 2

```
*|I --globaltag:<Sonata_Form>
```

Result	Concrete source	Reason
Match	`{GLOBAL_TAGS}</Sonata_Form> C|I;G|V;`	GLOBAL_TAGS contains Sonata_Form, satisfying the document filter. Only C|I has the required Roman I.
No match	`</Sonata_Form>C|I;`	C|I satisfies the Roman condition, but Sonata_Form appears only in MAIN and fails the global-tag condition.

Query 3

```
--globaltag:<DTC:C,a>
```

Result	Concrete source	Reason
Match	`{GLOBAL_TAGS}</DTC:a,C> C|I;`	The global pair a,C is the same unordered pair as C,a.
No match	`<DTC:a,C>C|I;`	A local DTC satisfies an event condition but not this metadata-only filter.

8.14 Form and DTC state

Syntax

```
<Pop_Form:Verse> / <DTC:M,m>
```

A form condition tests the implicit or explicit form state on the actual traversal.
The termination marker FORM_END is excluded from active search targets.
A double-tonic complex condition (DTC) tests an unordered pair of tonal references.
For example, C,a and a,C select the same pair.
Each member can use an exact reference, M, m, * or ?.
M,m selects one major and one minor reference in any key relationship.
TC and Roman conditions must be specified separately when required.
A global form classification or global DTC also supplies the active state for event queries.
Use –globaltag to require that it originates in metadata.

Query 1

```
<Pop_Form:Verse>
```

Result	Concrete source	Reason
Match	`</Pop_Form:Verse>C|I;G|V;`	The implicit Verse marker establishes an active state for C|I and the following G|V, so both events match.

Query 2

```
<DTC:C,a>
```

Result	Concrete source	Reason
Match	`<DTC:a,C>C|I;Am|vi;`	The active pair a,C contains the same references as C,a. Both events inherit this unordered pair and match.
Match	`{GLOBAL_TAGS}</DTC:a,C> C|I;Am|vi;`	Both events inherit the global DTC pair a,C.

Query 3

```
<DTC:M,m>
```

Result	Concrete source	Reason
Match	`<DTC:D,b>C|I;`	The active pair D,b contains one ordinary major reference and one ordinary minor reference, satisfying M,m.
No match	`<DTC:C,G>C|I;`	The active pair C,G contains two major references and lacks the minor reference required by M,m.

Query 4

```
<DTC:*,*>
```

Result	Concrete source	Reason
Match	`<DTC:C,G>C|I;`	The two wildcards accept the two members C and G of the active DTC pair.
No match	`C|I;`	The source event has no active DTC pair, so neither wildcard has a pair member to match.

8.15 Seq/SeqInv list query

Syntax

```
<Seq:A2,*> / <SeqInv:?> / <Seq:A2*> / <Seq:A2?>
```

A Seq or SeqInv condition matches the complete active type list in its written order.
Seq and SeqInv select their respective tag families.
Exact items use the literal spellings in the rule configuration.
A ? matches one type, and a * matches one or more types.
An exact base-type item such as A2 matches only that literal type; a derived type requires its complete literal name.
Appending * or ? to a configured base type instead selects one member of its type family.
For example, A2* and A2? each match either A2 or one of its configured derived types, such as A2(+5/-4).

Both family forms consume exactly one list item.
These conditions apply to active event tags and cannot be used as GLOBAL_TAGS filters.

Query 1

```
<Seq:A2,*>
```

Result	Concrete source	Reason
Match	`<Seq:A2,D2>C|I;F|IV</Seq>;`	The active Seq list begins with A2, and the nonempty * matches the remaining D2. Both events lie in that tagged range.
No match	`<Seq:A2>C|I;F|IV</Seq>;`	The Seq list ends after A2, leaving no further type for the required nonempty *.

Query 2

```
<SeqInv:?>
```

Result	Concrete source	Reason
Match	`<SeqInv:A2>C|I;F|IV</SeqInv>;`	The active SeqInv list contains exactly one type A2, satisfying ?. Both events lie in that tagged range.
No match	`<SeqInv:A2,D2>C|I;F|IV</SeqInv>;`	The SeqInv list contains A2 and D2, while ? requires exactly one type in the complete list.

Query 3

```
<Seq:A2>
```

Result	Concrete source	Reason
Match	`<Seq:A2>C|I</Seq>;`	The complete active Seq list is the single exact type A2.
No match	`<Seq:A2(+5/-4)>C|I</Seq>;`	The derived type A2(+5/-4) has a different literal name from exact A2.

Query 4

```
<Seq:A2*>
```

Result	Concrete source	Reason
Match	`<Seq:A2(+5/-4)>C|I</Seq>;`	A2* selects one member of the configured A2 family, which includes the derived type A2(+5/-4).
No match	`<Seq:A3>C|I</Seq>;`	A3 belongs to a different type family and fails the A2* item.

Query 5

```
<Seq:A2?>
```

Result	Concrete source	Reason
Match	`<Seq:A2>C|I</Seq>;`	A2? selects one member of the A2 family, including the base type A2 itself.
Match	`<Seq:A2(+4/-3)>C|I</Seq>;`	The derived type A2(+4/-3) is one configured member of the A2 family and satisfies A2?.
No match	`<Seq:A2,D2>C|I</Seq>;`	The family item A2? consumes exactly one type; it cannot match the complete two-item list A2,D2.

8.16 Visible and hidden BPM settings

Syntax

```
{BPM:1/4=[90~120]}
```

A BPM condition selects an explicit BPM or BPMhidden setting attached to an event.
Inherited tempo and tempo derived from MM are outside this condition.
The beat numerator, beat denominator and tempo count each accept *.
An exact tempo may use a decimal value.
A tempo range [lo~hi] includes both integer endpoints and requires lo<=hi.
Both visible and hidden settings use BPM in the query.
The query setting name BPMhidden is unsupported.

Query 1

```
{BPM:1/4=[90~120]}
```

Result	Concrete source	Reason
Match	`{BPM:1/4=90}C|I;{BPMhidden:1/4=120}G|V;`	C|I carries BPM 1/4=90, and G|V carries BPMhidden 1/4=120. Both use the required beat unit and lie at included endpoints of 90~120.
No match	`{BPM:1/4=121}C|I;`	The explicit tempo 121 exceeds the query range upper endpoint 120.

Query 2

```
{BPM:*/*=*}
```

Result	Concrete source	Reason
Match	`{BPM:1/4=120}C|I;{BPMhidden:3/8=80}G|V;`	The query accepts explicit BPM and BPMhidden settings. Its three wildcards accept both 1/4=120 and 3/8=80.
No match	`C|I;`	C|I has no attached explicit BPM or BPMhidden setting, which this query requires.

8.17 Explicit metric modulation

Syntax

```
{MM:1/4=1/8} / {MM:*}
```

An MM condition selects the event carrying an explicit metric modulation setting.
The condition applies once at that event.
Later events inheriting the resulting tempo do not satisfy it through inheritance.
The components of the conversion can use *.
A complete value * selects any explicit MM.

Query 1

```
{MM:1/4=1/8}
```

Result	Concrete source	Reason
Match	`{BPM:1/4=120}C|I;{MM:1/4=1/8}G|V;`	G|V carries the exact explicit MM 1/4=1/8. The preceding C|I carries BPM only, so only G|V matches.
Match	`C|I;{MM:1/4=1/8}G|V;`	The explicit MM matches without any preceding numeric BPM.

Query 2

```
{MM:*}
```

Result	Concrete source	Reason
Match	`{BPM:1/4=120}C|I;{MM:1/4=1/8}G|V;{MM:1/8=1/2}C|I;`	The wildcard accepts MM 1/4=1/8 on G|V and MM 1/8=1/2 on the final C|I. The first C|I carries BPM only and falls outside the MM condition.
No match	`{BPM:1/4=120}C|I;`	The event carries BPM 1/4=120 but has no explicit MM setting.

8.18 NR sequences

Syntax

```
[NR:P?] / [NR:*]
```

An NR condition selects an explicit transformation annotation bound to an event.
Literal P, L and R strings and single N, S or H use the source grammar.
Within the transformation pattern, ? matches one step and * matches one or more steps.
For example, P? accepts PL but excludes a lone P.
The match concerns the stored transformation claim.

Query 1

```
[NR:P?]
```

Result	Concrete source	Reason
Match	`C|I;[NR:PL]Cm|i;`	The Cm|i event carries NR PL. The fixed P matches its first step, and ? matches its second step L.
No match	`C|I;[NR:P]Cm|i;`	The annotation P contains one step, while P? requires P followed by one more step.

Query 2

```
[NR:*]
```

Result	Concrete source	Reason
Match	`C|I;[NR:PLR]Cm|i;[NR:N]Db|bII;`	The Cm|i and Db|bII events carry explicit NR annotations PLR and N. The * accepts each complete nonempty annotation.
No match	`C|I;Cm|i;`	Neither event carries an NR annotation, so the required explicit transformation is absent.

8.19 M/MH constraints

Syntax

```
{M:G|V} / {MH:*|*}
```

An M or MH condition selects the corresponding explicit modulation event.
The target accepts a concrete ordinary or quoted reference, an ordinary-reference wildcard * or an ambiguous-reference wildcard "*".
Bare M and m mode wildcards are excluded from the target.
The relation accepts a basic Roman symbol or *.
An OR group can choose between complete M and MH query events.
These two settings exclude OR groups inside their values.

Query 1

```
{M:G|V}
```

Result	Concrete source	Reason
Match	`{M:G|V}D|ii&V{/M};G|I;`	The D|ii&V event carries explicit pivot modulation M with target G and relation V, satisfying both exact values.

Query 2

```
{MH:*|*}
```

Result	Concrete source	Reason
Match	`C|I;{MH:G|V}G|I;`	The G|I event carries explicit MH with target G and relation V. The two * values accept that target and relation.
No match	`C|I;{TC:G}G|I;`	The G|I event changes TC to G but carries no MH setting, so the required modulation type is absent.

Query 3

```
({M:G|V}||{MH:G|V})
```

Result	Concrete source	Reason
Match	`{M:G|V}D|ii&V{/M};G|I;`	The D|ii&V event satisfies the first OR branch {M:G|V}.
Match	`C|I;{MH:G|V}G|I;`	The G|I event satisfies the second OR branch {MH:G|V}.

8.20 Relative chord notes

Syntax

```
(0)m|*,(2)m|*,(4)|*
```

A relative note uses a parenthesized integer from -24 to 24 in place of an absolute chord note.
It can occupy a chord root, slash bass or the corresponding positions on either side of a polychord.
All relative notes in the query share one unknown offset modulo 12.
The integer gives a semitone position relative to that offset.
For example, roots (0), (2) and (4) request successive positions two semitones apart under one transposition.
Complete relative chords use the same final-bass and pitch-set equivalence as complete absolute chords.
For example, (0)m6|*,(2)m6|* matches Am7b5/C followed by Bm7b5/D, preserving the final-bass interval C to D.
Concrete and relative notes can be combined; concrete notes keep their fixed pitches.
The –notallowdifferentnotation option restores notation matching while retaining one shared relative offset.
Relative-note notation is available only in queries.

Query 1

```
(0)m|*,(2)m|*,(4)|*
```

Result	Concrete source	Reason
Match	`Cm|i;Dm|ii;E|III;`	With C as the shared offset, roots C, D and E occupy relative positions 0, 2 and 4. The first two chords also have the required minor quality, and E has the required major quality.
Match	`Dm|ii;Em|iii;F#|#IV;`	With D as the shared offset, roots D, E and F# occupy relative positions 0, 2 and 4. Their minor, minor and major qualities also satisfy the three query fields.
No match	`Cm|i;Dm|ii;F|IV;`	The first two roots establish C as the shared offset. The last root F is 5 semitones above C, while the final query requires position 4.

Query 2

```
(0)/(7)on(2)/(9)|*
```

Result	Concrete source	Reason
Match	`C/GonD/A|;`	With C as the shared offset, the written notes C, G, D and A occupy positions 0, 7, 2 and 9 in the required root and bass roles.
No match	`C/GonD/F#|;`	C, G and D fit positions 0, 7 and 2 with offset C. The final F# occupies position 6 and fails the required position 9.

Query 3

```
(0)m6|*,(2)m6|*
```

Result	Concrete source	Reason
Match	`Am7b5/C|;Bm7b5/D|;`	The two sets match minor-sixth chords with final basses C and D under one shared offset.
No match	`Am7b5/C|;Cm7b5/Eb|;`	The final basses C and Eb are three semitones apart, while the query requires two.

8.21 Final pitch-class set

Syntax

```
C[notes:0,4,7]|*
```

A [notes:...] query compares the complete final pitch-class set of a chord.
Each number is a semitone interval modulo 12 from the final root after slash and on operations.
The root can be a concrete note, a relative note or *.
A concrete root retains its specified note spelling; the set itself ignores constituent-note spelling and octave doubling.
For example, a C# root selector does not select a final root spelled Db.
The set contains at least one distinct integer from 0 to 11 in strictly increasing order.
Matching requires equality of the complete sets.
For example, C/E has final root E and set 0,3,8.
The [notes:...] form ends the chord field and cannot be followed by slash, add or on syntax.
The set excludes wildcards and OR, while an OR group may choose complete [notes:...] forms.
The –notallowdifferentnotation option leaves this explicit set query unchanged.

Query 1

```
C[notes:0,4,7]|*
```

Result	Concrete source	Reason
Match	`C|I;`	The chord C has final root C and complete relative pitch-class set 0,4,7, satisfying both parts of the chord query.
No match	`C7|IDM7;`	C7 has complete relative pitch-class set 0,4,7,10. The extra 10 makes it differ from the required complete set 0,4,7.

Query 2

```
E[notes:0,3,8]|*
```

Result	Concrete source	Reason
Match	`C/E|I6;`	The slash operation makes E the final root of C/E. The notes E, G and C then give the required complete set 0,3,8.
No match	`C|I;`	The chord C has final root C and relative set 0,4,7, while the query requires root E and set 0,3,8.

Query 3

```
*([notes:0,3,7]||[notes:0,4,7])|*
```

Result	Concrete source	Reason
Match	`C|I;Dm|ii;`	The root * accepts C and D. C has complete set 0,4,7, and Dm has complete set 0,3,7, so each satisfies one OR branch.
No match	`G7|V7;`	G7 has complete relative set 0,4,7,10, which equals neither listed three-note set.

Query 4

```
C#[notes:0,3,7,9]|*
```

Result	Concrete source	Reason
Match	`C#m6|;`	The final root is spelled C# and the complete relative set is 0,3,7,9.
No match	`Bbm7b5/Db|;`	The relative set is equal, but the final root is spelled Db rather than the requested C#.

8.22 Finite OR

Syntax

```
(A||B)
```

An OR group uses parentheses around alternatives separated by ||.
A match can satisfy any one alternative.
The enclosing grammar determines whether alternatives are field fragments, complete query events, permitted complete setting values or complete tag parameters.
Each event-level alternative occupies exactly one query position.
Groups contain nonempty alternatives and cannot nest.
A query with several groups satisfies one permitted alternative in each group.
One OR group within a chord field uses pitch-set equivalence when every resulting chord is concrete and contains no * or ?.
For example, (Cm6||G/C)|* accepts Am7b5/C and CM9no3 with final bass C.
A chord field containing wildcards or multiple independent OR groups matches notation.
Cm(7||9||11||13)*|* accepts the listed tension spellings and any valid later suffix, including no5.

Query 1

```
(C||G)|*
```

Result	Concrete source	Reason
Match	`C|I;G|V;`	C|I satisfies the exact chord branch C, and G|V satisfies the exact chord branch G.
No match	`Cm|i;`	The complete chord Cm equals neither exact alternative C nor exact alternative G.

Query 2

```
(*|I||*|V)
```

Result	Concrete source	Reason
Match	`C|I;G|V;`	C|I satisfies the single-event branch *|I, and G|V satisfies the single-event branch *|V. The query produces two separate windows.
No match	`G7|V7;`	The exact Roman V7 equals neither required alternative I nor V.

Query 3

```
{TC:(C||"C")}*|I
```

Result	Concrete source	Reason
Match	`C|I;{TC:"C"}C|I;`	The first C|I uses ordinary TC C, and the second uses ambiguous TC ”C”. The setting OR accepts both references, and both Roman fields are I.
No match	`{BN:1}{TS:4/4}{D:1}{TC:D} D|I;`	Roman I matches, but active TC D equals neither allowed reference C nor ”C”.

Query 4

```
(Cm6||G/C)|*
```

Result	Concrete source	Reason
Match	`Am7b5/C|;CM9no3|;`	Each event has bass C and the complete set of one concrete OR branch.
No match	`Am7b5|;CM9no3/G|;`	The final bass of each event differs from C.

Query 5

```
Cm(7||9||11||13)*|*
```

Result	Concrete source	Reason
Match	`Cm7|;Cm9|;Cm11|;Cm13|;Cm9no5|;`	The OR chooses a written tension and the suffix wildcard also accepts no5.
No match	`Cm6|;CmM7|;`	Neither 6 nor M7 begins with one of the listed tension spellings.

8.23 Complete-field NOT

Syntax

```
P!(N1||N2)
```

A NOT suffix !(N1||N2) excludes complete patterns from a positive chord or Roman field P.
Each field permits one suffix after the complete positive pattern, on the same side of |.
A chord exclusion appears before |, while a Roman exclusion appears after it.
Each exclusion is a complete concrete or wildcard pattern.
Concrete chord exclusions use the default final-bass and pitch-set equivalence unless –notallowdifferentnotation is present; wildcard exclusions match notation.
The positive pattern matches, and every main or alternative interpretation must avoid every exclusion.
Roman exclusions inspect all analysis layers.
The positive pattern determines shared relative offsets and tonal-path continuity.
Cm(7||9||11||13)*!(*no*)|* selects those written tension families and excludes every notation containing no, including Cm9no5.
NOT requires an actual harmonic field and cannot negate X.
Exclusions cannot contain relative notes or [notes:...] sets.
NOT groups cannot nest or appear inside an OR branch.

Query 1

```
Cm*!(*/*||*on*)|*
```

Result	Concrete source	Reason
Match	`Cm|i;Cm7|i7;`	Both Cm and Cm7 satisfy the positive Cm* pattern. Both avoid the excluded slash pattern */* and polychord pattern *on*.
No match	`Cm/Eb|i6;CmonAm|;`	Both chords satisfy Cm*. Cm/Eb matches the exclusion */*, and CmonAm matches the exclusion *on*, so both are rejected.

Query 2

```
*|V*!(V7)
```

Result	Concrete source	Reason
Match	`G|V;G/B|V6;`	Roman V and V6 both satisfy V*. Neither equals the exact exclusion V7.
No match	`G7|V7;`	Roman V7 satisfies the positive V* pattern but also equals the exclusion V7, so the field is rejected.

Query 3

```
Cm(7||9||11||13)*!(*no*)|*
```

Result	Concrete source	Reason
Match	`Cm7|;Cm9|;Cm11|;Cm13|;Cm7/G|;`	All five notations satisfy the positive pattern and contain no no keyword.
No match	`Cm9no5|;Cm11no5|;`	The chord-side NOT excludes both no5 suffixes.

Query 4

```
*!(Cm6)|*
```

Result	Concrete source	Reason
Match	`Am7b5|;`	The final bass A differs from C, so this chord is not excluded as an equivalent Cm6.
No match	`Am7b5/C|;`	The concrete exclusion uses the same final bass and complete set as Cm6.

8.24 Special events and unsupported controls

Syntax

```
X / 0 / N.C. / Clus / ClusWK / ClusBK
```

Complete-event queries accept X, 0, N.C., Clus, ClusWK and ClusBK.
An X inside a harmonic query field selects that field's placeholder.
For example, X|V requires a chord-field placeholder and an exact Roman V.
Bare - and field-level - cannot be queried directly.
Sequence matching skips continuations as transparent events.
Event queries exclude BN, D, T, TEMPO and LB settings, along with COMMENT and F annotations.

Query 1

```
X
```

Result	Concrete source	Reason
Match	`X;`	The source contains a complete placeholder event X, satisfying the complete-event query X.

Query 2

```
ClusWK
```

Result	Concrete source	Reason
Match	`ClusWK;`	The complete source event ClusWK exactly matches the requested white-key cluster.

Query 3

```
X|V
```

Result	Concrete source	Reason
Match	`X|V;`	The chord field contains the required placeholder X, and the Roman field equals the required V.
No match	`G|V;`	Roman V matches, but the actual chord G fails the required chord-field placeholder X.

8.25 Hard limits

A query permits at most 4096 source characters, 32 query events and 16 global-tag options.
Each harmonic field permits at most 256 characters and 16 wildcards.
OR permits at most 8 alternatives per group, 8 groups per query and 32 alternatives in total.
A query permits at most 32 NOT groups, with at most one per field.
OR groups inside NOT count toward the same OR limits.
The positive and exclusion patterns share the field-length and wildcard limits.
The parser's deterministic work score must be at most 512.
These limits govern query admission, while result-window counts depend on the matched source.

9 Rendering rules

9.1 Time order and semantic layers

MStruct presents an analysis as a sequence of systems, where a system is one horizontal line of the rendered score.
Within a system, measures and timed slots advance from left to right.
At each position, different kinds of information occupy designated vertical regions.
The horizontal arrangement communicates musical order and duration, while the vertical arrangement separates harmony, structural annotations and explanatory material.
A reader can follow the harmonic progression across the page and inspect the related information above and below the same position.
This chapter records these display conventions and their layout relationships.
The syntax and musical meanings of the displayed items are defined in Chapters 2 to 6.

9.2 The vertical grid, from top to bottom

Each system uses a shared vertical grid.
The following list gives the visual order of its regions.
Optional regions are allocated for the whole system when visible content requires them.
Measures, slots and omission marks in that system share the resulting row positions.

1. **Top spacing**

   Space above the highest symbol separates adjacent systems and accommodates raised measure numbers.

2. **Form-symbol row**

   Boxed form symbols such as `[F:A||]` occupy an optional row above the tempo markings.
   Their position identifies a formal boundary or section.

3. **Tempo markings and raised measure numbers**

   Visible TEMPO, MM and BPM markings form a vertical stack.
   A measure number displaced above a volta bracket can share this height band at a different horizontal position.
   A system with endings alone reserves space for the displaced numbers.

4. **Measure-number and time-label reference row**

   Ordinary measure numbers and written time labels such as `{T=0:15}` share a reference level above the music.
   Volta brackets are anchored to this region and span their applicable measures or slots.

5. **Musical content**

   Time signatures, barlines, chord symbols, Roman numerals, standalone events and repeat boundaries occupy the central region.
   The final barline `{||END}` is drawn with a thin left stroke and a thick right stroke.
   The harmony region can contain several aligned subrows.

6. **Duration row**

   Duration labels appear below the musical content.
   The renderer offers fractional and note-value forms for these labels.

7. **Comment row**

   Comment markers and the anchors of comment ranges occupy a separate optional row.
   When a system has no visible comments, this whole row and its additional gaps collapse.

8. **Analytical-tag lanes**

   Ordinary and semantic tags appear below the comments.
   Concurrent tag ranges use separate lanes, with their labels and continuation lines aligned within each lane.
   Visible tag lanes are packed together for the system.

9. **Bottom spacing**

   Space below the last occupied row separates this system from the next one.

9.3 Form, tempo and the reference row

The boxed form-symbol row sits above the tempo group.
Within a slot, tempo items are stacked in the order TEMPO, then MM relations in source order, then BPM.
The stack aligns to the bottom of its row, so a single tempo item shares the bottom alignment of a larger stack.
`BPMhidden` contributes to playback state while its marking remains hidden.
A written time label `{T=...}` occupies the measure-number reference row.
A time label alone uses the ordinary system layout, while visible TEMPO, MM or BPM content activates the tempo row.
A volta bracket uses the measure-number region as its anchor.
Only measure numbers horizontally covered by the bracket move above its horizontal line.
Their clearance follows the bracket geometry, so adding tempo items preserves the relation between the bracket and these numbers.

9.4 The internal arrangement of a harmonic event

When both fields are present, the chord-symbol field C appears above the Roman field R.
Slots in the same system share the relevant harmony-row heights, including positions where one field is empty.
A modulation can display source and destination Roman interpretations on separate subrows within the musical region.
The main Roman symbols provide the alignment reference, with tonal prefixes and supplementary explanations arranged around them.
Chord and Roman typography distinguishes roots or degrees, qualities, extensions, alterations, bass information and alternative readings.
These components retain their internal relationships when the available width or display scale changes.
Time signatures occupy space at their musical boundary.
Barlines and repeat boundaries cover the musical region, while durations and analytical annotations occupy their own rows.

9.5 Duration, comments and analytical tags

The duration row associates visible duration labels with the musical positions above it.
Fractional and note-value displays express the same underlying timing information.
The comment row displays compact numbered markers linked to the prose in the COMMENT section.
A comment range connects its start and end positions along this row.
The tag region identifies analytical features through labels and, where the tag uses a range, continuation lines.
A point tag is attached to its own position.
Boxed form symbols `[F:...]` use the upper form row, while form tags `<...>` are placed with the analytical tags.
The presence of a comment anywhere in a system determines whether its comment row is needed, and all slots use the resulting layout.
When that row is absent, the tag region moves upward as a whole.

9.6 Ranges and continuity across systems

Volta brackets, comment ranges and tag ranges are attached to musical start and end positions.
The renderer locates these positions after horizontal layout and draws the visible segments through the systems that the range crosses.
A range crossing a line break continues through the end of one system and resumes in the next.
Its final segment reaches the corresponding endpoint.
The relationship between the range and the musical events therefore survives reflow at a different page or window width.
Within each system, tag lanes provide consistent heights for labels and range lines.
This combines a continuous analytical scope with a score that can wrap into several readable lines.

9.7 Horizontal spacing and line breaking

Each slot receives a natural width from its duration and the space needed by its visible content.
Longer durations contribute more rhythmic space, while the symbols and annotations impose a readable minimum.
Tonal prefixes, time signatures and repeat signs add the clearance needed at their positions.
Extra space in a justified system is distributed using duration-based stretch weights.
Visual spacing thus combines rhythmic proportion with the width requirements of the notation.
Complete measures normally remain together when the renderer chooses line breaks.
A measure wider than an entire system can be divided between slots and continued on subsequent systems.
An individual slot remains intact.
An explicit line break ends the current system, and an omission mark `...` closes the system containing it.
The final system and a system ending with an omission mark retain their natural spacing.
Paged score export places page boundaries between complete systems.

9.8 Visibility and reflow

The renderer groups tempo markings, form information, analytical tags, comments and Neo-Riemannian annotations into selectable display layers.
Visible content determines the space required by the corresponding rows and lanes.
Hiding a layer changes the displayed material and can reduce the width or height of the score.
The underlying events, analytical relationships and playback timing remain in the document.
Changes in window width, display scale or selected layers trigger layout using the same slot order and semantic row relationships.
The shared renderer applies this organisation to the editor, viewer and exported score.

Licence and fonts

The document text, original examples and rule descriptions are licensed under CC BY 4.0.
Embedded fonts retain their respective licences: Latin Modern uses the GUST Font License; Fandol uses the GPL with its font exception; Academico, Bravura, Crimson Pro and MStruct Text Accidentals (derived from Noto Music) use the SIL Open Font License 1.1.
Some illustrations include licensed subsets of Windows Arial, Cambria Math and Microsoft YaHei.
These fonts are embedded in the PDF for document use; the document licence does not relicense the font software.

Creative Commons Attribution 4.0 International (CC BY 4.0)

Citation

Polymath9. (2026). MStruct: Technical Note (Version 1.0.0).
`https://doi.org/10.5281/zenodo.23637500`.

References

1. Celeca. Notations of simai. `https://w.atwiki.jp/simai/pages/1003.html`. Accessed: 2026-10-25.
2. The LaTeX Project. An introduction to LaTeX. `https://www.latex-project.org/about/`. Accessed: 2026-10-25.

Part II
中文

摘要

MStruct 是由 Polymath9 开发、受 simai 记法（Notations of simai）和 LaTeX 启发的基于文本的音乐结构描述与呈现系统。
它按时间顺序记录和声与结构分析。
它以小节和带时值的槽位组织和弦记号、罗马数字、分析标签及音乐设置。
本文定义文档模型，并按条目说明基础事件、和弦记号、罗马数字、标签、录音时间轴和搜索查询。
语法说明配有源码示例。
搜索部分逐条列出具体匹配与不匹配的片段，并解释结果。
本文还说明规则包的身份、兼容更新和历史解释方式。
渲染规则章记录谱面的纵向 Grid、对齐方式、范围连续性与横向布局。
英文全文在前，中文全文在后。

关键词： MStruct；音乐分析；和声分析；文本记谱；查询语言；数字音乐学

作者与 AI 辅助说明

本文在结构整理、起草、翻译、文字修订、一致性检查和核验脚本编写中使用了包括 OpenAI Codex 在内的生成式 AI 工具。
Polymath9 主导 MStruct 的语言设计和音乐学定义，并作出最终编辑决定。
技术说明以实现代码和规则表为依据。
Polymath9 为本文作者，并对本文内容承担责任。

1 MStruct 定义

1.1 定义与来源

MStruct 是由 Polymath9 开发、受 simai 记法（Notations of simai）[1] 和 LaTeX [2] 启发的基于文本的音乐结构描述与呈现系统。
它用于编写、呈现和检索音乐分析。
其核心是一种音乐批注语言，按时间顺序记录对和声与结构的解释。
这种分析把作品缩编为选定的和声关系与曲式组织。

1.2 用途与局限

MStruct 以和声事件为骨架组织音乐批注。
其主要用途及表示范围的局限如下。

1. **分析呈现与演示**

   MStruct 编辑器将写好的分析渲染为排版清楚、便于阅读的分析谱面。
   分析谱面及其播放演示可用于制作演示视频。

2. **音乐实例检索**

   MStruct 的各个组成部分遵循明确的语法约束，尤其是和弦记号与罗马数字。
   随着分析资料积累，学习者和研究者可以检索其中记录的技法与和弦进行，寻找用于学习和研究的音乐实例。

3. **分析解释的整理与后续利用**

   MStruct 本质上是一种音乐批注语言。
   结合 COMMENT 功能，它将分析者的主观解释与文字批注组织为格式化的文本文件。
   这些文件未来可能用于统计处理，以及与大型语言模型结合的工作流程。

4. **表示范围的局限**

   以和声事件为骨架的表示方式不能完整记录独立声部的对位关系、音列组织或声谱演变。
   它也不能表达申克分析所需的多层次声部进行结构。
   当某一乐段具有可记录的和声层面时，仍可记录该层面；上述其他组织关系则超出其表示范围。

1.3 应用范围

MStruct 的设计主要服务于和弦记录与和声 / 罗马数字分析，并以和声事件作为时间骨架，进一步记录调性、音阶 / 调式与曲式结构。
对于不同音乐风格及其组织方式，能够记录的有效信息量有所不同。
以下按 MStruct 设计时的优先级从高到低排列。

1. **以共晓时期和声结构为基础的音乐**

   这类音乐包括共晓时期（Common Practice Period）的绝大多数古典音乐，以及基于相关和声结构、以和声发展为导向的大多数现代流行音乐、爵士乐与电子音乐。
   这是设计时最优先服务的音乐类型。
   MStruct 通常可以为其记录较多有效的分析信息，并纳入档案库的搜索索引。

2. **功能性较弱或调性中心模糊的和声**

   这类音乐仍主要使用三度或四度叠置和弦，但和弦功能性较弱，或调性中心较为模糊。
   例如晚期浪漫主义与印象主义古典音乐、调式爵士（Modal Jazz），以及采用类似和声结构的其他音乐。
   和弦记号、曲式结构及文字注释通常仍能保存有用的信息。
   罗马数字分析在其中的作用减弱，部分乐段可能不适用。

3. **以节奏与音色变化为主要推进方式的流行音乐和电子音乐**

   这类音乐可能仍处于调性和声框架中，但和弦进行较为简单，主要音乐信息体现在节奏或音色变化上。
   MStruct 可以记录整体和声结构，作为可检索的和声索引和简单实例。
   对于推动音乐发展的其他非和声特征，它能提供的定量信息较为有限。

4. **具有精细对位关系的调性复调音乐**

   例如巴赫和贝多芬的部分作品。
   MStruct 可以记录其中的和声变化。
   由于和声缩编不编码各个独立的旋律线条，对位法分析所需的信息会有所缺失。

5. **主要由横向线条或音列组织的音乐**

   这类音乐包括格里高利圣咏、部分现代复调音乐，以及纵向结构大体处于传统三度 / 四度和声体系之外的序列主义音乐。
   音列的次序及其横向展开对音列组织十分重要，单靠和弦记号序列不能保留这些关系。
   无调性本身并不排除 MStruct 的所有用途。
   部分自由无调性作品可能在尽量保持十二个音权重相近的同时使用三和弦；其中的和声层面仍可能进行有限的定量分析。

6. **主要由音色与声谱变化组织的音乐**

   这类音乐以音色或声谱的演变作为主要组织过程，例如频谱音乐中的相关实践。
   和声事件批注不能捕捉对这些过程作定量分析所需的主要信息。

7. **十二平均律之外的音乐**

   这类音乐包括异律音乐（Xenharmonic Music），例如 Jacob Collier 的部分作品。
   MStruct 的十二音框架不能保留这类音乐特有的调律与微分音关系。
   对于最后三类音乐，其作用一般限于整理描述性元数据，以及标注能够表示的局部调式或音阶。

1.4 文档分区

语法

```
METADATA: ... END_METADATA / MAIN: ... END_MAIN / COMMENT: ... END_COMMENT
```

元数据分区（`METADATA:` / `END_METADATA`）保存作品名称、文档选项等基本资料。
分析分区（`MAIN:` / `END_MAIN`）按时间顺序记录音乐分析。
注释分区（`COMMENT:` / `END_COMMENT`）定义带编号的文字注释，供分析正文引用。
完整文件包含 METADATA 与 MAIN，后面可接 COMMENT 分区。
分区标记独占一行。
后文主要讨论 MAIN，其他范围会在对应章节说明。
下面的例子展示分区边界，其中的正文设置见第 2 章。

示例

```
METADATA:
{SONG_NAME}Example
{IS_STABLE_TEMPO}False
END_METADATA

MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;
END_MAIN
```

1.5 分析正文的组成

MAIN 用小节（以 ; 结束）组织音乐时间。
每个小节包含依次排列的槽位（用 , 分隔）。
槽位是安放音乐内容的一段时间（长度由 {D:} 设置）。
音乐事件描述槽位中的和声（C|I）、休止（0）或延长（-）。
设置（{TYPE:value}）指定小节号（{BN:}）、拍号（{TS:}）、时值（{D:}）、速度（{BPM:}）或调性参照（{TC:}）等条件。
调性参照（{TC:}）提供解释和声级数时采用的调。
控制记号（如 {||} 或 {||:} / {:||}）标记边界，或安排反复等演奏顺序。
标注（如 [F:A||] 或 [NR:P]）为某个位置补充信息。
分析标签（如 </PAC> 或成对的 <Tonality> / </Tonality>）为某个位置或范围内的音乐特征命名。
一个和声事件（如 C|I）可以把两种解释放在独立字段中。
C 字段（如 [C:C] 或 C|）用根音和和弦类型来命名和弦。
R 字段（如 [R:I] 或 |I）使用罗马数字来描述和声相对于调性参照的级数及性质。

1.6 字符与嵌套

分隔符

```
{} [] <> , ; |
```

结构标点使用半角 ASCII 字符。
空白和换行可以分隔完整记号。
分号结束普通小节。
成对括号列表内部的逗号分隔该字段的项目。
结束标签使用 /。
和声字段内部的两个反斜杠引入附加解释，具体写法见第 3 章。
设置类型名接受大小写字母。
音名、标签名和许多设置值要求使用各自定义中的大小写。

1.7 规则包

规则包是 MStruct 使用的一组配置表。
这些表定义可用的音乐记号、记号的含义及输入工具箱。
第 1 章至第 8 章的规则说明以截至 2026 年 11 月 4 日的配套规则表为依据。

1.8 格式、规则与应用版本

字段

```
MSTRUCT_FORMAT_VERSION / MSTRUCT_PROFILE_ID / MSTRUCT_PROFILE_HASH /
↪  AUTHORING_APP_VERSION
```

文档记录四个系统字段。
格式版本（{MSTRUCT_FORMAT_VERSION}）标识文档语法。
规则标识（{MSTRUCT_PROFILE_ID}）标识一套规则及其版本系列。
内容哈希（{MSTRUCT_PROFILE_HASH}）标识精确规则内容。
应用版本（{AUTHORING_APP_VERSION}）记录写入文件的软件。
文档语法、规则与应用各有自己的版本。

示例

```
{MSTRUCT_FORMAT_VERSION}1
{MSTRUCT_PROFILE_ID}org.mstruct.official/rulesets/1.0.0
{MSTRUCT_PROFILE_HASH}sha256:<64 lowercase hexadecimal digits>
{AUTHORING_APP_VERSION}<application version>
```

1.9 完整规则包

清单字段

```
manifestVersion = 2
```

规则包包含文件清单（manifest）与配置文件。
清单按固定顺序列出十二张 CSV。
十张表定义音乐语义：和弦表 special-chords；罗马数字表 roman-basic、roman-advanced、roman-figured-bass、roman-special；标签表 tags、cadence-tags、tonal-tags、sequence-tags、form-tags。
两张输入表 toolbox-menus、toolbox-events 定义输入工具箱。
文件名均带.csv 扩展名。
校验覆盖完整文件集、各表内容及表间引用。
旧 manifest v1 包保留原十一表封装。

1.10 内容哈希与完整性

取值格式

```
sha256: + 64 lowercase hexadecimal digits
```

规则包哈希使用 SHA-256，写作 sha256: 后接 64 位小写十六进制数字。
CSV 规范化去掉开头 BOM、统一 LF，并只保留一个末尾换行。
行顺序保持原样。
哈希根据规范封装的清单与文件内容计算。
加载时重新计算哈希，并与声明值核对。
它为完整性校验、缓存和历史精确读取标识内容。
规则标识与版本属于独立身份信息：相同内容可以具有不同身份，同一兼容版本系列也可以包含多个内容哈希。
仅凭哈希不同不能判定不兼容。

1.11 同版本兼容

规则版本标识一个向后兼容的系列。
新增内容及显示或排序调整可以留在同一系列，并保持已有定义与行为。
兼容性分别考虑音乐语义与输入行为。
删除或重定义既有规则必须使用新的大版本。
在这一约定下，文档格式及完整规则标识（含版本）相同、且通过校验的完整规则包可以用于编辑。

1.12 获取规则包

不同提供者可以分发各自的规则包与版本。
MStruct 档案库的规则目录地址为 https://mstruct-archive.com/api/mstruct-profiles。
同步索引地址为 https://mstruct-archive.com/api/mstruct-profiles?index=1。
使用 profileId 与 hash 查询参数可以请求精确构建。
编辑器可以从该服务同步目录并下载规则包，访问遵循该服务当前的访问策略。
下载或导入的规则包均须通过相同的内容与完整性校验后才能使用。
指定 profileId 与 hash 的请求返回完整规则包 JSON。

1.13 不兼容的规则版本

对既有定义或行为的不兼容改动要求使用新的规则大版本。
既有文档采用该版本时需要显式迁移。
搜索可以把音乐含义兼容的规则归入同一语义族。
语义不兼容建立新的语义族；仅输入行为存在差异时，仍可保留同一音乐语义族。

1.14 编辑时采用兼容更新

编辑器可以在工作副本中采用可用且通过校验的同版本规则包。
这一选择更新工作副本的规则身份，并保留分析源码。
保存时记录选定的哈希。
本地独立编辑器可以使用已安装或内置的完整规则包，无须下载文件原先使用的精确构建。
编辑会话沿用选定规则包。

1.15 历史精确解释

保存的规则标识与哈希共同标识某次文档修订所用的解释规则。
保留对应规则包，可以复现该次解释。
编辑时选择兼容规则，与使用精确规则读取已保存修订，分别服务于不同用途。

1.16 离线规则与应用能力

规则包提供配置数据，应用更新提供新增编译器语法。
使用新增语法还需要具备相应实现的应用版本。
本地独立编辑器的安装包包含完整且通过校验的规则包，供离线运行。
其他已验证规则包可以安装或导入。
本机没有文档格式及完整规则标识匹配的规则包时，须先取得匹配包再编辑。

1.17 规则与派生搜索索引

搜索索引由分析修订及其精确规则身份派生。
索引格式具有独立于规则版本的实现版本。
索引格式改变时，可以保留分析及其规则身份并重新生成索引。
语义查询可以在兼容规则族内解释，再与每份分析自身的索引匹配。

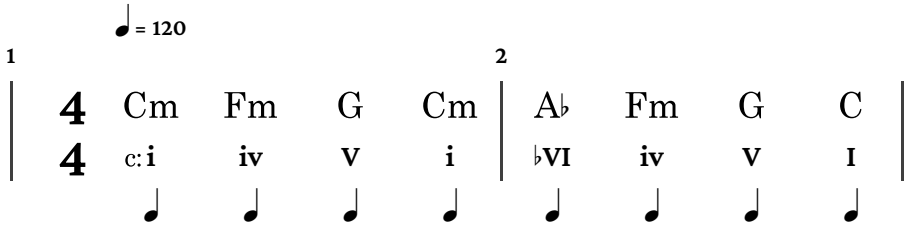
2 基本语法

2.1 一个完整 MAIN 示例

BN 设置小节号，TS 设置拍号。
D 以全音符为单位指定每个槽位的时值。
TC 设置调性参照，BPM 设置播放速度的数值。
示例从第 1 小节开始，采用 4/4 拍，以 c 小调为参照，并按每分钟 120 个四分音符播放。
简写设置 `{1/4}` 为每个槽位指定四分音符时值。
逗号推进一个槽位。
分号关闭最后一个槽位并结束小节。
竖线分隔 C 和 R 字段，因此 Cm|i 给出和弦 Cm 与罗马数字分析 i。
后续条目分别解释各项设置及可用的字段形式。

示例

```
{BN:1}{TS:4/4}{BPM:1/4=120}{TC:c}
{1/4}
Cm|i,Fm|iv,G|V,Cm|i;
Ab|bVI,Fm|iv,G|V,C|I;
```



2.2 BN：小节号

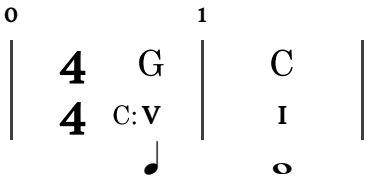
语法

```
{BN:n}
```

BN 是 Bar Number（小节号）的缩写。
首个事件前必须定义 BN。
BN 使用非负十进制整数来指定当前源码小节号。
后续普通小节的小节号自动递增。
后续 BN 设置写在小节边界。
BN 为 0 表示预备拍小节。
预备拍小节首次经过时的累计时值必须严格小于数字 TS。

示例

```
{BN:0}{TS:4/4}{D:1/4}{TC:C}G|V;
{D:1}C|I;
```



2.3 TS：拍号

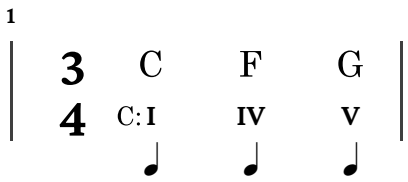
语法

```
{TS:p/q} / {TS:adlib}
```

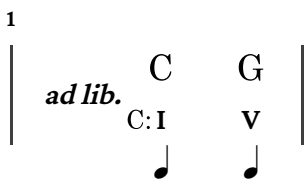
TS 是 Time Signature（拍号）的缩写。
开头必须定义 TS，包括 BN 为 0 的情况。
分子和分母接受任意正整数。
普通有拍小节必须恰好累计为 p/q 个全音符。
TS 在小节边界改变。
精确小写值 `adlib` 允许各小节使用可变总时值。
自由拍子预备拍小节同样遵循这一可变时值规则。
自由拍子段落仍须定义 D，由 D 决定每个槽位的时值。

示例

```
{BN:1}{TS:3/4}{D:1/4}{TC:C}C|I,F|IV,G|V;
```




```
{BN:1}{TS:adlib}{D:1/4}{TC:C}C|I,G|V;
```



2.4 BPM：节拍速度

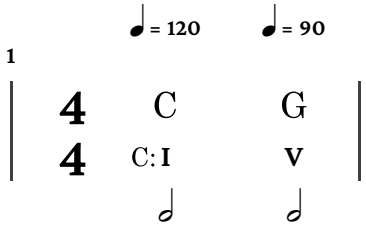
语法

```
{BPM:p/q=v}
```

BPM 是 Beats Per Minute（每分钟拍数）的缩写。
拍单位 p/q 是以全音符为单位的正有理时值。
v 是有限正十进制数，表示每分钟的拍数。
1/4=120 表示每分钟 120 个四分音符。
速度设置持续到后续速度变化。
BPM 设置作用于所在的整个槽位，在同槽位和声事件之前及之后书写均可。
时值为 d 个全音符的槽位耗时 `60000*d/((p/q)*v)` 毫秒。
稳定速度选项（元数据中的 `{IS_STABLE_TEMPO}True`）根据槽位时值和数值速度设置计算播放时间。
因此，每个参与这种播放的事件槽位都必须有确定的数值速度。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}{BPM:1/4=120}C|I,{BPM:1/4=90}G|V;
```



2.5 BPMhidden：隐式速度

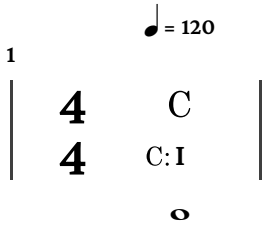
语法

```
{BPMhidden:p/q=v}
```

BPMhidden 使用与 BPM 相同的数值语法来设置播放速度。
谱面显示 BPM 的节拍速度标注，并隐藏 BPMhidden 的标注。
两种设置都改变当前播放速度。
同槽位的设置按源码从左到右应用。
最后的值决定播放速度，较早的 BPM 标注仍可保留。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}{BPM:1/4=120}{BPMhidden:1/4=119.5}C|I;
```



上面的 MStruct 在预览时，实际 BPM 为每分钟 119.5 个四分音符，可见标注仍为 120。

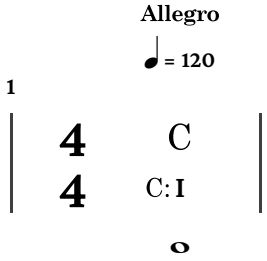
2.6 TEMPO：文字速度记号

语法

```
{TEMPO:text}
```

TEMPO 在 BPM 行上方附加一段非空文字标记。
Allegro 等文字提供速度术语。
稳定速度文件需要另外通过 BPM 或 BPMhidden 设置速度数值。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}{TEMPO:Allegro}{BPM:1/4=120}C|I;
```



2.7 TC：调性参照

语法

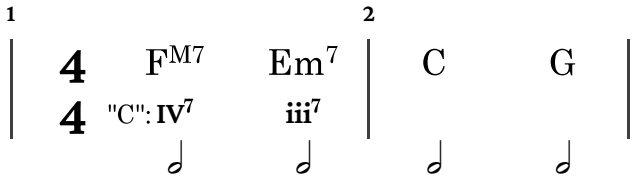
```
{TC:C} / {TC:c} / {TC:"C"} / {TC:None}
```

TC 是 Tonal Center（调性中心）的缩写。
普通 TC 使用 A-G 表示大调，使用 a-g 表示小调。
音名字母后可以接单个 b 或 #。
成对半角双引号标记模糊中心或仅作参照的中心。
带引号的中心保留大小调性质，供罗马数字解释使用。
调性参照在初始时保持未定义。

参照建立后会跨后续事件持续，直到设置改变它。
后续 TC 从对应事件边界开始改变参照。
精确值 None 显式清除参照。
恢复有效调性参照后，正文可以继续使用罗马数字字段。
第 3 章的转调条目说明各自对参照的要求。
使用罗马数字字段的槽位必须具有已定义且非 None 的调性参照。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:"C"}FM7|IV7,Em7|iii7;
{TC:None}C|,G|;
```



2.8 D：槽位时值

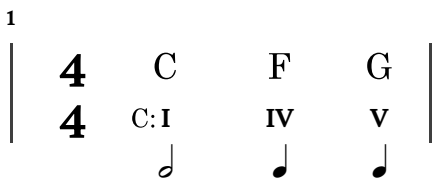
语法

```
{D:p/q} / {p/q} / {n}
```

D 是 Division（划分）的缩写。
D 用正整数或正分数表示槽位相对于全音符的时值。
{1/4} 是 {D:1/4} 的简写。
{1} 给出一个全音符的时值。
首个槽位前必须定义 D。
D 的值跨逗号和普通小节线持续生效。
后续 D 改变当前及后续槽位的时值。
例如，D 为 1/8 时，每个槽位在任何 TS 下都占一个八分音符。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,{1/4}F|IV,G|V;
```



2.9 逗号、分号与空槽位

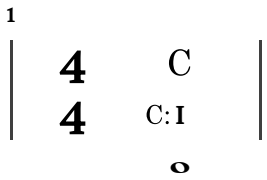
语法

```
, / ;
```

逗号按当前 D 关闭槽位。
分号关闭当前槽位并结束小节。
换行充当记号之间的空白。
普通末小节也必须使用分号结束。
逗号分隔出的空槽位把前一事件延长一个 D。
字段成对列表内部的分隔符分隔该字段的项目。
紧接在分号前的逗号会额外创建一个槽位。
终止小节线 {||END} 写在最后小节的分号前，并在该小节结束后终止 MAIN。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I,,,;
```



2.10 和声事件字段

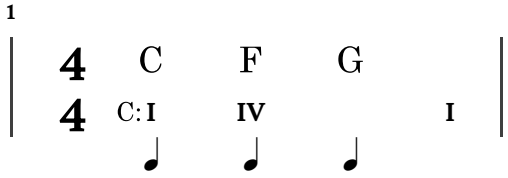
语法

```
[C:chord|R:roman] / chord|roman / chord| / |roman
```

显式形式包含一个 C 字段、一个 R 字段或两者。
每个写出的字段都必须有非空值。
每个简写事件都必须包含竖线。
C| 只提供和弦字段，|I 只提供罗马数字字段。
[C:C] 与 [R:I] 是这两种单字段事件的合法显式形式。
罗马数字必须有调性参照。
两个字段同时存在时，各自给出对同一位置的独立解释。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}[C:C|R:I],F|IV,[C:G],[R:I];
```



3 更多事件语法

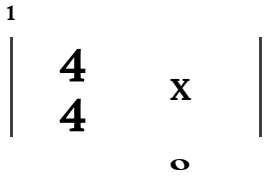
3.1 X：占位事件

语法

```
X
```

裸写大写 X 是完整的占位事件。
它为分析者暂不便明确指定的和声保留一个槽位。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
X;
```



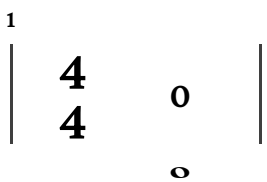
3.2 0：休止

语法

```
0
```

裸写数字 0 表示休止。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
0;
```



3.3 -：延长

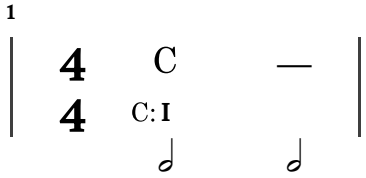
语法

```
-
```

裸写半角连字符把前面的音乐内容延续到当前槽位。
裸写形式延续整个事件，C:- 和 R:- 则标记单个字段。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,-;
```



3.4 N.C.：无和弦

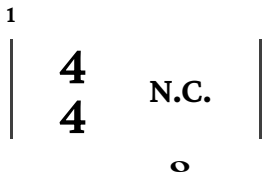
语法

```
N.C.
```

N.C. 记录无和弦事件。
它的精确拼写包含两个句点。
当前调性参照继续生效。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
N.C.;
```



3.5 Clus、ClusWK 与 ClusBK

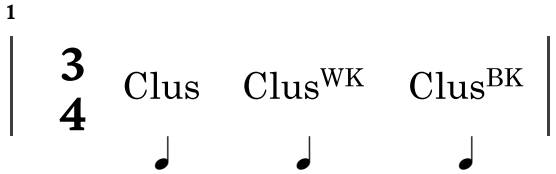
语法

```
Clus / ClusWK / ClusBK
```

Clus 是表示各种类型音块的一般事件。
特别地，ClusWK 表示白键音块，ClusBK 表示黑键音块。
这些事件对音块进行分类，不指定根音或完整音高列表。
它们的拼写必须保持此处的大小写。

示例

```
{BN:1}{TS:3/4}{D:1/4}{TC:None}Clus,ClusWK,ClusBK;
```



3.6 字段级 X 与 -

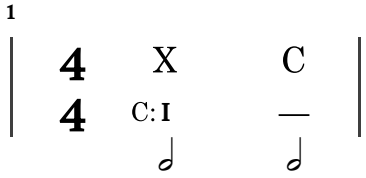
语法

```
[C:X|R:I] / [C:C|R:-]
```

C 或 R 的完整值为 X 或 - 时，该记号标记对应的单个字段。
C:- 延续前面的和弦字段，R:- 延续前面的罗马数字字段。
特别地，R:- 也保留第 5 章定义的活动数字低音链。
R:X 表示未指定该罗马数字字段；活动数字低音链须在它之前结束。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}[C:X|R:I],[C:C|R:-];
```



3.7 次要和声事件

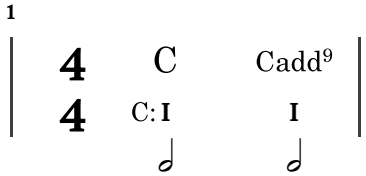
语法

```
'C|I' / '[C:C|R:I]'
```

成对半角单引号把和声事件标记为次要事件。
单引号必须包住完整事件。
事件内每个写出的字段都必须包含和弦或罗马数字记号。
次要和声事件通常记录根音乃至基础和弦类型保持不变，而旋律或内声部进行使瞬时纵向和声产生微小变化的情况。
搜索默认跳过次要事件，使用 `--notignore2nd` 可以将其纳入。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,'Cadd9|I';
```



3.8 同领域附加解释

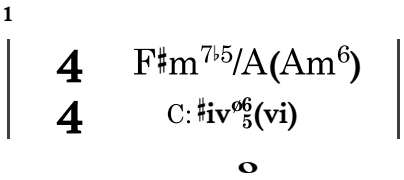
语法

```
C:C\\Am / R:I\\vi
```

恰好两个连续反斜杠为字段增加一个完整的附加解释。
两侧必须是同领域的非空合法记号。
字段标记不能附加解释。
每个记号分别获得独立解释。
数字低音链以主罗马数字记号为准。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F#m7b5/A\\Am6|#ivho65\\vi;
```



3.9 NR：变换标注

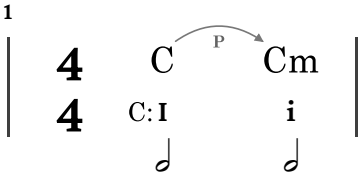
语法

```
[NR:P] / [NR:PLR] / [NR:N]
```

NR 是 Neo-Riemannian Transformation（新里曼变换）的缩写。
NR 记录分析者对相邻和弦之间变换关系的判断。
NR 的名称必须使用大写字母。
它的值由 P、L、R 中的 1-6 个字符组成，或使用单个 N、S、H。
NR 写在同槽位的 C 和弦记号前。
每个槽位最多允许一个 NR。
前一音乐事件必须包含 C 和弦记号或使用裸写延长符。
字段标记不能单独提供和弦端点。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,[NR:P]Cm|i;
```



3.10 M：枢纽转调

语法

```
{M:target|relation} ... {/M}
```

M 是 Modulation（转调）的缩写。
M 要求已有调性参照，并从该参照向目标调开启转调。
relation 使用第 5 章定义的基础罗马数字写法来描述目标调与原调的关系。
relation 所表示的主音音高及大小调性质必须与 target 一致。
枢纽是同时在原调与目标调中获得解释的和弦。
它的 R 字段分别给出两个调中的两层分析。
每个枢纽事件都须提供 C 和弦记号及两层 R 分析。
{/M} 结束枢纽范围。
该范围必须在另一转调、冲突的 TC 改写或正文省略一段音乐之前结束。
第 5 章给出了双层分析示例。

3.11 MH：强行转调

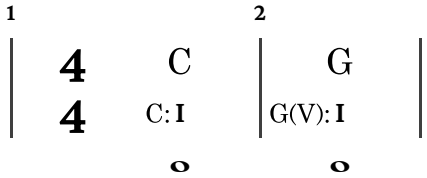
语法

```
{MH:target|relation}
```

MH 是 Modulation (Hard)（强行转调）的缩写。
MH 与 M 使用相同的目标和关系语法。
它要求有效的原调参照。
它在下一个合法和声事件处直接改变参照。
MH 是单个设置，无须结束标记。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}C|I;
{MH:G|V}G|I;
```



3.12 MM：节奏转换

语法

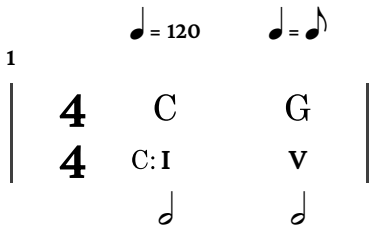
```
{MM:old=new}
```

MM 是 Metric Modulation（节奏转换）的缩写。
两侧均指定正分数拍单位。
这一关系表示转换前的旧音符时值与转换后的新音符时值在实际时长上相等。
MM 可以独立记录这一关系，无须先设置 BPM 或 BPMhidden。

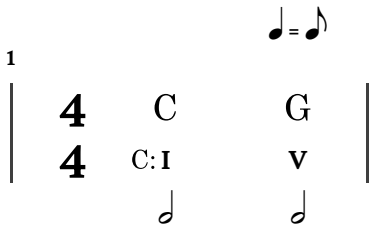
已有数值速度时，将当前拍单位乘以 new/old，并保留每分钟的拍数。
例如，1/4=120 后接 MM:1/4=1/8 得到 1/8=120，等价于每分钟 60 个四分音符。
左侧时值可以不同于当前 BPM 的拍单位，仍按同一比例换算。
没有数值速度时，MM 只记录拍值关系，不赋予播放速度数值。
后续 BPM 或 BPMhidden 建立新的显式速度。
稳定速度播放仍要求每个事件槽位具有确定的数值速度。

示例

```
{BN:1}{TS:4/4}{D:1/2}{TC:C}{BPM:1/4=120}C|I,{MM:1/4=1/8}G|V;
```




```
{BN:1}{TS:4/4}{D:1/2}{TC:C}C|I,{MM:1/4=1/8}G|V;
```



3.13 T：时间标签

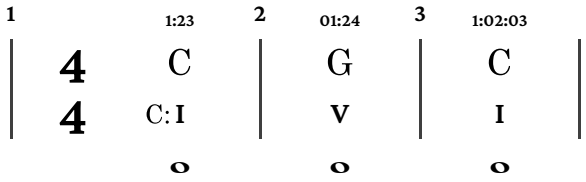
语法

```
{T=m:ss} / {T=mm:ss} / {T=h:mm:ss} / {T:m:ss}
```

T 是 Timestamp（时间戳）的缩写，为阅读附加时间标签。
它接受 m:ss、mm:ss 与 h:mm:ss，首部分可以包含任意位数字。
两部分写法中，前项表示非负分钟数，后项表示 00-59 的两位秒数。
三部分写法中，小时数非负，分钟与秒均使用 00-59 的两位数字。
T 在取值前接受 = 或:，并保留输入的文字。
标签可以按源码顺序递增或递减。
T 不改变播放时间，也不决定被省略段落的时长。
录音同步使用第 7 章介绍的 MStructTime 时间戳。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}{T=1:23}C|I;{T=01:24}G|V;{T=1:02:03}C|I;
```



3.14 省略号与继续分析

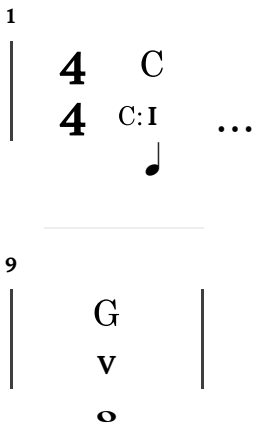
语法

```
... / {BN:n}
```

三个句点标记一段省略的内容。
省略号可以截断未完整的小节。
继续分析的下一个小节必须显式定义 BN。
D 和 TS 保留已有值，直到后续设置改变它们。
稳定速度播放计划不推测被省略段落的时长。
恢复段中的 T 标签仍只供阅读。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I...
{BN:9}{D:1}G|V;
```



3.15 LB：谱表行换行

语法

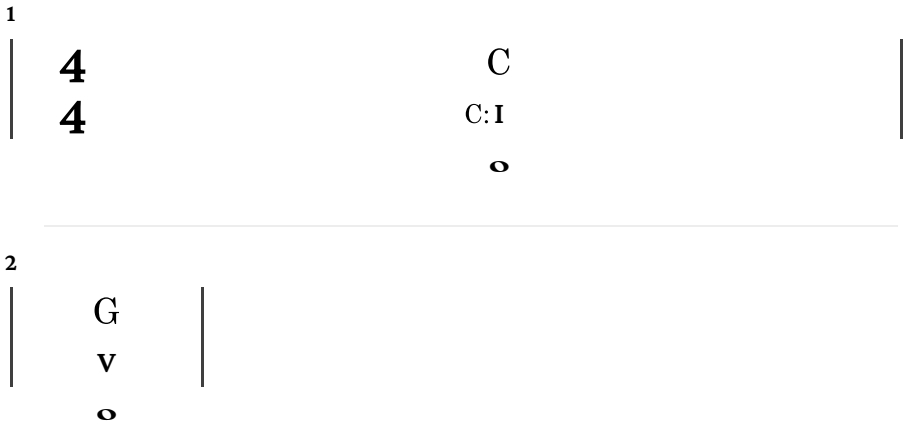
```
{LB}
```

LB 是 Line Break（换行）的缩写。
LB 在完整小节后请求谱表行换行。

它写在结束该小节的分号之后。
D、TC 和播放时间跨谱表行换行继续。
谱面用满可用宽度时也会自动换行。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;{LB}G|V;
```



3.16 双小节线

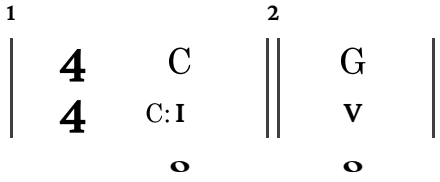
语法

```
{||}
```

{||} 在槽位边界显示双小节线。
分号仍须在规定时值结束后关闭小节。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I{||};G|V;
```



3.17 终止小节线

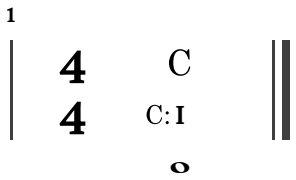
语法

```
{||END}
```

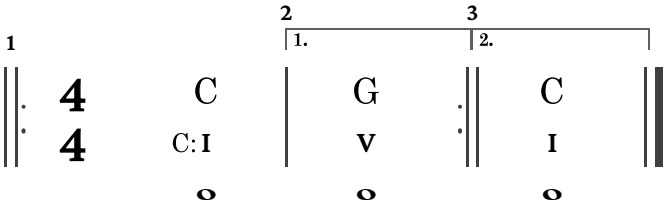
{||END} 是零时值的终止小节线，用于结束 MAIN。
它只能出现在最后小节中，紧邻该小节的分号之前。
标记与分号之间可以有空白。
分号之后的 MAIN 只能包含空白。
小节线左细右粗。
最后一个槽位可以为空，以其 D 延长前一事件。
在同一边界，{||END} 与双小节线或任一反复记号互斥。
最终结尾可以先用 {/ENDING} 闭合，再紧接 {||END}。
小节仍须通过当前拍号和实际演奏路线的时值检验。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}C|I,,,{||END};
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;
{ENDING:1}G|V{/ENDING:1}{:||};
{ENDING:2}C|I{/ENDING}{||END};
```



3.18 反复记号与总遍数

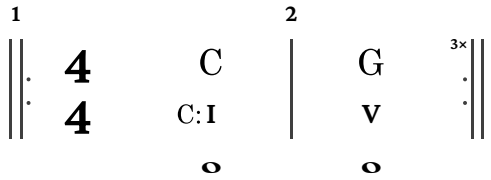
语法

```
{||:} / {:||} / {:||Nx}
```

左记号开启反复区，右记号结束反复区。
普通右反复表示总共演奏两遍。
显式 Nx 表示总共演奏 N 遍，N 的范围为 2-1000。
省去左记号时，反复以 MAIN 开头为隐式起点。
每个反复区必须在下一个反复区开始前结束。
回跳恢复目标位置保存的时值、拍号及速度状态。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;G|V{:||3x};
```



3.19 不同结尾

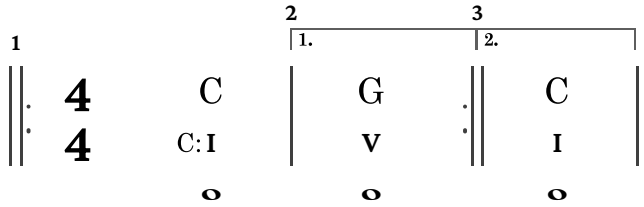
语法

```
{ENDING:1,2} ... {/ENDING}
```

结尾选择器列出采用该结尾的演奏遍数。
列表使用 1-1000 范围内互不重复的递增整数，并以逗号分隔。
这些逗号分隔控制标记内部的遍数。
各结尾必须覆盖所需遍数，并使用合法的反复记号位置。
带编号的结束标记（如 {/ENDING:1}）关闭非最终结尾，其编号列表须与起始标记完全一致。
不带编号的 {/ENDING} 关闭最终结尾，并结束整组不同结尾。
例如，第 1 遍使用 {ENDING:1} ... {/ENDING:1}，后接反复记号；第 2 遍使用 {ENDING:2} ... {/ENDING} 结束整组结尾。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}{||:}C|I;
{ENDING:1}G|V{/ENDING:1}{:||};
{ENDING:2}C|I{/ENDING};
```



3.20 连续显示小节号

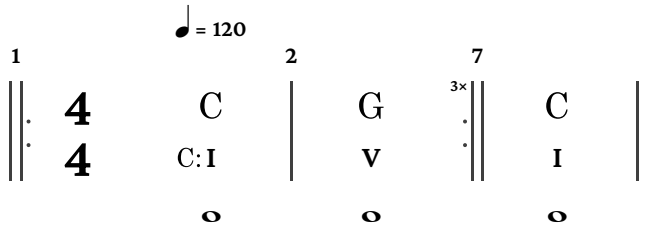
语法

```
{:||>>} / {:||Nx>>}
```

>> 标记要求文件使用稳定速度。
它在反复的各遍之间显示累计小节号。
源码为每个小节保留一份内容。
示例中的两小节反复三遍，随后的小节显示为第 7 小节。

示例

```
METADATA:
{IS_STABLE_TEMPO}True
{OFFSET}0
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}{BPM:1/4=120}{||:}C|I;G|V{:||3x>>};C|I;
END_MAIN
```



3.21 F：局部曲式记号

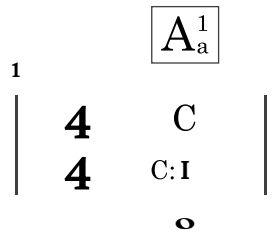
语法

```
[F:body|sup|sub]
```

F 为所在槽位的音乐事件附加一个可见曲式符号。
主体必须有值，上标和下标可以为空。
三部分均接受 [、]、| 以外的可打印 ASCII 字符。
每个槽位最多允许一个 F。
F 要求同槽位有音乐事件。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
[F:A|1|a]C|I;
```



3.22 COMMENT 引用与范围

语法

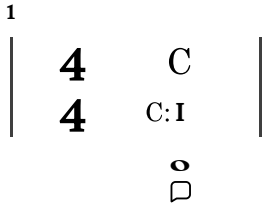
```
[COMMENT:n] ... [/COMMENT] / [/COMMENT:n]
```

文档 COMMENT 分区必须定义所引用的编号。
[COMMENT:n] 开启注释范围。
[/COMMENT] 结束当前范围。
[/COMMENT:n] 为一个位置附加注释引用。
同一时刻只能有一个活动范围。
同槽位内切换范围时，旧范围必须先结束，新范围才能开始。
定义可以分别提供简体中文（zh_cn）和英文（en）版本。
同一语言中的每个编号必须唯一。
引用位置会显示注释图标，在交互式谱面中可以由该图标打开注释。

示例

```
COMMENT:
[COMMENT:1<en>]
A tonic arrival.
[/COMMENT:1<en>]
[COMMENT:1<zh_cn>]
主和弦落点。
[/COMMENT:1<zh_cn>]
END_COMMENT

METADATA:
{IS_STABLE_TEMPO}False
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}[/COMMENT:1]C|I;
END_MAIN
COMMENT:
[COMMENT:1<en>]
A tonic arrival.
[/COMMENT:1<en>]
[COMMENT:1<zh_cn>]
主和弦落点。
[/COMMENT:1<zh_cn>]
END_COMMENT
```



4 和弦记号

4.1 后缀顺序与结构有效性

语法

```
root quality M extension altered sus add no /bass on lower
```

标准和弦后缀按上方所示顺序排列。
每个可选组最多出现一次，并遵循各自的性质限制。
最终音程结构必须包含根音音程 1。
不带 on 的和弦必须包含 1–12 个音高类互异的音。
此类和弦中的重复音程与同音异名音程均会导致结构错误，包括八度等价的情况。
下文的专用形式分别规定各自允许的后缀。
后续条目逐一解释这些组成部分及其限制。

示例

`C7(b9)sus4add#11no5` $\mathrm{C}^{7\flat 9}\mathrm{sus}^{4}\mathrm{add}^{\sharp 11}\mathrm{no}^{5}$

4.2 根音与升降号

语法

```
A-G + bb / b / # / x
```

和弦根音（A-G）使用一个大写音名字母。
字母后可以接一个升降记号：bb 表示重降，b 表示降，# 表示升，x 表示重升。
自然音根音只写字母。
音高类把相差整数个八度的音归为同类。
不同拼写对应相同音高类时，解析器保留各自的根音拼写。
小写根音、前置升降号及 Unicode 升降号均为无效输入。

示例

`C#` $\mathrm{C}^{\sharp}$

`Bbb` $\mathrm{B}^{\flat\flat}$

`Fx` $\mathrm{F}^{\times}$

4.3 三和弦性质

语法

```
root / rootm / rootaug / rootdim
```

和弦音程（如 1、b3、5）以根音为参照，采用大调音程命名。
例如，1 是根音，3 是大三度，b3 是小三度，5 是纯五度。
默认大三和弦包含 1、3、5。
性质后缀 m 给出 1、b3、5。
后缀 aug 给出 1、3、#5。

后缀 dim 给出 1、b3、b5。
性质记号区分大小写。
别名 maj/min 为无效输入。
记号显示偏好决定这些性质的显示形式，并保留源码和音程结构。

示例

`C` C

`Cm` Cm

`Caug` Caug

`Cdim` Cdim

4.4 七和弦与大七音

语法

```
7 / M7 / m7 / mM7 / dim7 / dimM7
```

后缀 7 为大、小或增三和弦增加 b7。
对于减和弦性质，7 增加 bb7。
M 选择自然 7，后面必须接 7、9、11 或 13。
数字 9、11、13 指定最高扩展音，具体定义见下一条。
因此，C7、CM7、Cm7、CmM7 和 Cdim7 分别指定不同结构。
单独的 CM 为无效输入。
m7b5 把小七和弦与变化五音组合在一起，具体写法见变化张力音条目。

示例

`C7` C^{7}

`CM7` C^{M7}

`CmM7` Cm^{M7}

`Cdim7` $Cdim^{7}$

`Cm7b5` $Cm^{7\flat5}$

4.5 最高扩展音

语法

```
9 / 11 / 13
```

最高扩展音后缀 9 增加七音与九音。
后缀 11 增加七音、九音与十一音。
后缀 13 增加七音、九音、十一音与十三音。
七音遵循前述性质及 M 的规则。
生成结构包含每个列出的音程，包括十三和弦中的十一音。
下文的变化张力音语法可以替换结构中允许改变的成分。

示例

`C9` C^{9}

`CM11` $\mathrm{C}^{\mathrm{M11}}$

`Cm13` Cm^{13}

4.6 六和弦系列

语法

```
6 / (b6) / 6/9 / 6/b9 / (b6/9) / (b6/b9)
```

六和弦系列后缀与最高扩展音共用同一个后缀位置。
选择六和弦系列后，就不再另写七音或最高扩展音。
六和弦系列后缀增加所写的六音，并在写出九音时增加九音。
这些后缀不生成七音。
它们允许大、小、增三和弦性质。
降六音形式必须使用此处所示的括号。
六和弦系列不接受 M 修饰及下文定义的 sus 后缀。

示例

`C6` C^{6}

`Cm6/9` $\mathrm{Cm}^{6/9}$

`C(b6/b9)` $\mathrm{C}^{\flat 6/\flat 9}$

4.7 变化张力音

语法

```
(b5,#5,b9,#9,b11,#11,b13,#13,b15,#15)
```

变化张力音在 7、9、11 或 13 后使用逗号分隔的括号列表。
可用音级为 5、9、11、13、15，每级允许 b 或 #。
项目按音级递增排列，同级同时出现两种变化时，b 写在 # 前面。
变化操作仅在该级已有的未变化音为自然附带成分时替换它。
该级没有未变化音时，操作直接增加指定的变化音。
明确写出的最高扩展音保留人工来源标记，具体来源规则见下文 no 条目。
因此，C9(b9)、C11(#11) 与 C13(b13) 均为无效的最高扩展音替换。
增、减性质不接受此列表中的变化五音。
解析器也接受旧的相邻连写形式，并把它显示为与括号形式相同的记号。

示例

`C13(b5,b9,#9)` $\mathrm{C}^{13\flat 5\flat 9\sharp 9}$

`C13b5b9#9` $\mathrm{C}^{13\flat 5\flat 9\sharp 9}$

4.8 sus：省三音与挂留音

语法

```
sus / sus2 / sus4 / sus(b2,2,4,#4)
```

sus 后缀删除和弦性质提供的三音。
裸写 sus 不增加替代音程。
可选的单项音程或括号列表按递增顺序增加从 b2、2、4、#4 中选出的成分。
该后缀允许六和弦系列以外的大、增性质。
Cmsus4 为无效输入。
该后缀定义当前和弦结构，后续解决由分析者另行判断。

示例

`Csus` Csus

`Csus4` Csus^{4}

`C7sus(2,4)` $\mathrm{C}^{7}\mathrm{sus}^{(2,4)}$

4.9 add：明确增加音级

语法

```
add9 / add(2,#4)
```

add 后缀增加一个明确音程，或增加括号递增列表中的各个音程。
允许的音程为 b2、2、b3、3、4、#4、b5、5、#5、b6、6、b7、7、b8、b9、9、#9、b10、10、11、#11、b12、12、b13、13、#13、b15、#15。
例如，add9 只增加九音，不增加七音。
add 中含 x 的音程为无效输入。
结果结构必须通过下文结构有效性条目所述的重复音与同音异名冲突检查。

示例

`Cadd9` Cadd^{9}

`Cadd(2,#4)` $\mathrm{Cadd}^{(2,\sharp 4)}$

`Edim7addb8onE` $\mathrm{Edim}^{7}\mathrm{add}^{\flat 8}\mathit{on}\mathrm{E}$

4.10 no：删除精确的自然附带音

语法

```
no3 / no(b3,5)
```

no 后缀删除生成结构中来源标记为自然附带的音程。
这个标记记录解析器提供音程的方式，也可用于 b3 等带升降号的音程。
允许的项目为 b3、3、b5、5，按音级递增排列，降音在前。
单项直接写在 no 后面，多项必须使用括号。

删除要求升降号、音级与自然附带来源标记完全匹配。
解析器把 Cm 的 b3 标为人工来源，因此 Cmnob3 为无效输入。
对于普通减三和弦，b3 标为自然附带，b5 标为人工来源。
对于减七和弦及更高扩展，两者均标为自然附带。
因此，Cdimnob5 为无效输入，Cdim7nob5 为有效输入。

示例

`Cno3` Cno^{3}

`Cno(3,5)` Cno$^{(3,5)}$

`Cdim7nob5` Cdim^{7}no$^{\flat 5}$

4.11 斜杠低音

语法

```
upper/bass
```

斜杠低音（/低音）在和弦后缀之后指定绝对低音。
低音遵循前述根音拼写规则。
解析器以该低音为参照重新计算音程结构。
例如，C/E 相对 E 具有音程 1、b6、b10。
每个和弦最多接受一个斜杠低音。

示例

`C/E` C/E

`Baug/F` Baug/F

4.12 on：复合和弦

语法

```
upper on lower
```

on 运算把上方和弦与下方和弦组合为复合和弦。
源码用 on 无空格连接两个完整和弦记号，例如 CM7onDm。
两侧各自可以带一个斜杠低音。
下方和弦的音程结构必须恰好包含三个音。
下方和弦内部的第二个 on 为无效输入。
最终结构以下方根音或低音为参照合并两侧，并保留复音程拼写。

示例

`CM7onDm` C$^{\mathrm{M7}}$*on*Dm

`C/GonF/A` C/G*on*F/A

4.13 1 与 5：终结式有音高形式

语法

```
C1 / C5
```

根音后接 1 时，结构只包含音程 1。
根音后接 5 时，结构包含音程 1 与 5。
两种形式均在此结束和弦记号，后面不接受七音、add、斜杠低音、on 等后缀。
根音可以带升降号。

示例

`C1` C^{1}

`C5` C^{5}

`F#5` $\mathrm{F}^{\sharp 5}$

4.14 q4：四度堆叠

语法

```
C[q4:(4,b7,b10)]
```

q4 形式提供根音音程 1，并接受一至三个相对该根音的音程。
列表从连续层级中各选一项：先选 4/#4，再选 7/b7，最后选 10/b10。
例如，4、b7、b10 用相对根音的累积音程描述三层四度堆叠。
该形式接受 add、斜杠低音与 on。
该形式不接受 sus 与 no。

示例

`C[q4:(4,b7,b10)]` $\mathrm{Cq4}^{(4,\flat 7,\flat 10)}$

4.15 q5：五度堆叠

语法

```
C[q5:(5,9,13)]
```

q5 形式提供根音音程 1，并接受一至三个相对该根音的音程。
列表从连续层级中各选一项：先选 5/b5，再选 9/b9，最后选 13/b13。
层级必须从第一层开始，并按此顺序排列。
该形式接受 add、斜杠低音与 on，不接受 sus 与 no。

示例

`C[q5:(5,9,13)]` $\mathrm{Cq5}^{(5,9,13)}$

4.16 nf：明确半音偏移

语法

```
C[nf:(0,4,7)]
```

nf 是 Normal Form（标准形式）的缩写。
nf 形式指定相对所写根音的半音偏移。
列表包含至少两个 0–36 的互不重复整数，并严格递增排列。
这些偏移直接生成音程结构。
结果结构必须通过前述结构检查，包括必须含有根音音程 1。
该形式在此结束和弦记号，后面不接受其他后缀。

示例

`C[nf:(0,4,7)]` $C^{[0,4,7]}$

4.17 sp：配置化命名结构

语法

```
C[sp:sowhat]
```

sp 形式从 special-chords.csv 中选择命名结构。
sp: 后的名称必须匹配该表中的编译名。
表格提供精确音程结构与显示名。
当前内置名称为 sowhat（1、4、b7、b10、12）、blk（1、#4、#6、9）与 mys（1、#4、b7、10、13、16）。
该形式接受斜杠低音与 on，不接受 sus、add 与 no。

示例

`C[sp:sowhat]` C^{sowhat}

`C[sp:blk]` C^{blk}

`C[sp:mys]` C^{mys}

4.18 7alt：保留原记法

语法

```
C7alt / C7alt/G
```

精确记号 7alt 保留尚未明确变化音的原有记法。
它构造骨架 1、3、b7。
7alt 后面只允许接可选的斜杠低音。
该形式不接受 add、sus、no 与 on，也不能作为 on 后面的下方和弦。
新分析应在已知变化音时明确写出相应记号。

示例

`C7alt` $C^{7}alt$

`C7alt/G` $C^{7}alt/G$

5 罗马数字

5.1 参照与独立语法

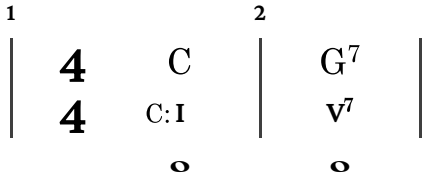
语法

```
{TC:C} ... |I
```

罗马数字记号指定相对当前调性参照的音级内容。
它继承该参照的大小调性质，包括带引号的参照。
四张罗马数字配置表定义可用的基础记号、性质修饰、数字低音与特殊记号。
后续条目分别定义各个组成部分及允许的组合。
罗马数字字段及其附加解释必须有调性参照，具体条件见第 2 章。
在 MStruct 中，和弦记号与罗马数字共同描述同一音乐事件，起到互补作用。
因此，罗马数字字段通常着重记录和声级数、七和弦性质及转位，9、11、13 等张力音由和弦字段补充。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;G7|V7;
```



5.2 基础项、大小写与升降

语法

```
I / i / bIII / nIII
```

基础罗马数字记号必须匹配 roman-basic.csv 中的一个记号。
其音级相对调性参照采用大调音程命名。
小写记号具有小三和弦性质。
大写记号具有大三和弦性质，V 则具有配置中的属性质。
III/iii 与 VI/vi 只允许用于大调。
nIII/niii 与 nVI/nvi 只允许用于小调。
在两种调式中，b 均表示大调音级降低半音，n 均表示对应的未变化音级。
因此，小调中的 bIII 指 b3 级，nIII 指自然 3 级。

示例

`bIII` **♭III**

`nIII` **♮III**

`III` **III**

5.3 紧凑数字低音

语法

```
6 / 64 / 7 / 65 / 43 / 42
```

紧凑数字低音是选择转位并在规定时增加七音的后缀。
后缀 6 与 64 分别代表三和弦的第一、第二转位。
后缀 7 增加七音，并保持原位。
后缀 65、43、42 增加七音，并分别代表第一、第二、第三转位。
配置性质与调性参照决定七音，性质修饰或特殊记号可以提供覆盖值。
例如，大调中的 I7 包含自然 7，IDM7 使用 b7。
下一条定义 DM 及其他性质修饰，包括各自的性质限制与推荐关系。

示例

`I6` $\mathbf{I}^{6}$

`I64` $\mathbf{I}^{6}_{4}$

`I7` $\mathbf{I}^{7}$

`IDM7` $\mathbf{I}^{\mathbf{DM7}}$

`V65` $\mathbf{V}^{6}_{5}$

5.4 高级性质修饰

语法

```
+ / o / ho / DM / oM / M
```

性质修饰（+、o、ho、DM、oM、M）写在基础记号之后、数字低音之前。
修饰 + 把五音改为 #5。
修饰 o 把五音改为 b5，并在记号含七音时选择 bb7。
修饰 ho 提供 b5 与 b7，并要求记号含有七音。
DM 选择 b7。
oM 选择 b5 与自然 7。
M 选择自然 7。
后三种修饰要求接增加七音的数字低音后缀。
修饰 + 与 DM 要求基础性质为大或属。
修饰 o、ho 与 oM 要求基础性质为小。
M 接受小或属基础性质；下文列出的特殊导音记号也可在减三和弦上提供大七音。
V 以外的大写大三和弦基础项原本已具有大七音，因而拒绝冗余的 M；IM7 与 IVM65 均为无效输入。
VM7 仍有效，因为 M 把属七音改为大七音。
Io7、i+ 等其他性质冲突同样属于错误。
基础表为每个基础项列出推荐修饰。
推荐清单以外的组合在满足性质、七音、非冗余及数字低音要求时仍为有效输入，并产生 advanced-not-recommended 警告。
例如，iM7 属于推荐组合，bIIDM7 则为带有该警告的有效输入。
本章末尾的表格列出推荐关系与数字低音要求。

示例

`I+` $\mathbf{I}^{+}$

`io7` $\mathbf{i}^{o7}$

`iiho7` $\mathbf{ii}^{ø7}$

`iM7` $\mathbf{i}^{M7}$

`ioM7` $\mathbf{i}^{oM7}$

5.5 sus 与变化五音数字低音

语法

```
sus / 7sus / 7b5 / 7b5sus
```

数字低音后缀 sus 删除三音，不增加替代四音。
后缀 7sus 增加七音并删除三音。
这些 sus 形式要求基础性质为大或属，并要求性质修饰为空或为 +、DM、M 之一。
每种组合还必须符合该修饰的数字低音白名单。
后缀 7b5 增加七音并降低五音。
后缀 7b5sus 另删除三音，并遵循 sus 的限制。
isus 与 i7b5 为无效罗马数字输入。

示例

`Isus` $\mathbf{I}^{sus}$

`V7sus` $\mathbf{V}^{7sus}$

`IDM7b5sus` $\mathbf{I}^{DM7sus}_{\flat 5}$

5.6 副调罗马数字后缀（副和弦）

语法

```
V7/V / V7/V/V
```

副调后缀（/目标）按某个基础罗马数字目标的配置音级平移前面的罗马数字结构。
每个目标必须是基础记号，不带性质修饰或数字低音后缀。
连续目标最多允许两层。
连续目标的音级平移依次累积。
主记号及其目标均按当前调性参照的大小调性质检查。
配置音级相同的目标产生相同平移，例如 V7/II 与 V7/ii。
下文定义的特殊记号与通用数字低音列表也可在各自允许时接副调后缀。
后续条目说明数字低音链如何保留副调目标列表。

示例

`V7/V` $\mathbf{V^7/V}$

`V7/V/V` $\mathbf{V^7/V/V}$

`viio42/V` $\mathbf{vii^{o4}_2/V}$

5.7 通用数字低音列表

语法

```
V[9,#7,5,3-]
```

通用数字低音列表用方括号指定相对保留低音或根音参照的音程。
它替代紧凑数字低音后缀。
列表包含二至四项，每项的数字范围为 2 至 14。
数字按数值降序排列，允许相等，并可带 n、b、bb、#、x。
未变化的数字表示大音程或纯音程，临时记号对该音程作相应改变。
每个重复数字都保留显示，相同结构音则合并。
数字低音链把同一保留参照下的前后列表连接起来。
新命名的通用数字低音记号必须在列表末尾用 - 延续链，或用 -x 在该事件终止链。
因此，独立的 I[5,3] 为无效输入，I[5,3-x] 则提供了明确终止标记。
下一条定义后续列表与链边界。

示例

`V[9,#7,5,3-x]` $\mathbf{V^{\substack{9\\\sharp 7\\5\\3}}}$

`V[n6,6,6-x]` $\mathbf{V^{\substack{\flat 6\\6\\6}}}$

5.8 数字低音链的延续与终止

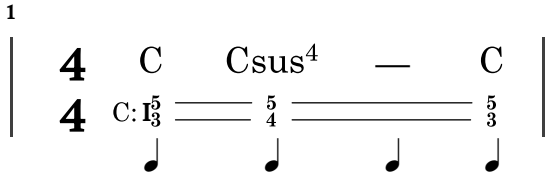
语法

```
I[5,3-] ... [5,4] / I[5,3-x]
```

最后一个数字后的 - 保持链活动。
后续列表省去初始罗马数字名称，例如 [5,4]。
它保留初始根音参照，并且必须使用相同的数字层数与副调目标列表。
末尾没有标记的后续列表结束链。
标记 -x 明确在当前事件结束链。
纯和弦事件、裸写 - 与 R:- 可以出现在列表之间，并保留链。
链活动期间，新命名的罗马数字、R:X 或其他特殊事件会产生中断错误。
改变调性参照或开始转调同样要求活动链已经结束。
结束标记 -x 也可表达分析者所判断的“欲言又止”。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}
C|I[5,3-],Csus4|[5,4-],-,C|[5,3];
```



5.9 配置中的增六记号

语法

```
Ger+6 / Gero3 / Fr+6 / It+6 / Tr+6 / Blk+6
```

特殊表把每个增六记号定义为具有独立结构的完整记号。
例如，Ger+6 具有 b6、1、b3、#4，Fr+6 具有 b6、1、2、#4。
Gero3 还选择对应的音名拼写约定。
末尾的 +6 属于列出的特殊记号，不能作为基础罗马数字记号的通用后缀。

示例

`Ger+6` **Ger^{+6}**

`Gero3` **Ger^{o3}**

`Fr+6` **Fr^{+6}**

`It+6` **It^{+6}**

`Tr+6` **Tr^{+6}**

`Blk+6` **Blk^{+6}**

5.10 VS、VSBlk 与导音记号

语法

```
VS7 / VSBlk / viio7 / viiho7
```

S 是 Substitution（代理）的缩写。
VS 系列表示降 II 级上的属七和弦，通常用作属和弦的三全音代理。
VS 必须接配置中的后缀 7、65、43、42、7b5，或接通用数字低音列表。
裸写 VS 为无效输入。
VSBlk 是独立的完整特殊记号。
特殊表还为 viio、viio6、viio64、viio7、viio65、viio43、viio42、viioM7 以及 viiho7、viiho65、viiho43、viiho42 定义精确结构。
列出的每个导音记号都是该表中的完整记号。

示例

`VS7` **V(S)^{7}**

`VSBlk` **V(S)^{Blk}**

`viio7` **vii^{o7}**

`viiho65` **vii$^{ø6}_{5}$**

5.11 共同音记号

语法

```
cto7 / ct+6
```

共同音减七记号 cto7 保留记法，不提供固定的罗马数字音高结构。
它接受副调后缀。
当前默认表包含 cto7，未列入转位记号 cto65、cto43、cto42。
共同音记号 ct+6 具有明确结构 1、#2、#4、b6。

示例

`cto7/V` **CT^{o7}/V**

`ct+6` **CT^{+6}**

5.12 M 中的双层罗马数字

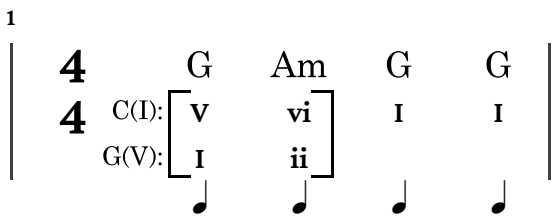
语法

```
old&new
```

在第 3 章定义的 M 枢纽范围内，& 分隔同一事件的两层罗马数字。
左层使用原调性参照，右层使用目标调性参照。
每层分别维护自己的数字低音链上下文。
在示例的 C 到 G 转调中，G 在 C 中为 V，在 G 中为 I。
Am 在 C 中为 vi，在 G 中为 ii。

示例

```
{BN:1}{TS:4/4}{D:1/4}{TC:C}
{M:G|V}G|V&I,Am|vi&ii{/M},G|I,G|I;
```



5.13 当前基础项与推荐修饰

下表列出各基础项的推荐修饰。
三音性质及数字低音规则决定组合的有效性；有效但未推荐的组合产生警告。

基础项	音级 / 性质	调性门槛	推荐修饰
I	1 / M	*	+,DM
i	1 / m	*	o,ho,oM,M
bII	b2 / M	*	-
bii	b2 / m	*	M
II	2 / M	*	DM
ii	2 / m	*	o,ho,M
bIII	b3 / M	*	+,DM
biii	b3 / m	*	M
III	3 / M	M	+
iii	3 / m	M	M,ho
nIII	3 / M	m	-
niii	3 / m	m	ho
IV	4 / M	*	DM
iv	4 / m	*	ho,M
#IV	#4 / M	*	-
#iv	#4 / m	*	M,ho
bV	b5 / M	*	-
bv	b5 / m	*	M
V	5 / D	*	+,M
v	5 / m	*	M,ho
bVI	b6 / M	*	-
bvi	b6 / m	*	M
VI	6 / M	M	-
vi	6 / m	M	M,ho
nVI	6 / M	m	-
nvi	6 / m	m	M,ho
bVII	b7 / M	*	DM
bvii	b7 / m	*	M
nVII	7 / M	*	-
nvii	7 / m	*	M

5.14 当前高级修饰的数字低音准入

修饰	须七音	七音覆盖	数字低音
(empty)	-	none	6,64,7,65,43,42,7b5,sus,7sus,7b5sus,[]
+	-	7	6,64,7,65,43,42,sus,7sus
o	-	bb7	6,64,7,65,43,42
ho	True	b7	6,64,7,65,43,42
DM	True	b7	6,64,7,65,43,42,7b5,sus,7sus,7b5sus
oM	True	7	7,65,43,42
M	True	7	7,sus,7sus,7b5sus,65,43,42

5.15 特殊罗马数字的精确结构

Token	音级结构	附加数字规则
Ger+6	b6 1 b3 #4	-
Gero3	#4 b6 1 b3	-
Fr+6	b6 1 2 #4	-
It+6	b6 1 #4	-
Tr+6	b6 7 2 #4	-
Blk+6	b6 b7 2 #4	-
VS	b2 4 b6	7,65,43,42,7b5,[]（必填）
VSBlk	b2 b3 5 7	-
viio	7 2 4	-
viio6	2 4 7	-
viio64	4 7 2	-
viio7	7 2 4 b6	-
viio65	2 4 b6 7	-
viio43	4 b6 7 2	-
viio42	b6 7 2 4	-
viioM7	7 2 4 #6	-
viiho7	7 2 4 6	-
viiho65	2 4 6 7	-
viiho43	4 6 7 2	-
viiho42	6 7 2 4	-
cto7	(none)	-
ct+6	1 #2 #4 b6	-

6 标签事件

6.1 标签拼写与持续模式

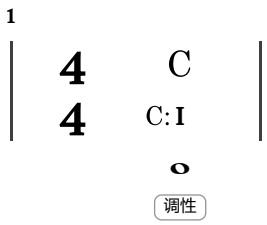
语法

```
<name> ... </name> / </name>
```

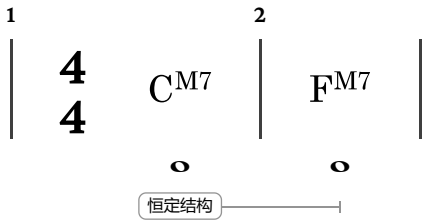
分析标签（<name> 或 </name>）为某个槽位的音乐特征命名。
该槽位还必须包含音乐事件。
标签范围从起始标签（<name>）开始，在匹配的结束标签（</name>）处结束。
该范围从起始槽位到结束槽位均处于活动状态。
普通标签配置为每个标签指定持续模式：1、2 或 3。
模式 1 仅使用作用于一个位置的一次性标签（</name>）。
模式 2 允许成对范围；同名范围未活动时，其结束形式用作一次性标签。
模式 3 要求成对范围。
每个活动范围必须在同名标签再次开启前以及文档结束前关闭。
标签名称和参数区分大小写。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Tonality>C|I;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Constant_Structure>CM7|;FM7|</Constant_Structure>;
```



6.2 分离式视觉终点

语法

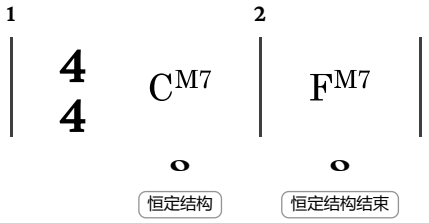
```
<*/name>
```

分离式终点（<*/name>）关闭持续模式 2 或 3 的活动标签范围。
谱面以分离的起止标签代替连续线。
该范围在搜索和播放中仍活动到结束槽位。
普通标签使用配置名称，例如 <*/Constant_Structure>。

Mode 与 Scale 使用不带主音参数的家族名称，例如 `<*/Mode_Dorian>` 与 `<*/Scale_WholeTone>`。
pedal 与 Centricity 分别使用 `<*/pedal>` 与 `<*/Centricity>`，同样不带参数。
分离式终点要求存在匹配的活动范围，不能用作一次性标签。
模式 1 标签包含终止式及隐式曲式，均没有可用这种方式关闭的成对范围。
DTC 使用自身的清除命令（`</DTC>`）。
后面的模进与显式曲式条目分别说明它们对分离式终点的支持。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Constant_Structure>CM7|;FM7|<*/Constant_Structure>;
```



6.3 调性、调式性及相关分类

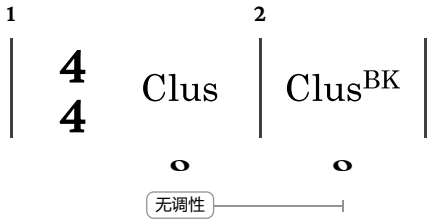
可用值

```
Tonality / Modality / Atonality / Polytonality / Pantriadic
```

Tonality、Modality、Atonality、Polytonality 和 Pantriadic 对所分析音乐的调性组织进行分类。
当前普通标签表为这五项均指定持续模式 2。
每项可以标记一个位置或成对范围。
调性参照（`{TC:}`）独立于这些分类，继续决定罗马数字的解释。
分析缺少用于解释罗马数字的单一调性参照时，使用 `{TC:None}`。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:None}<Atonality>Clus;ClusBK</Atonality>;
```



6.4 中心性

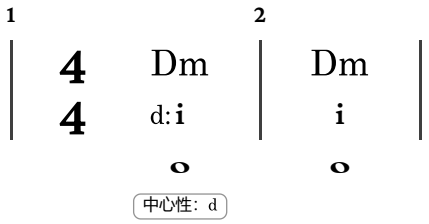
语法

```
<Centricity:C> ... </Centricity> / </Centricity:"C">
```

Centricity 通常用于具有明显音高中心性、但不具有调性和声的曲目或段落。
Centricity 标识调性中心，并立即把 TC 设为其参数。
参数接受具体或带引号的调性参照，并允许一个可选的 b 或 #。
Centricity 使用持续模式 2。
一次性形式也会改变 TC。
Centricity 范围结束后，TC 保留该值。
同一时刻只能有一个活动的 Centricity 范围。
活动范围内禁止 TC、M 与 MH 设置，包括重复同一中心的设置。
应用这些设置前，须先关闭该范围。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}</Centricity:d>Dm|i;Dm|i;
```



6.5 调式

语法

```
<Mode_Dorian:D> ... </Mode_Dorian>
```

调式通常描述较大尺度的和声组织，例如整首曲目或某个片段的低音乃至和弦均处于某一调式内。
其用途应与下文通常描述较小旋律片段的音阶标签区分。
Mode 标签（`<Mode_Dorian:D>`）标识配置中的调式及其主音。
可用家族必须在 tonal-tags.csv 中具有非空调式名称。
主音参数使用 A-G 或 a-g，后接可选的 bb、b、# 或 x。
参数保留字母大小写。
Mode 使用持续模式 2，结束标签不带参数（`</Mode_Dorian>`）。
同一时刻只能有一个活动的 Mode 范围。
TC 独立于 Mode，罗马数字的解释遵循 TC。
TC 以及转调设置 M、MH 可以在 Mode 范围内改变。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}<Mode_Dorian:D>Dm|ii;G|V</Mode_Dorian>;
```



6.6 音阶

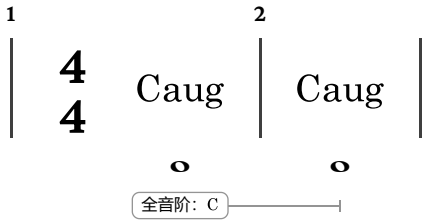
语法

```
<Scale_WholeTone:C> ... </Scale_WholeTone>
```

音阶通常描述某一较短片段中，旋律配合当前和声所使用的音阶。
其典型尺度小于调式标签所描述的和声组织。
Scale 标签（`<Scale_WholeTone:C>`）标识配置中的音阶及其主音。
可用家族使用 tonal-tags.csv 中的音阶名称列。
主音参数采用与 Mode 相同的音名拼写，包含大小写字母以及可选的 bb、b、# 或 x 升降号。
Scale 使用持续模式 2。
Scale 开始或结束时，TC 保留原值。
AlteredScale、MixedBluesScale、MinorBluesScale、MajorBluesScale 和 BebopScale 仅提供 Scale 家族。
后面的词表列出全部配置家族。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Scale_WholeTone:C>Caug|;Caug|</Scale_WholeTone>;
```



6.7 DTC：两个调性

语法

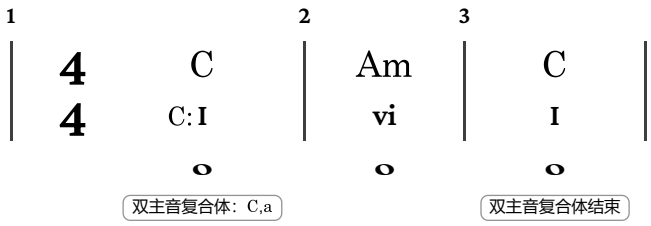
```
<DTC:C,a> / </DTC>
```

DTC（`<DTC:C,a>`）用两个具名调性标记双主音复合体。
两个参数均使用不带引号的具体大调或小调参照，并允许一个可选的 b 或 #。
两个参照必须在主音音高类或大小调性质上存在区别。
同主音的大调与小调参照可以并列。

大小调性质相同的同音异名参照视为重复项。
显示时保留两个参照的输入顺序。
同一时刻只能有一组活动的调性对，每个新 DTC 都替换前一组。
调性对持续活动，直到另一个 DTC 替换它或 </DTC> 清除它。
清除未活动的 DTC 后，其状态仍为未活动。
DTC 无须在文档结尾添加结束标签。
TC 独立于 DTC，在 DTC 改变或清除时保留原值。
全曲标签元数据字段（{GLOBAL_TAGS}）接受一组完整调性对，例如 </DTC:C,a>。
全曲 DTC 作用于整份文档，并保留 TC 原值。
存在全曲 DTC 时，MAIN 中禁止出现任何 DTC 命令，包括清除及结束命令。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<DTC:C,a>C|I;Am|vi;</DTC>C|I;
```



6.8 隐式曲式分类与段落

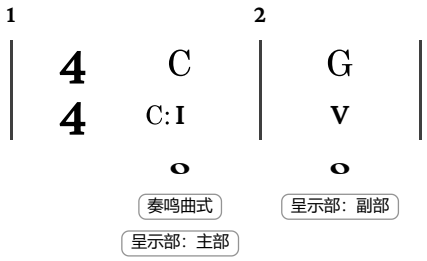
语法

```
</Sonata_Form> / </Sonata_Form:E_Sub1>
```

曲式标签对曲式进行分类（</Sonata_Form>），或标识其中的段落（</Sonata_Form:E_Sub1>）。
曲式使用独立的七列配置表。
allow_global 列为 1 时，该条目允许进入 GLOBAL_TAGS。
这类描述性曲式分类与其段落共存，后续段落不会覆盖该分类。
模式 1 的曲式标签建立隐式状态：该标签从所在槽位起持续生效，直到曲式命令替换或清除它。
parent 列把段落归入家族。
parent 为空的条目自身构成一个家族。
replace_previous 的值决定新隐式项如何影响同家族的先前项。
值为 1 时，新项只在自身及先前项的 allow_global 均为 0 时替换该先前项；值为 0 时，它们可以共存。
替换操作丢弃先前项的隐式状态；清除替换项后，先前项仍保持未活动。
活动的隐式曲式标签参与搜索和播放。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Sonata_Form></Sonata_Form:E_Sub1>C|I;</Sonata_Form:E_Sub2>G|V;
```



6.9 FORM_END：结束隐式曲式状态

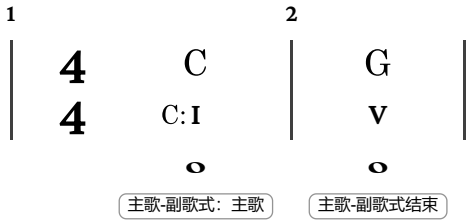
语法

```
</FORM_END> / </FORM_END:family> / </FORM_END:family:section>
```

FORM_END 清除隐式曲式状态。
无参数形式（</FORM_END>）清除全部隐式曲式项。
家族目标（</FORM_END:family>）清除该家族的全部隐式项。
完整目标（</FORM_END:family:section>）仅清除具名项。
每个目标都必须存在于配置中。
清除已配置但未活动的目标后，它仍保持未活动。
FORM_END 作为命令执行，搜索不能把它用作活动标签。
显式成对曲式范围需要用各自的结束标签关闭。
在反复及时间轴跳转（LOOP，见第 7 章）中，曲式命令遵循实际演奏顺序。
回跳恢复目标之前保存的隐式曲式状态。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
</Pop_Form:Verse>C|I;</FORM_END:Pop_Form>G|V;
```



6.10 显式曲式范围

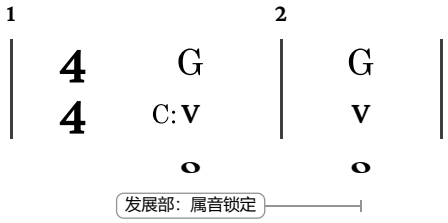
语法

```
<Sonata_Form:D_DL> ... </Sonata_Form:D_DL>
```

持续模式为 2 或 3 的曲式配置可以开启显式成对范围。
该范围持续到匹配的结束标签。
范围也接受分离式结束标签（<*/Sonata_Form:D_DL>）。
隐式替换和 FORM_END 仅作用于隐式状态，显式范围继续活动。
当前 D_DL、12_Bar_Blues 和 16_Bar_Blues 条目使用模式 2。
配置中的大多数其他曲式使用隐式模式 1。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Sonata_Form:D_DL>G|V;G|V</Sonata_Form:D_DL>;
```



6.11 终止式标签

语法

```
</PAC>
```

终止式标签（</PAC>）对和声落点或乐句之间的衔接进行分类。
配置中的全部终止式标签均为一次性标签。
标签写在定义所指定的事件上，通常为落点和弦。
分析者依据原音乐中的乐句结构、高音及声部进行判断终止式。
编译器检查标签拼写及其允许的参数。
下列公式给出相对于所述主音的音级。
例子使用第 5 章定义的 MStruct 罗马数字拼写。

6.12 PAC

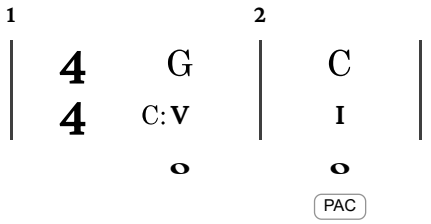
语法

```
</PAC>
```

PAC 是 Perfect Authentic Cadence（完全正格终止）的缩写。
完全正格终止由属和弦进行到主和弦。
两个和弦均为原位，落点高音为主音。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;C|I</PAC>;
```



6.13 IAC:not_tonic

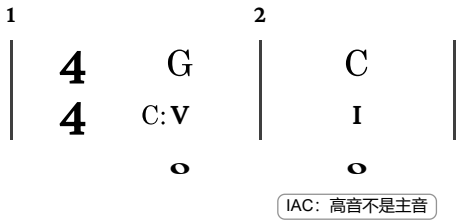
语法

```
</IAC:not_tonic>
```

IAC 是 Imperfect Authentic Cadence（不完全正格终止）的缩写。
这一不完全正格终止由属和弦进行到主和弦，落点高音为主音以外的音级。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;C|I</IAC:not_tonic>;
```



6.14 IAC:inverted

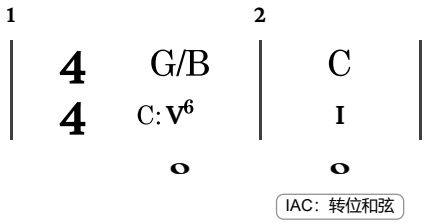
语法

```
</IAC:inverted>
```

IAC 是 Imperfect Authentic Cadence（不完全正格终止）的缩写。
这一不完全正格终止在属到主的进行中使用转位和弦。
和声事件写明具体哪一个和弦转位及其转位形式。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G/B|V6;C|I</IAC:inverted>;
```



6.15 IAC:viio

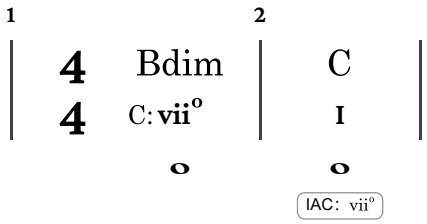
语法

```
</IAC:viio>
```

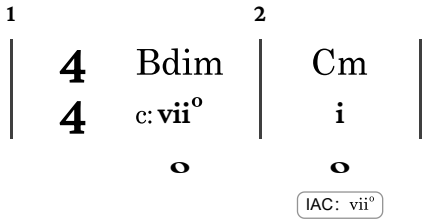
IAC 是 Imperfect Authentic Cadence（不完全正格终止）的缩写。
这一不完全正格终止由导音减三和弦进行到大主和弦或小主和弦：viio 到 I 或 i。
标签写在主和弦落点。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Bdim|viio;C|I</IAC:viio>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}Bdim|viio;Cm|i</IAC:viio>;
```



6.16 HC

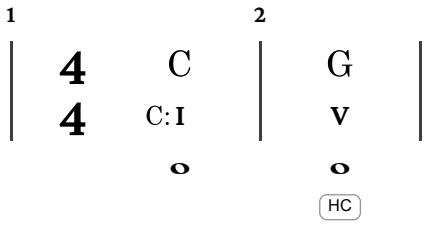
语法

```
</HC>
```

HC 是 Half Cadence（半终止）的缩写。
半终止以属和声结束乐句。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
C|I;G|V</HC>;
```



6.17 HC:Phrygian

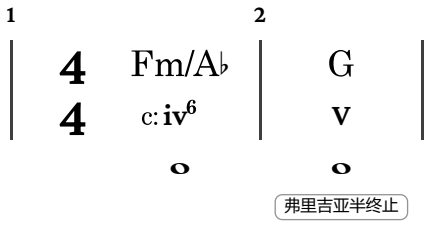
语法

```
</HC:Phrygian>
```

HC 是 Half Cadence（半终止）的缩写。
弗里吉亚半终止在小调参照下由 ivb6 进行到 V。
其低音下行半音。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}Fm/Ab|iv6;G|V</HC:Phrygian>;
```



6.18 HC-PAC

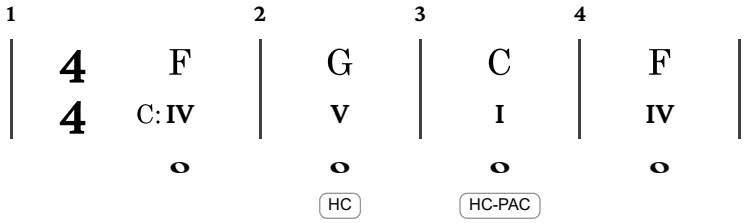
语法

```
</HC-PAC>
```

HC-PAC 连接两个平行乐句。
前句以半终止结束。
后句开头立即将这个属和弦解决到满足 PAC 条件的主和弦。
属和弦与主和弦均为原位，主和弦落点高音为主音。
前句末尾的属和弦标 HC，后句开头的主和弦标 HC-PAC。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;G|V</HC>;C|I</HC-PAC>;F|IV;
```



6.19 HC-IAC

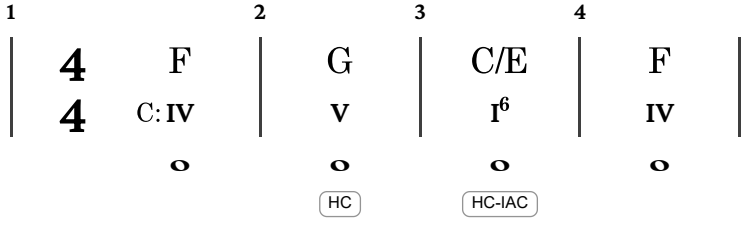
语法

```
</HC-IAC>
```

HC-IAC 连接两个平行乐句。
前句以半终止结束。
后句开头立即将这个属和弦解决到满足 IAC 条件的主和弦。
前句末尾的属和弦标 HC，后句开头的主和弦标 HC-IAC。
例子使用第一转位主和弦来满足 IAC 的转位条件。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;G|V</HC>;C/E|I6</HC-IAC>;F|IV;
```



6.20 DC:vi

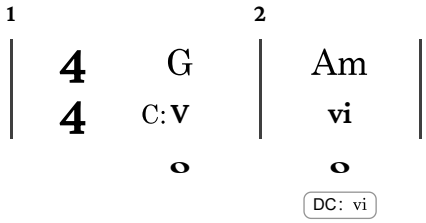
语法

```
</DC:vi>
```

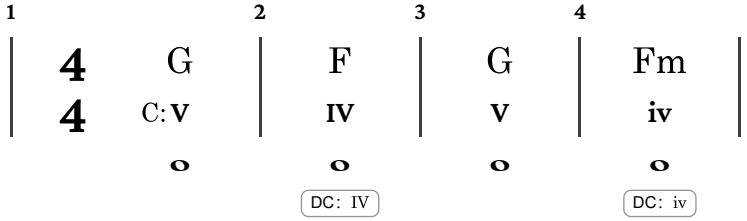
DC 是 Deceptive Cadence（阻碍终止）的缩写。
阻碍终止把预期的属到主解决转向其他和声。
参数写属和弦之后的目标和声。
参数必须使用后面终止式参数词表所列的目标之一。
V 转向大下属和弦或小下属和弦，避开预期的终止主和弦落点时，可使用 IV 或 iv。
若乐句在 V 上结束，IV 或 iv 开启下一乐句，且没有这种预期解决，则前句末尾标为 HC。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;Am|vi</DC:vi>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
G|V;F|IV</DC:IV>;G|V;Fm|iv</DC:iv>;
```



6.21 PC:IV

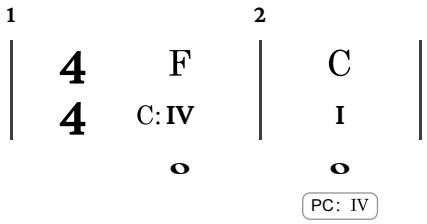
语法

```
</PC:IV>
```

PC 是 Plagal Cadence（教会终止）的缩写。
教会终止由参数所指的和声进行到主和弦。
允许的参数为 IV、iv、ii、iio 和 iiho7。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F|IV;C|I</PC:IV>;
```



6.22 EC

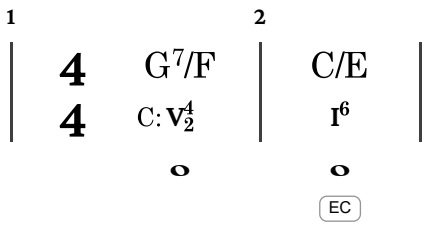
语法

```
</EC>
```

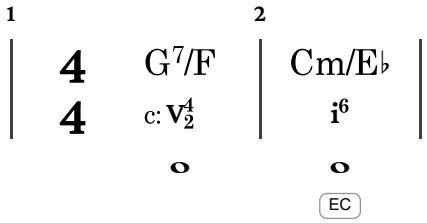
EC 是 Evaded Cadence（回避终止）的缩写。
回避终止由 V42 进行到 I6 或 i6。
第三转位属七和弦解决到第一转位大主和弦或小主和弦。
大调例子中的低音由 F 下行半音到 E。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
G7/F|V42;C/E|I6</EC>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}G7/F|V42;Cm/Eb|i6</EC>;
```



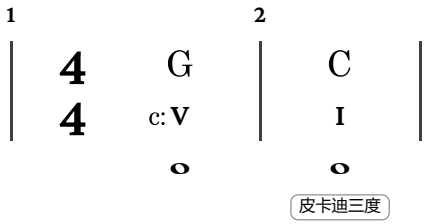
6.23 Picardy_Third

语法

```
</Picardy_Third>
```

皮卡第三度在小调结尾升高主和弦三音，形成大主和弦。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}G|V;C|I</Picardy_Third>;
```



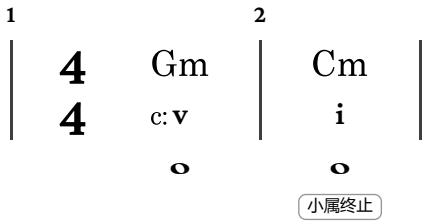
6.24 Minor_Dominant_Cadence

语法

```
</Minor_Dominant_Cadence>
```

小属终止把小属三和弦解决到大主和弦或小主和弦：v 到 I 或 i。
属三和弦的根音上方为小三度。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Gm|v;Cm|i</Minor_Dominant_Cadence>;
```



6.25 Backdoor_Cadence

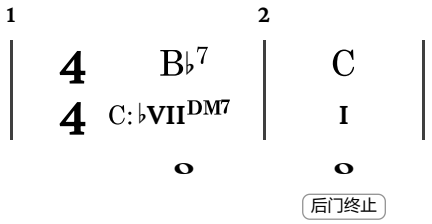
语法

```
</Backdoor_Cadence>
```

后门终止把降 VII 级上的属七和弦解决到大主和弦或小主和弦。
以 C 为主音时，Bb7 解决到 C 或 Cm。
例子使用 bVIIDM7 指定属七和弦性质。
在 MStruct 中，裸 bVII7 为该基础项添加其默认的大七音。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Bb7|bVIIDM7;C|I</Backdoor_Cadence>;
```



6.26 Aeolian_Cadence

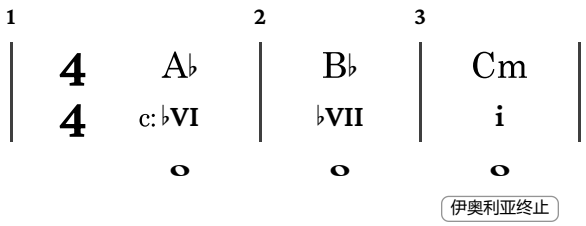
语法

```
</Aeolian_Cadence>
```

伊奥利亚终止式由 bVI 经 bVII 进行到 i。
前两个和弦为大三和弦，主和弦为小三和弦。
以 C 为主音时，进行为 Ab 到 Bb 到 Cm。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Ab|bVI;Bb|bVII;Cm|i</Aeolian_Cadence>;
```



6.27 Mario_Cadence

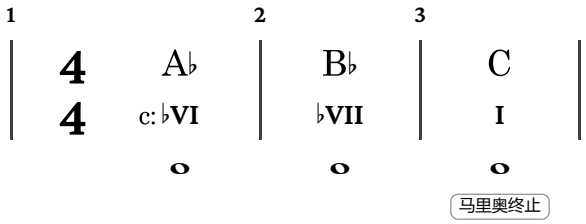
语法

```
</Mario_Cadence>
```

Mario 终止由 bVI 经 bVII 进行到 I。
三个和弦均为大三和弦。
以 C 为主音时，进行为 Ab 到 Bb 到 C。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Ab|bVI;Bb|bVII;C|I</Mario_Cadence>;
```



6.28 Aeolian_Cadence:minor

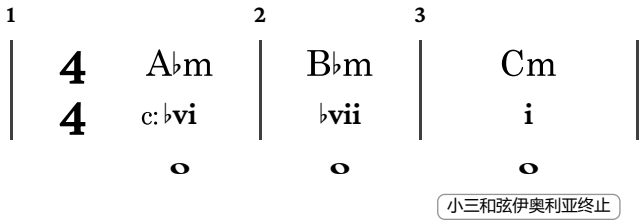
语法

```
</Aeolian_Cadence:minor>
```

小三和弦伊奥利亚终止由 bvi 经 bvii 进行到 i。
三个和弦均为小三和弦。
以 C 为主音时，进行为 Abm 到 Bbm 到 Cm。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:c}
Abm|bvi;Bbm|bvii;Cm|i</Aeolian_Cadence:minor>;
```



6.29 Phrygian_Cadence_to_Tonic:bII

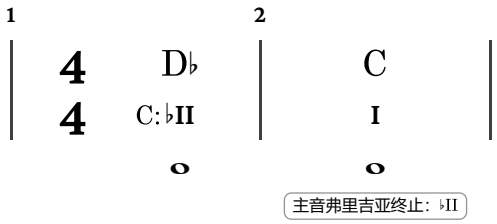
语法

```
</Phrygian_Cadence_to_Tonic:bII>
```

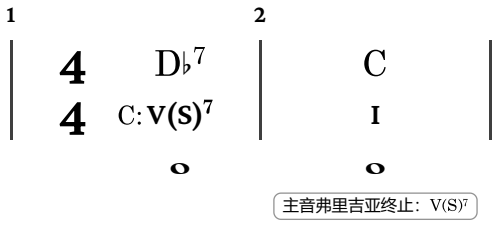
这一弗里吉亚终止由根音位于降 II 级的和声进行到大主和弦或小主和弦。
参数写主和弦之前的和声。
参数必须为 bII、bII7、VS7、VS7b5 或 VSBlk 之一。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
Db|bII;C|I</Phrygian_Cadence_to_Tonic:bII>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:C}
Db7|VS7;C|I</Phrygian_Cadence_to_Tonic:VS7>;
```



6.30 Tritone_Cadence

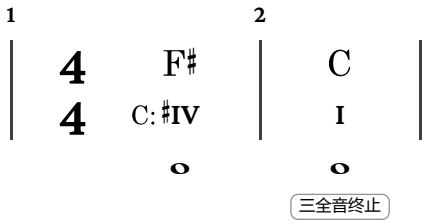
语法

```
</Tritone_Cadence>
```

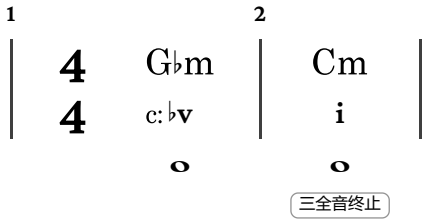
三全音终止把根音距主音三全音的和声解决到 I 或 i。
接近主和弦的三和弦可以为大三和弦或小三和弦，其罗马数字拼写为 #IV、#iv、bV 或 bv。
以 C 为主音时，F#、F#m、Gb 或 Gbm 可以进行到主和弦。
罗马数字拼写保留升 IV 级与降 V 级的书写区别。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
F#|#IV;C|I</Tritone_Cadence>;
```



```
{BN:1}{TS:4/4}{D:1}{TC:c}Gbm|bv;Cm|i</Tritone_Cadence>;
```



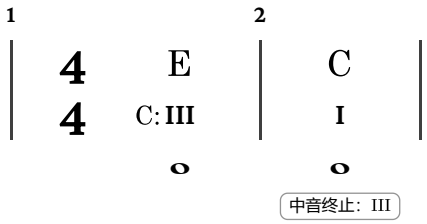
6.31 Mediant_Cadence:III

语法

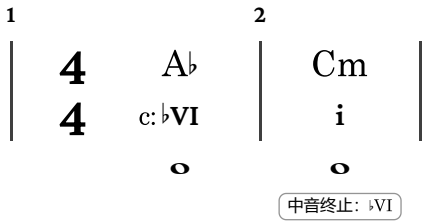
```
</Mediant_Cadence:III>
```

中音终止把 III 级或 VI 级和声解决到主和弦。
参数写主和弦之前的和声。
允许的参数为 III、iii、bIII、biii、nIII、niii、VI、vi、bVI、bvi、nVI 和 nvi。
示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
E|III;C|I</Mediant_Cadence:III>;
```




```
{BN:1}{TS:4/4}{D:1}{TC:c}Ab|bVI;Cm|i</Mediant_Cadence:bVI>;
```



6.32 pedal：持续音音级标签

语法

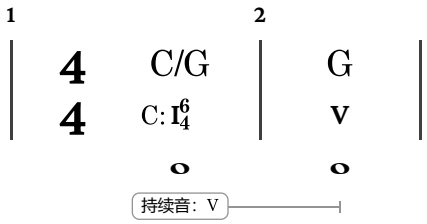
```
<pedal:V> ... </pedal>
```

pedal（<pedal:V>）用音级标记持续音。
其参数必须为当前基础表中的罗马数字基础项。
该标签使用持续模式 3，在 MAIN 中要求成对范围。

结束标签为 </pedal>，不带罗马数字参数。
和声事件各自指定低音。
应用 pedal 时，TC 保留原值。
带参数的结束形式（</pedal:V>）也可用于 GLOBAL_TAGS。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<pedal:V>C/G|I64;G|V</pedal>;
```



6.33 Seq 与 SeqInv 模进

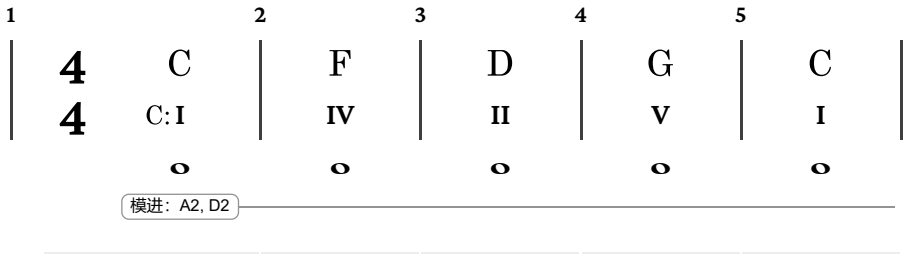
语法

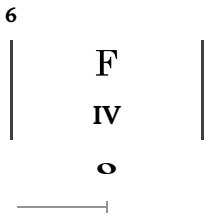
```
<Seq:A2,D2> ... </Seq>
```

Seq（<Seq:A2,D2>）和 SeqInv（<SeqInv:A2,D2>）标记配置中的模进类型。
每个参数列表包含 sequence-tags.csv 中一个或多个互不重复的精确类型。
列表用逗号分隔项目，禁止空格和空项。
每个类型仅标识其精确拼写；派生拼写需要单独列出。
Seq 和 SeqInv 的活动范围相互独立，可以重叠。
二者均要求成对范围，并以无参数形式结束：</Seq> 或 </SeqInv>。
二者也接受分离式终点：<*/Seq> 或 <*/SeqInv>。
GLOBAL_TAGS 不接受 Seq 或 SeqInv。
后面的词表列出配置中的分类码。
六小节示例以 C-F 两和弦为模型：先整体上行二度至 D-G，再整体下行二度回到 C-F。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<Seq:A2,D2>C|I;F|IV;D|II;G|V;C|I;F|IV</Seq>;
```





6.34 配置中的和弦进行标签

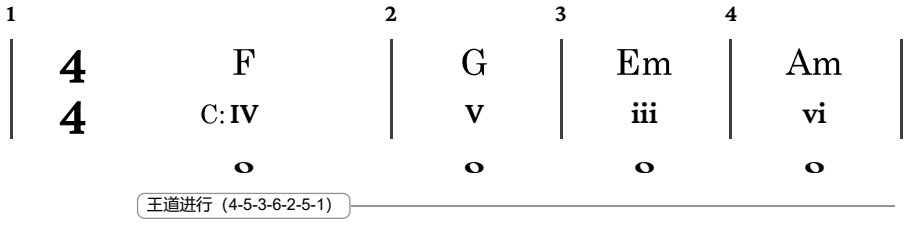
语法

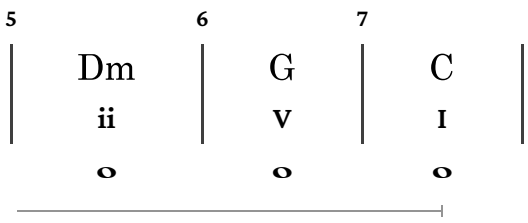
```
<Descending_Line_Cliche> ... </Descending_Line_Cliche>
```

和弦进行标签按具名进行对一段和声事件进行分类。
当前标签为 Descending_Line_Cliche、Ascending_Line_Cliche、Omnibus_Progression、Coltrane_Changes、Constant_Structure、4536251 和 4536251V。
每项均使用持续模式 3，并要求显式成对范围。
分析者为范围内的和声选择分类。

示例

```
{BN:1}{TS:4/4}{D:1}{TC:C}
<4536251>F|IV;G|V;Em|iii;Am|vi;Dm|ii;G|V;C|I</4536251>;
```





6.35 全曲标签

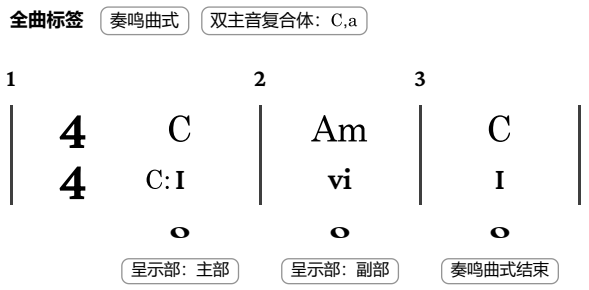
语法

```
{GLOBAL_TAGS}</Tonality></Sonata_Form>
```

GLOBAL_TAGS（{GLOBAL_TAGS}）是保存全曲分析标签的元数据字段。
其值由连续排列的完整结束形式标签组成。
每个标签规范化后的身份必须唯一。
该字段接受普通标签及上文所述支持的带参数标签。
曲式配置的 allow_global=1 时，该条目即可进入全曲标签，是否具有 parent 不再作为准入条件。
当前表允许 Binary_Form、Ternary_Form、Rondo_Form、Arch_Form、Variation_Form、Sonata_Form、12_Bar_Blues、16_Bar_Blues、Pop_Form、Canon、Fugue、StropheBridge_Form 与 EDM_Form。
局部段落改变，或 FORM_END 清除局部曲式状态时，这些全曲分类继续生效。
该字段也允许一组 DTC，例如 </DTC:C,a>；此时 MAIN 中禁止出现任何局部 DTC 命令。
FORM_END、Seq、SeqInv 及 allow_global=0 的曲式不可用于该字段。
每个全曲标签作用于文档中的全部搜索及播放事件。
TC 独立于全曲标签保留原值。
同名局部范围结束后，全曲标签继续生效。
目录元数据字段（{TAG}）单独保存目录关键词列表。

示例

```
METADATA:
{IS_STABLE_TEMPO}False
{GLOBAL_TAGS}</Sonata_Form></DTC:C,a>
END_METADATA
MAIN:
{BN:1}{TS:4/4}{D:1}{TC:C}</Sonata_Form:E_Sub1>C|I;
</Sonata_Form:E_Sub2>Am|vi;</FORM_END:Sonata_Form>C|I;
END_MAIN
```



6.36 普通标签完整词表

Token	名称	模式
<Coltrane_Changes>	柯川变换	3
<Constant_Structure>	恒定结构	3
<Omnibus_Progression>	Omnibus 进行	3
<Atonality>	无调性	2

Token	名称	模式
<Polytonality>	多调性	2
<Pantriadic>	泛三和弦	2
<Modality>	调式性	2
<Tonality>	调性	2
<4536251>	王道进行（4-5-3-6-2-5-1）	3
<4536251V>	王道进行（4-5-3-6-2-5-1）变体	3

6.37 调式与音阶家族完整词表

家族	调式名	音阶名
WholeTone	全音阶调式	全音阶
Hexatonic	六声音阶调式	六声音阶
Octatonic	八声音阶调式	八声音阶
MoLT3	有限移位调式 3	有限移位音阶 3
MoLT4	有限移位调式 4	有限移位音阶 4
MoLT5	有限移位调式 5	有限移位音阶 5
MoLT6	有限移位调式 6	有限移位音阶 6
MoLT7	有限移位调式 7	有限移位音阶 7
GongPentatonic	宫五声调式	宫五声音阶
ShangPentatonic	商五声调式	商五声音阶
JuePentatonic	角五声调式	角五声音阶
ZhiPentatonic	徵五声调式	徵五声音阶
YuPentatonic	羽五声调式	羽五声音阶
Miyakobushi	都节调式	都节音阶
Ryukyu	琉球调式	琉球音阶
Ionian	爱奥尼亚调式	爱奥尼亚音阶
Melodic_Minor	旋律小调调式	旋律小调音阶
Dorian	多利亚调式	多利亚音阶
Dorianb2	多利亚 b2 调式	多利亚 b2 音阶
Phrygian	弗里吉亚调式	弗里吉亚音阶
PhrygianDominant	大弗里吉亚调式	大弗里吉亚音阶
Lydian	利底亚调式	利底亚音阶
Lydian#5	利底亚 #5 调式	利底亚 #5 音阶
Lydianb7	利底亚 b7 调式	利底亚 b7 音阶
Mixolydian	混合利底亚调式	混合利底亚音阶
Aeolian	伊奥利亚调式	伊奥利亚音阶
Melodic_Major	旋律大调调式	旋律大调音阶
Locrian	洛克里亚调式	洛克里亚音阶
Locriann2	洛克里亚 n2 调式	洛克里亚 n2 音阶
SuperLocrian	超级洛克里亚调式	超级洛克里亚音阶
AlteredScale	-	变化音阶
HarmonicMinor	和声小调调式	和声小调音阶
HarmonicMajor	和声大调调式	和声大调音阶
DoubleHarmonicMajor	双和声大调调式	双和声大调音阶
MixedBluesScale	-	混合布鲁斯音阶
MinorBluesScale	-	小调布鲁斯音阶
MajorBluesScale	-	大调布鲁斯音阶

家族	调式名	音阶名
BebopScale	-	Bebop 音阶

6.38 终止式参数白名单

家族	允许的完整罗马数字
<PC:>	IV,iv,ii,iio,iiho7
<Phrygian_Cadence_to_Tonic:>	bII,bII7,VS7,VS7b5,VSBlk
<Mediant_Cadence:>	III,iii,bIII,biii,nIII,niii,VI,vi,bVI,bvi,nVI,nvi
<DC:>	III,iii,bIII,biii,nIII,niii,IV,iv,VI,vi,bVI,bvi,nVI,nvi

6.39 模进类型完整词表

基础码	允许的派生码
A1	-
A2	A2(+5/-4),A2(+4/-3)
A3	A3(+4/-2)
A4	-
A5	-
D1	D1(+4/-A4)
D2	D2(-5/+4)
D3	D3(-5/+3)
D4	-
D5	-

6.40 曲式完整词表

“全曲”列对应 allow_global，1 表示允许用于 GLOBAL_TAGS，0 表示仅用于局部。

Token	名称	模式 / 替换	全曲
<Binary_Form>	二部曲式	1 / 1	1
<Ternary_Form>	三部曲式	1 / 1	1
<Rondo_Form>	回旋曲式	1 / 1	1
<Arch_Form>	拱形曲式	1 / 1	1
<Variation_Form>	变奏曲式	1 / 1	1
<Sonata_Form>	奏鸣曲式	1 / 1	1
<Sonata_Form:E_Intro>	奏鸣曲式：引子	1 / 1	0
<Sonata_Form:E_Sub1>	呈示部：主部	1 / 1	0
<Sonata_Form:E_Transition>	呈示部：连接部	1 / 1	0
<Sonata_Form:E_MC>	呈示部：中间停顿	1 / 1	0
<Sonata_Form:E_Sub2>	呈示部：副部	1 / 1	0
<Sonata_Form:E_EEC>	呈示部：EEC	1 / 0	0
<Sonata_Form:E_Closing>	呈示部：结束部	1 / 1	0
<Sonata_Form:E_Codetta>	呈示部：小尾声	1 / 1	0

Token	名称	模式 / 替换	全曲
<Sonata_Form:D>	奏鸣曲式：发展部	1 / 1	0
<Sonata_Form:D_Retransition>	发展部：再连接部	1 / 1	0
<Sonata_Form:D_DL>	发展部：属音锁定	2 / 0	0
<Sonata_Form:R_Sub1>	再现部：主部	1 / 1	0
<Sonata_Form:R_Transition>	再现部：连接部	1 / 1	0
<Sonata_Form:R_Sub2>	再现部：副部	1 / 1	0
<Sonata_Form:R_ESC>	再现部：ESC	1 / 0	0
<Sonata_Form:R_Closing>	再现部：结束部	1 / 1	0
<Sonata_Form:Coda>	奏鸣曲式：尾声	1 / 1	0
<12_Bar_Blues>	12 小节蓝调	2 / 1	1
<16_Bar_Blues>	16 小节蓝调	2 / 1	1
<Pop_Form>	主歌-副歌式	1 / 1	1
<Pop_Form:Intro>	主歌-副歌式：引子	1 / 1	0
<Pop_Form:UpfrontChorus>	主歌-副歌式：冒头副歌	1 / 1	0
<Pop_Form:Verse>	主歌-副歌式：主歌	1 / 1	0
<Pop_Form:Prechorus>	主歌-副歌式：预副歌	1 / 1	0
<Pop_Form:Chorus>	主歌-副歌式：副歌	1 / 1	0
<Pop_Form:PostChorus>	主歌-副歌式：后副歌	1 / 1	0
<Pop_Form:InstBreak>	主歌-副歌式：副歌间奏	1 / 1	0
<Pop_Form:Verse2>	主歌-副歌式：主歌第二遍	1 / 1	0
<Pop_Form:Prechorus2>	主歌-副歌式：预副歌第二遍	1 / 1	0
<Pop_Form:Chorus2>	主歌-副歌式：副歌第二遍	1 / 1	0
<Pop_Form:ChorusExtension>	主歌-副歌式：副歌第二遍延长	1 / 1	0
<Pop_Form:Bridge>	主歌-副歌式：桥部	1 / 1	0
<Pop_Form:Bridge_Solo>	主歌-副歌式：桥部乐器独奏	1 / 1	0
<Pop_Form:PreChorus3>	主歌-副歌式：预副歌第三遍	1 / 1	0
<Pop_Form:BreakdownChorus>	主歌-副歌式：静奏副歌	1 / 1	0
<Pop_Form:Chorus3+>	主歌-副歌式：副歌第三遍 +	1 / 1	0
<Pop_Form:Outro>	主歌-副歌式：尾声	1 / 1	0
<Canon>	卡农	1 / 1	1
<Fugue>	赋格	1 / 1	1
<StropheBridge_Form>	主段-桥部式	1 / 1	1
<StropheBridge_Form:Intro>	主段-桥部式：引子	1 / 1	0
<StropheBridge_Form:Strophe>	主段-桥部式：主段	1 / 1	0
<StropheBridge_Form:Bridge>	主段-桥部式：桥部	1 / 1	0
<StropheBridge_Form:InstBridge>	主段-桥部式：器乐桥部	1 / 1	0
<StropheBridge_Form:Refrain>	主段-桥部式：叠句	1 / 0	0
<StropheBridge_Form:Outro>	主段-桥部式：尾声	1 / 1	0
<Strophic_Form>	分节歌式	1 / 1	0
<Strophic_Form:Intro>	分节歌式：引子	1 / 1	0
<Strophic_Form:Strophe>	分节歌式：主段	1 / 1	0
<Strophic_Form:Refrain>	分节歌式：叠句	1 / 0	0
<Strophic_Form:InstStrophe>	分节歌式：器乐主段	1 / 1	0
<Strophic_Form:InstBreak>	分节歌式：器乐间奏	1 / 1	0
<Strophic_Form:Outro>	分节歌式：尾声	1 / 1	0
<EDM_Form>	EDM 曲式	1 / 1	1
<EDM_Form:Intro>	EDM 曲式：引子	1 / 1	0
<EDM_Form:Groove>	EDM 曲式：节拍主段	1 / 1	0
<EDM_Form:Breakdown>	EDM 曲式：Breakdown 段	1 / 1	0
<EDM_Form:BuildUp>	EDM 曲式：蓄势段	1 / 1	0
<EDM_Form:Drop>	EDM 曲式：Drop 段	1 / 1	0

Token	名称	模式 / 替换	全曲
`<EDM_Form:Outro>`	EDM 曲式：尾声	`1 / 1`	`0`

7 时间轴语法

7.1 一份录音与一份时间轴

时间轴（.mstructtime）把分析文件的事件对应到一份录音中的时间。
分析文件提供和弦与罗马数字内容。
时间轴提供录音元数据、显式经过路线（LOOP）及时间戳（MAIN）。
关联时间轴要求分析采用非稳定速度。
已关联时间轴的分析必须保留非稳定速度设置。

7.2 METADATA、LOOP 与 MAIN

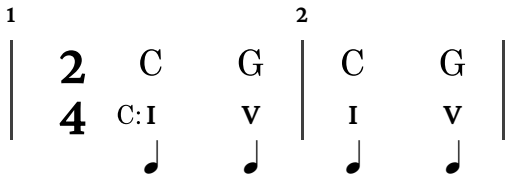
语法

```
METADATA: ... LOOP: ... MAIN:
```

时间轴按顺序包含 METADATA、LOOP 和 MAIN。
每个分区均使用对应的 END_ 标记结束。
METADATA 描述录音。
LOOP 定义父分析位置之间的跳转。
空 LOOP 按顺序经过结构源码中的位置。
MAIN 在这些位置记录时间戳。
下例经过第 1 小节两遍、第 2 小节一遍。
父分析展示在对应时间轴之前。

示例

```
{BN:1}{TS:2/4}{D:1/4}{TC:C}
C|I,G|V;C|I,G|V;
```




```
METADATA:
{MSTRUCTTIME_NAME}Example recording
{PERFORMER_1}Example performer
{RECORD_YEAR}2026
{MUSIC_LENGTH}0:06
{URL_APPLE_MUSIC}https://music.apple.com/example
{SCORE_URL_1}https://example.org/score
{URL_ADDITIONAL_1:Recording details}https://example.org/recording
END_METADATA

LOOP:
1,2/4,1,1,1/4;
1,2/4,2,2,1/4;
END_LOOP

MAIN:
```

```
{BN:1}{TS:2/4}{D:1/4}
[0,1][2000,2],[1000,1][3000,2];
{BN:2}{D:1/4}[4000,1],[5000,1];
END_MAIN
```

7.3 录音元数据

字段

```
{MSTRUCTTIME_NAME} / {PERFORMER_n} / {MUSIC_LENGTH}
```

录音名称（{MSTRUCTTIME_NAME}）为必填项。
文件须至少填写一位演奏者的姓名（{PERFORMER_n}）。
文件最多保留 100 位演奏者。
录音时长（{MUSIC_LENGTH}）为必填项，使用 m:ss 或 h:mm:ss。
名称、演奏者或专辑值含非 ASCII 字符时，必须提供对应的非空 ASCII 罗马化字段。
名称使用 {MSTRUCTTIME_NAME_ROMANIZED}。
每位演奏者使用 {PERFORMER_n_ROMANIZED}。
专辑使用 {RECORD_ALBUM_ROMANIZED}。
可选的录音年份（{RECORD_YEAR}）使用非负整数字符串。
录音专辑（{RECORD_ALBUM}）为可选项。
每份分析（.mstruct）与每份录音时间轴（.mstructtime）在 METADATA 中分别保存自己独立的一套链接。
平台链接使用 {URL_NETEASE}、{URL_TME}、{URL_YOUTUBE}、{URL_BILIBILI}、{URL_SPOTIFY} 和 {URL_APPLE_MUSIC}，字段后填写对应 URL；这些字段均为可选项。
乐谱链接使用 {SCORE_URL_n}，n 为 1 至 9。
附加链接使用 {URL_ADDITIONAL_n:网站名}，字段后填写 URL，n 为 1 至 9；编辑表单要求网站名称与 URL 同时填写。
旧的未编号输入 {URL_ADDITIONAL:网站名} 仍可读取；保存时写为编号字段。
时间轴没有链接字段时，其链接集合为空。
主站分别显示分析链接和当前选中时间轴的录音链接。

7.4 LOOP 跳转指令

语法

```
sourceMeasure,sourceBeat,pass,targetMeasure,targetBeat;
```

LOOP 的每个非空行包含一条跳转指令。
来源和目标各自指定小节号与拍位坐标。
pass 是该来源位置的正整数经过次数。
路线在指定的该次经过后跳转到目标位置。
来源位置没有适用指令时，路线继续到下一个结构源码位置。
来源小节、来源拍位和 pass 的每个组合必须唯一。
两端坐标都必须对应到父分析中的位置。
路线必须在经过步数上限内结束。

示例

```
1,2/4,1,1,1/4;
1,2/4,2,2,1/4;
```

7.5 来源终点与目标起点

来源拍位 p/q 指向小节开始后 p/q 个全音符处的槽位终点。
目标拍位 p/q 指向小节开始后 (p-1)/q 个全音符处的槽位起点。
显示的事件拍位使用相同的从 1 起算的起点记法。
它们的分母采用拍号（TS）与槽位时值（D）的共同分母。
例如，4/4 小节包含两个二分音符槽位。
第二槽位从小节一半处开始，因此它的目标坐标为 3/4。
它在小节边界结束，因此它的来源坐标为 4/4。
该槽位结束后跳回自身起点的指令因此使用来源 4/4 和目标 3/4。

7.6 LOOP end 终止指令

语法

```
sourceMeasure,sourceBeat,pass,end;
```

四字段 end 指令在指定来源的该次经过后结束路线。
该次经过保留为路线的最后一个位置。
最后一个已录事件保持选中，直到录音时长（MUSIC_LENGTH）结束。
end 不区分大小写。

示例

```
1,2/4,1,1,1/4;
1,2/4,2,end;
```

7.7 时间戳记录

语法

```
[milliseconds,pass] / [,pass]
```

录音时间戳通常通过时间轴编辑器录入。
时间戳 [milliseconds,pass] 为源码位置的一次经过指定录音时间。
时间使用从录音开始起算的整数毫秒。
pass 使用正整数。
[,2] 等空时间字段标记尚未录入的经过。
显式 0 标记录音起点。
同位置多次经过的记录写在相邻的方括号中。
BN 选择源码小节。
每个逗号按 D 推进位置，每个分号推进到下一小节。

示例

```
{BN:1}{TS:2/4}{D:1/4}
[0,1][2000,2],[1000,1][,2];
```

7.8 哪些位置产生事件行

事件行类型

```
START / event / ...
```

时间轴表格以隐式 START 行开始，其时间为 0 ms。
音乐事件和可见省略号分别产生事件行。
空逗号槽位延长前面的事件。
只有设置的槽位不产生事件行。
省略号具有自己的小节、起始位置及遍数。
它的时间戳可以在时间轴 MAIN 中写为 `...[5000,1]`。
已录时间必须沿实际 LOOP 路线非递减。
连续多行可以使用相同时间。

7.9 未录入与残余时间戳

语法

```
[,1]
```

播放经过未录入事件时保留原有高亮。
下一个已录事件在其时间戳处获得高亮。
残余时间戳是当前路线以外的经过记录。
文件保留这些记录，供后续恢复。
从某小节重置会清除该小节及以后所有小节的时间戳，包括残余记录。

7.10 导入分析反复

反复导入把经过验证的分析路线及其结尾选择转换成显式 LOOP 指令。
转换使用槽位终点作为来源，使用槽位起点作为目标。
生成的时间轴遵循 LOOP 路线。
采用连续小节编号（>>）的反复不能导入。
父分析结构改变后，必须重新检查路线坐标及既有时间戳。

7.11 偏移量与文件关联

语法

```
{MSTRUCTTIME_n}filename.mstructtime
```

父元数据通过 MSTRUCTTIME_n 字段关联时间轴。
它最多保留 100 个不同的关联文件名。
生成文件名采用 parent-time-八位字符.mstructtime 形式，每个新文件使用新的标识。
非稳定分析按关联时间轴的时间戳播放，并叠加用户全局延迟及会话临时偏移。
这些偏移不改动时间轴中保存的时间戳。
稳定分析不关联时间轴，按 BPM 路线播放，并叠加用户全局延迟、会话临时偏移及分析文件的 OFFSET。
分析文件的 OFFSET 只用于稳定播放。

8 搜索语法

以下每个查询分别列出可匹配的具体源码、反例及原因。“匹配”表示整条查询能在该源码中找到至少一个窗口。除非表中另写设置，例子均以 {BN:1}{TS:4/4}{D:1}{TC:C} 开头；分号分隔源码小节。全曲标签行中的 {GLOBAL_TAGS} 属于元数据。

8.1 单事件与有序序列

语法

```
C|I / C|I,G|V
```

查询事件描述一个待匹配的源码事件。
和声查询事件同时提供和弦字段与罗马数字字段，并用 | 分隔。
整字段 * 使该侧不受限制。
等价的显式形式使用 [C:chord|R:roman]。
C| 在 MAIN 中合法，但搜索时须写成 C|*。
顶层逗号按所写顺序请求一串相邻事件。
查询使用这些逗号跨越源码小节边界，不接受结束小节的分号。
附着在同一查询事件上的所有条件必须同时成立。
结果窗口标识满足该序列的源码事件。
对同一组事件的多次经过可以共用一个结果窗口。

查询 1

```
C|I,G|V
```

结果	具体源码	原因
匹配	C|I;G|V;	C|I 跨小节线后紧接 G|V，因此这两个事件按顺序满足查询。
不匹配	C|I;F|IV;G|V;	普通事件 F|IV 位于 C|I 与 G|V 之间，打断查询要求的相邻关系。

查询 2

```
[C:C|R:I]
```

结果	具体源码	原因
匹配	C|I;	事件 C|I 的和弦精确为 C，罗马数字精确为 I，因此它同时满足两个查询字段。

8.2 完整字段与和弦异记法

语法

```
Cm6|* / *|V7
```

完整的具体和弦查询默认匹配相同的最终低音音高类与完整音高类集合。
最终低音是完成 slash 和 on 运算后所得结构的根音。
Cm6 与 Am7b5/C 的低音都是 C，构成音都是 C、Eb、G、A。

G/C 与 CM9no3 的低音都是 C，构成音都是 C、D、G、B。
即使绝对音高类集合不变，最终低音不同也属于不同匹配。
匹配忽略同音异名、八度重复与构成音的排列。
在查询后附加 –notallowdifferentnotation 可改按和弦记法比较，每条查询最多允许一个该选项。
记法比较仍会统一正规与旧式变化音列表。
已配置的特殊和弦使用其解释所得音集，未知的历史精确特殊 token 仍可按源码身份搜索。
精确罗马数字查询保留书写身份。
主记号或同领域的附加解释均可满足正向查询。
双层罗马数字分析中任一合法层可以匹配，连续的罗马数字匹配沿连续调性路径进行。

查询 1

```
C7|*
```

结果	具体源码	原因
匹配	`C7|IDM7;`	C7 满足精确和弦字段，* 接受罗马数字 IDM7。
不匹配	`CM7|I7;`	和弦 CM7 含大七度，因此其完整记号与要求的 C7 不同。

查询 2

```
*|V7
```

结果	具体源码	原因
匹配	`G7|V7;`	罗马数字字段精确为 V7，* 接受和弦 G7。
不匹配	`G7/B|V65;`	罗马数字字段 V65 指定的转位与精确查询 V7 不同。

查询 3

```
Cm6|*
```

结果	具体源码	原因
匹配	`Am7b5/C|;`	Am7b5/C 的最终低音为 C，完整音集 C、Eb、G、A 与 Cm6 相同。
不匹配	`Am7b5|;Cm6/Eb|;`	两个和弦的最终低音都不是 C，因此均不与 Cm6 等价。

查询 4

```
Cm6|* --notallowdifferentnotation
```

结果	具体源码	原因
匹配	`Cm6|;`	该选项要求记法匹配，源码正是 Cm6。
不匹配	`Am7b5/C|;`	Am7b5/C 的最终音集与低音等价，但采用不同记法。

查询 5

```
G/C|*
```

结果	具体源码	原因
匹配	`CM9no3|;`	两个和弦的最终低音都是 C，完整音集都是 C、D、G、B。
不匹配	`CM9no3/G|;`	完整绝对音集不变，但最终低音为 G。

8.3 数字低音链的 Roman 查询

语法

```
V[6,4-] / *[6,4] / [6,4-x]
```

对数字低音链中的后续事件，搜索会读取完整原始 Roman 链头，包括大小写、性质修饰、特殊记号与副调目标，并结合该事件自身的数字列表判断。
匹配位置是后续事件；返回的源码与谱面保留原有文字和记法。
查询列表会独立校验，不要求新的链起点标记。
查询数字列表仍须有二至四项，按 2 至 14 降序排列，并使用允许的临时记号。
例如，V[6,4] 可以匹配 V[5,3-],[6,4-],[5,3] 的中间事件，而 vi[6,4] 不匹配。
在查询中，省略结尾标记或写 -，均可接受源码列表继续、自然结束或以 -x 结束。
查询明确写 -x 时，只接受源码中明确以 -x 结束的列表。

查询 1

```
*|V[6,4-]
```

结果	具体源码	原因
匹配	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	中间事件通过继承原始 V 链头并使用自身 6、4 列表而匹配。结果定位到 C/G，并保留源码记法。

查询 2

```
*|V[6,4]
```

结果	具体源码	原因
匹配	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];`	查询可以省略新链起点标记。它匹配中间事件，并接受源码中的延续标记。
不匹配	`{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5-],G|[5,3];`	源码数字列表与要求的 6、4 列表不同。

查询 3

```
*|*[6,4-]
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];	链头通配符接受继承的 V，数字列表通配符接受 6、4。
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];	源码数字列表与要求的 6、4 列表不同。

查询 4

```
*|[6,4-]
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];	裸数字列表查询接受任意继承的 Roman 链头，同时仍校验列表。
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];	源码数字列表与要求的 6、4 列表不同。

查询 5

```
*|[6,4]
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];	省略查询结尾标记时，可以匹配源码延续标记。
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4];	省略查询结尾标记时，也可以匹配链自然结束。
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-x];	省略查询结尾标记时，也可以匹配显式 -x 结束。
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,5];	源码数字列表与要求的 6、4 列表不同。

8.4 OR 与 NOT 中的 Roman 数字列表

语法

```
(V[6,4]||*[6,4-]) / V[6,4]!(V[6,4-x])
```

Roman 数字列表查询在有限 OR 与完整字段 NOT 中使用相同的链头继承与结尾标记规则。每个 OR 分支都是完整 Roman 模式；NOT 排除项与实际源码事件比较，并检查其显式结尾标记。

查询 1

```
*|(V[6,4]||*[6,4-])
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G|V[5,3-],C/G|[6,4-],G|[5,3];	Roman OR 在中间事件接受继承的 V 分支。

查询 2

```
*|V[6,4]!(V[6,4-x])
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];	正向查询匹配中间事件，其延续标记不匹配显式 -x 排除项。
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-x];	该事件同时匹配正向查询和显式 -x 排除项，因此被 NOT 排除。

8.5 Roman 精确身份与显式链结尾

语法

```
vi[6,4] / V[6,4-x]
```

Roman 精确查询仍区分链头大小写。
查询明确使用 -x 时，只选择源码中在该事件明确结束链的列表。

查询 1

```
*|vi[6,4]
```

结果	具体源码	原因
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];	继承的链头是 V，而不是查询明确要求的小写 vi。

查询 2

```
*|V[6,4-x]
```

结果	具体源码	原因
匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-x];	显式 -x 查询匹配源码中明确终止链的事件。
不匹配	{BN:1}{TS:adlib}{D:1}{TC:C} G| V[5,3-],C/G|[6,4-],G|[5,3];	源码中间列表以 - 结尾，没有用 -x 明确终止链。

8.6 编译语素通配

语法

```
* / ?
```

通配符代表合法和弦或罗马数字记号的一部分。
含 * 或? 的和弦字段按记法匹配，默认开启异记法时也沿用此规则。
列表以外，? 恰好匹配一个编译语素，* 匹配零个或多个语素。
编译语素是记号解析产生的一个有意义单位。
根音及其升降号构成一个音名语素，因此? 可以整体匹配 F#。
固定关键字与数字组有各自的语素边界。

每个模式必须至少允许一个合法的具体记号。
整字段 * 接受省去的字段，排除显式字段占位 X 或字段延长 -。

查询 1

```
C*|*
```

结果	具体源码	原因
匹配	`C|I;Cm7|i7;C7/E|IDM65;`	每个和弦的根音都是 C。 后缀 * 分别匹配空后缀、m7 和 7/E，产生三个单事件窗口。
不匹配	`C#|;`	C# 是一个完整根音语素，因此固定根音 C 在应用后缀通配前就已不匹配。

查询 2

```
?m7|*
```

结果	具体源码	原因
匹配	`Cm7|i7;F#m7|#iv7;Bbm7|bvii7;`	? 分别匹配完整根音语素 C、F# 或 Bb。 这三个和弦随后都满足固定后缀 m7。
不匹配	`Cm|i;`	Cm 的后缀为 m，而查询要求完整后缀 m7。

查询 3

```
*|V*
```

结果	具体源码	原因
匹配	`G|V;G/B|V6;G7|V7;`	三个罗马数字记号都以 V 开始。 后缀 * 分别匹配空后缀、6 和 7。
不匹配	`Gm|v;`	源码罗马数字 v 为小写，而查询要求大写 V。

8.7 列表项通配

语法

```
C13(b5,*)|* / Cadd(?)|*
```

在 altered、add、sus、no、q4、q5 和 nf 列表中，通配符匹配完整列表项。
一个? 恰好匹配一项，一个 * 匹配一项或多项。
整个列表必须按书写顺序匹配。
例如，C13(b5,*) 要求 b5 后面至少还有一项。
A[q4:*]、D[q5:*] 和 C[nf:*] 分别简写了对相应完整非空列表的通配查询。

查询 1

```
C13(b5,*)|*
```

结果	具体源码	原因
匹配	`C13(b5,b9)|;C13(b5,b9,#9)|;`	两个列表都以要求的 b5 开始。 * 在第一个和弦中匹配 b9，在第二个和弦中匹配 b9,#9。
不匹配	`C13(b5)|;`	列表在 b5 后结束，没有项目可以满足要求非空的 *。

查询 2

```
Cadd(?)|*
```

结果	具体源码	原因
匹配	`Cadd(9)|;Cadd(#11)|;`	每个 add 列表都恰好包含一项。 ? 整体匹配项目 9 或 #11。
不匹配	`Cadd(9,11)|;`	源码 add 列表包含两项，而? 要求恰好一项。

查询 3

```
A[q4:*]|*
```

结果	具体源码	原因
匹配	`A[q4:(4,b7)]|;A[q4:(#4,7,10)]|;`	两个和弦都具有根音 A 和非空 q4 列表。 * 分别匹配完整列表 4,b7 或 #4,7,10。

查询 4

```
C[nf:*]|*
```

结果	具体源码	原因
匹配	`C[nf:(0,4,7)]|;C[nf:(0,3,7,10)]|;`	两个和弦都具有根音 C 和非空 nf 列表。 * 分别匹配完整列表 0,4,7 或 0,3,7,10。

8.8 实际相邻与透明事件

语法

```
C|*,G|*
```

相邻查询位置沿实际播放路线判断，包括反复回跳与选中的结尾。
因此，互斥结尾中的事件不能组成一串相邻序列。
透明事件是匹配器寻找下一个请求事件时跳过的事件。
裸写延长符（-）、未请求的休止（0）以及默认的次要事件属于透明事件。
显式请求的休止占用一个查询位置。
中间出现不满足下一个条件的普通和声事件时，窗口中断。
沿同一组源码事件 ID 的多次经过产生一个可见结果窗口。

查询 1

```
C|*,G|*
```

结果	具体源码	原因
匹配	`C|I;0;G|V;`	查询跳过未请求的休止 0，因此 C 和 G 满足连续的两个查询位置。
匹配	`C|I;'F|IV';G|V;`	查询默认跳过单引号标记的次要 F|IV，因此 C 和 G 满足两个查询位置。
不匹配	`C|I;F|IV;G|V;`	普通事件 F|IV 参与匹配，打断请求的 C 到 G 序列。

查询 2

```
C|*,0,G|*
```

结果	具体源码	原因
匹配	`C|I;0;G|V;`	三个源码事件依次满足请求的和弦 C、休止 0 及和弦 G。
不匹配	`C|I;G|V;`	源码在 C 与 G 之间没有休止，因此不满足中间查询位置 0。

8.9 纳入次要事件

语法

```
--notignore2nd
```

–notignore2nd 选项把单引号标记的次要事件纳入序列匹配。
它们可以满足请求位置，也可以像普通事件一样打断相邻关系。
该选项写在事件查询后，并用空白分隔。
每条查询最多允许一个该选项。

查询 1

```
C|*,G|* --notignore2nd
```

结果	具体源码	原因
匹配	`C|I;G|V;`	C 和 G 是相邻源码事件，并满足两个查询位置。
不匹配	`C|I;'F|IV';G|V;`	–notignore2nd 选项使次要 F|IV 参与匹配，因此它打断 C 到 G 序列。

8.10 TC 条件：普通、Ambi 与 None

语法

```
{TC:C} / {TC:M} / {TC:"M"} / {TC:None}
```

TC 条件检查事件处生效的调性参照，包括继承而来的参照。
C 等无引号具体中心选择普通参照。
无引号 M、m 和 * 分别选择任意普通大调参照、任意普通小调参照及任意普通参照。
带引号值”C”、”M”、”m” 和”*” 对模糊参照（Ambi）执行相应检查。
None 选择显式清除参照的状态，并排除初始未定义状态。
省去 TC 条件的查询允许罗马数字在普通或模糊参照下匹配。
连续罗马数字匹配必须沿同一底层音名与大小调路径进行。
仅从 C 改为”C” 的确定性变化保留该路径。

查询 1

```
{TC:M}*|V
```

结果	具体源码	原因
匹配	`G|V;{TC:D}A|V;`	G|V 继承普通 TC C，A|V 使用普通 TC D。 两个事件都在大调参照下具有罗马数字 V，满足 M 的要求。
不匹配	`{BN:1}{TS:4/4}{D:1}{TC:"C"} G|V;`	该事件具有罗马数字 V，但其带引号 TC ”C” 属于模糊参照。 无引号 M 条件要求普通大调参照。

查询 2

```
{TC:"M"}*|IV7
```

结果	具体源码	原因
匹配	`{BN:1}{TS:4/4}{D:1}{TC:"C"} FM7|IV7;`	该事件在模糊大调 TC ”C” 下具有精确罗马数字 IV7，满足带引号”M” 条件。
不匹配	`FM7|IV7;`	罗马数字 IV7 匹配，但普通 TC C 不满足模糊大调参照条件。

查询 3

```
{TC:None}C|*
```

结果	具体源码	原因
匹配	`{BN:1}{TS:4/4}{D:1}{TC:None} C|;`	该事件在显式清除的 TC None 下具有精确和弦 C。 整字段 * 接受省去的罗马数字字段。
不匹配	`C|I;`	和弦 C 匹配，但活动 TC C 不满足要求的 None 状态。

8.11 TS 条件

语法

```
{TS:3/4} / {TS:*/4} / {TS:adlib}
```

TS 条件检查事件处生效的拍号，包括继承而来的拍号。
完整值、分子或分母都可以使用 *。
例如，*/4 同时接受 3/4 和 4/4。
adlib 精确匹配自由拍子。
每个数值分量必须是至多 2147483647 的正整数。
事件搜索不接受 D 或 BN 条件。

查询 1

```
{TS:3/4}*|*
```

结果	具体源码	原因
匹配	{BN:1}{TS:3/4}{D:3/4}{TC:C} C|I;G|V;	C|I 和 G|V 都继承 TS 3/4，因此各自满足一个单事件查询窗口。
不匹配	C|I;	C|I 使用 TS 4/4，而查询要求精确 TS 3/4。

查询 2

```
{TS:*/4}*|*
```

结果	具体源码	原因
匹配	C|I;{TS:3/4}{D:3/4}G|V;	C|I 使用 TS 4/4，G|V 使用 TS 3/4。两个分母都等于要求的 4。
不匹配	{BN:1}{TS:6/8}{D:6/8}{TC:C} C|I;	C|I 使用 TS 6/8，其分母 8 不满足要求的分母 4。

8.12 局部与全曲活动标签

语法

```
<PAC> / <Mode_Dorian:*>
```

标签条件选择带有所请求活动标签的事件。
它把标签名称写在尖括号内，并省去 MAIN 结束斜杠。
活动集合合并局部范围、适用的一次性或结束标签，以及全曲标签（GLOBAL_TAGS）。
完整的精确标签身份可以独立于当前默认词表保留搜索。
带参数的终止式、pedal、Centricity 及配置的 Mode 或 Scale 标签允许在完整参数位置使用 * 或?。
每个通配符选择任意一个允许参数。
IAC:not_tonic 等含冒号的固定名称按名称匹配，而非按带参数家族匹配。

查询 1

```
<PAC>
```

结果	具体源码	原因
匹配	`G|V;C|I</PAC>;`	结束标签 </PAC> 标记 C|I 落点，因此只有该事件满足 <PAC>。

查询 2

```
<Mode_Dorian:*>
```

结果	具体源码	原因
匹配	`<Mode_Dorian:D>Dm|ii;G|V</Mode_Dorian>;`	两个事件都位于活动 Mode_Dorian:D 范围内。 * 接受其完整参数 D。
不匹配	`<Mode_Phrygian:E>Em|iii</Mode_Phrygian>;`	源码标签属于 Mode_Phrygian，而查询把家族固定为 Mode_Dorian。

查询 3

```
<PC:?>
```

结果	具体源码	原因
匹配	`F|IV;C|I</PC:IV>;`	C|I 事件带有结束标签 PC:IV。 ? 匹配完整的允许参数 IV。
匹配	`Fm|iv;C|I</PC:iv>;`	C|I 事件带有结束标签 PC:iv。 ? 也接受完整的允许参数 iv。

8.13 仅查元数据全曲标签

语法

```
--globaltag:<Tonality>
```

–globaltag 选项按元数据 GLOBAL_TAGS 中的项目筛选文档。
只写在 MAIN 中的标签不满足此元数据条件。
查询中每个不同的 –globaltag 条件都必须满足。
该选项可以单独使用，也可以与事件查询组合。
该选项接受 DTC 条件，包括无序配对及大小调通配，但不接受 Seq 或 SeqInv。
曲式准入遵循所绑定配置中的 allow_global 列。

查询 1

```
--globaltag:<Tonality>
```

结果	具体源码	原因
匹配	{GLOBAL_TAGS}</Tonality> C|I;	元数据 GLOBAL_TAGS 包含 </Tonality>，满足文档级条件。
不匹配	</Tonality>C|I;	Tonality 只出现在 MAIN 中，而 –globaltag 要求它出现在元数据 GLOBAL_TAGS 中。

查询 2

```
*|I --globaltag:<Sonata_Form>
```

结果	具体源码	原因
匹配	{GLOBAL_TAGS}</Sonata_Form> C|I;G|V;	GLOBAL_TAGS 包含 Sonata_Form，满足文档筛选。 只有 C|I 具有要求的罗马数字 I。
不匹配	</Sonata_Form>C|I;	C|I 满足罗马数字条件，但 Sonata_Form 只出现在 MAIN 中，不满足全曲标签条件。

查询 3

```
--globaltag:<DTC:C,a>
```

结果	具体源码	原因
匹配	{GLOBAL_TAGS}</DTC:a,C> C|I;	全曲调名对 a,C 与 C,a 是同一无序配对。
不匹配	<DTC:a,C>C|I;	局部 DTC 满足事件条件，但不满足这一仅查元数据的筛选。

8.14 曲式与 DTC 状态

语法

```
<Pop_Form:Verse> / <DTC:M,m>
```

曲式条件检查实际经过中的隐式或显式曲式状态。
终止标记 FORM_END 不属于活动搜索目标。
双主音复合体条件（DTC）检查一对无序调性参照。
例如，C,a 和 a,C 选择同一个调名对。
每个成员可以使用精确参照、M、m、* 或?。
M,m 选择一个大调参照和一个小调参照，二者可以具有任意调关系。
需要 TC 和罗马数字条件时，查询必须分别写出它们。
全曲曲式分类及全曲 DTC 也为事件查询提供活动状态。
需要限定标签来自元数据时，使用 –globaltag。

查询 1

```
<Pop_Form:Verse>
```

结果	具体源码	原因
匹配	`</Pop_Form:Verse>C|I;G|V;`	隐式 Verse 标记为 C|I 及后续 G|V 建立活动状态，因此两个事件都匹配。

查询 2

```
<DTC:C,a>
```

结果	具体源码	原因
匹配	`<DTC:a,C>C|I;Am|vi;`	活动调名对 a,C 与 C,a 包含相同参照。两个事件都继承该无序调名对并匹配。
匹配	`{GLOBAL_TAGS}</DTC:a,C> C|I;Am|vi;`	两个事件均继承全曲 DTC 调名对 a,C。

查询 3

```
<DTC:M,m>
```

结果	具体源码	原因
匹配	`<DTC:D,b>C|I;`	活动调名对 D,b 包含一个普通大调参照及一个普通小调参照，满足 M,m。
不匹配	`<DTC:C,G>C|I;`	活动调名对 C,G 包含两个大调参照，缺少 M,m 要求的小调参照。

查询 4

```
<DTC:*,*>
```

结果	具体源码	原因
匹配	`<DTC:C,G>C|I;`	两个通配符分别接受活动 DTC 调名对的成员 C 和 G。
不匹配	`C|I;`	源码事件没有活动 DTC 调名对，因此通配符没有可匹配的调名对成员。

8.15 Seq/SeqInv 列表查询

语法

```
<Seq:A2,*> / <SeqInv:?> / <Seq:A2*> / <Seq:A2?>
```

Seq 或 SeqInv 条件按书写顺序匹配完整的活动类型列表。
Seq 和 SeqInv 分别选择各自的标签家族。
精确项使用规则配置中的字面拼写。
一个? 匹配一个类型，一个 * 匹配一个或多个类型。
A2 等精确基础类型项只匹配该字面类型；精确查询派生类型时须写出其完整字面名称。
在已配置的基础类型后附加 * 或?，则选择该类型家族中的一个成员。

例如，A2* 与 A2? 均匹配 A2 或其一个已配置派生类型，如 A2(+5/-4)。
这两种家族写法都恰好匹配一个列表项。
这些条件用于活动事件标签，不能作为 GLOBAL_TAGS 筛选。

查询 1

```
<Seq:A2,*>
```

结果	具体源码	原因
匹配	`<Seq:A2,D2>C|I;F|IV</Seq>;`	活动 Seq 列表以 A2 开始，非空 * 匹配剩余 D2。 两个事件都位于该标签范围内。
不匹配	`<Seq:A2>C|I;F|IV</Seq>;`	Seq 列表在 A2 后结束，没有更多类型可以满足要求非空的 *。

查询 2

```
<SeqInv:?>
```

结果	具体源码	原因
匹配	`<SeqInv:A2>C|I;F|IV</SeqInv>;`	活动 SeqInv 列表恰好包含一个类型 A2，满足?。 两个事件都位于该标签范围内。
不匹配	`<SeqInv:A2,D2>C|I;F|IV</SeqInv>;`	SeqInv 列表包含 A2 和 D2，而? 要求整个列表恰好包含一个类型。

查询 3

```
<Seq:A2>
```

结果	具体源码	原因
匹配	`<Seq:A2>C|I</Seq>;`	完整活动 Seq 列表只有精确类型 A2。
不匹配	`<Seq:A2(+5/-4)>C|I</Seq>;`	派生类型 A2(+5/-4) 的字面名称与精确 A2 不同。

查询 4

```
<Seq:A2*>
```

结果	具体源码	原因
匹配	`<Seq:A2(+5/-4)>C|I</Seq>;`	A2* 选择已配置 A2 家族中的一个成员，该家族包含派生类型 A2(+5/-4)。
不匹配	`<Seq:A3>C|I</Seq>;`	A3 属于另一类型家族，不满足 A2* 项。

查询 5

```
<Seq:A2?>
```

结果	具体源码	原因
匹配	`<Seq:A2>C|I</Seq>;`	A2? 选择 A2 家族中的一个成员，也包括基础类型 A2 本身。
匹配	`<Seq:A2(+4/-3)>C|I</Seq>;`	派生类型 A2(+4/-3) 是 A2 家族的一个已配置成员，满足 A2?。
不匹配	`<Seq:A2,D2>C|I</Seq>;`	家族项 A2? 恰好匹配一个类型，不能匹配完整的两项列表 A2,D2。

8.16 可见与隐藏 BPM 设置

语法

```
{BPM:1/4=[90~120]}
```

BPM 条件选择附着在事件上的显式 BPM 或 BPMhidden 设置。
继承速度与 MM 推算速度不属于此条件的匹配对象。
拍单位分子、拍单位分母及速度数值各自接受 *。
精确速度可以使用十进制值。
速度区间 [lo~hi] 包含两端的整数，并要求 lo<=hi。
可见与隐藏设置都在查询中使用 BPM。
查询不支持 BPMhidden 设置名称。

查询 1

```
{BPM:1/4=[90~120]}
```

结果	具体源码	原因
匹配	`{BPM:1/4=90}C|I;{BPMhidden:1/4=120}G|V;`	C|I 带有 BPM 1/4=90，G|V 带有 BPMhidden 1/4=120。 两者都使用要求的拍单位，并分别位于 90~120 所包含的端点。
不匹配	`{BPM:1/4=121}C|I;`	显式速度 121 超过查询区间的上端点 120。

查询 2

```
{BPM:*/*=*}
```

结果	具体源码	原因
匹配	`{BPM:1/4=120}C|I;{BPMhidden:3/8=80}G|V;`	查询接受显式 BPM 和 BPMhidden 设置。 三个通配符同时接受 1/4=120 和 3/8=80。
不匹配	`C|I;`	C|I 没有附着的显式 BPM 或 BPMhidden 设置，而该查询要求此设置。

8.17 显式节奏转换

语法

```
{MM:1/4=1/8} / {MM:*}
```

MM 条件选择带有显式节奏转换设置的事件。
该条件只在该事件处应用一次。
后续事件继承转换后的速度时，不会仅因继承而满足该条件。
转换的各分量可以使用 *。
完整值 * 选择任意显式 MM。

查询 1

```
{MM:1/4=1/8}
```

结果	具体源码	原因
匹配	`{BPM:1/4=120}C|I;{MM:1/4=1/8}G|V;`	G|V 带有精确显式 MM 1/4=1/8。 前面的 C|I 只带有 BPM，因此只有 G|V 匹配。
匹配	`C|I;{MM:1/4=1/8}G|V;`	显式 MM 在此前未设置数值 BPM 时仍然匹配。

查询 2

```
{MM:*}
```

结果	具体源码	原因
匹配	`{BPM:1/4=120}C|I;{MM:1/4=1/8}G|V;{MM:1/8=1/2}C|I;`	通配符接受 G|V 上的 MM 1/4=1/8，以及末尾 C|I 上的 MM 1/8=1/2。 首个 C|I 只有 BPM，不满足 MM 条件。
不匹配	`{BPM:1/4=120}C|I;`	该事件带有 BPM 1/4=120，但没有显式 MM 设置。

8.18 NR 变换串查询

语法

```
[NR:P?] / [NR:*]
```

NR 条件选择显式绑定到事件的变换标注。
字面 P、L、R 串及单个 N、S、H 使用源码语法。
在变换模式中，? 匹配一步，* 匹配一步或多步。
例如，P? 接受 PL，但排除单独的 P。
匹配对象是已存储的变换判断。

查询 1

```
[NR:P?]
```

结果	具体源码	原因
匹配	`C|I;[NR:PL]Cm|i;`	Cm|i 事件带有 NR PL。 固定 P 匹配第一步，? 匹配第二步 L。
不匹配	`C|I;[NR:P]Cm|i;`	标注 P 只有一步，而 P? 要求 P 后面再接一步。

查询 2

```
[NR:*]
```

结果	具体源码	原因
匹配	`C|I;[NR:PLR]Cm|i;[NR:N]Db|bII;`	Cm|i 和 Db|bII 事件分别带有显式 NR 标注 PLR 和 N。 * 接受每个完整非空标注。
不匹配	`C|I;Cm|i;`	两个事件都没有 NR 标注，因此缺少要求的显式变换。

8.19 M/MH 条件

语法

```
{M:G|V} / {MH:*|*}
```

M 或 MH 条件选择对应的显式转调事件。
目标接受具体普通参照、带引号参照、普通参照通配 * 或模糊参照通配"*"。
目标不接受裸写 M 和 m 大小调通配。
关系接受基础罗马数字记号或 *。
OR 组可以在完整 M 与 MH 查询事件之间选择。
这两类设置禁止在值内部使用 OR 组。

查询 1

```
{M:G|V}
```

结果	具体源码	原因
匹配	`{M:G|V}D|ii&V{/M};G|I;`	D|ii&V 事件带有显式枢纽转调 M，其目标 G 与关系 V 满足两个精确值。

查询 2

```
{MH:*|*}
```

结果	具体源码	原因
匹配	`C|I;{MH:G|V}G|I;`	G|I 事件带有目标为 G、关系为 V 的显式 MH。 两个 * 值分别接受该目标与关系。
不匹配	`C|I;{TC:G}G|I;`	G|I 事件把 TC 改为 G，但没有 MH 设置，因此缺少要求的转调类型。

查询 3

```
({M:G|V}||{MH:G|V})
```

结果	具体源码	原因
匹配	`{M:G|V}D|ii&V{/M};G|I;`	D|ii&V 事件满足第一个 OR 分支 {M:G|V}。
匹配	`C|I;{MH:G|V}G|I;`	G|I 事件满足第二个 OR 分支 {MH:G|V}。

8.20 相对和弦音名

语法

```
(0)m|*,(2)m|*,(4)|*
```

相对音名用 -24 至 24 的括号整数替代绝对和弦音名。
它可以位于和弦根音、slash 低音或复合和弦两侧的对应位置。
查询中的所有相对音名共享一个未知的模 12 偏移。
整数给出相对于该偏移的半音位置。
例如，根音 (0)、(2) 和 (4) 请求在同一次移调下依次相隔两个半音的位置。
完整相对和弦与完整绝对和弦采用相同的最终低音和音集等价规则。
例如，(0)m6|*,(2)m6|* 匹配依次出现的 Am7b5/C 与 Bm7b5/D，并保留最终低音 C 到 D 的音程。
查询可以混用具体音名和相对音名，具体音名保持固定音高。
–notallowdifferentnotation 选项恢复记法匹配，同时保留统一的相对偏移。
相对音名记法只用于查询。

查询 1

```
(0)m|*,(2)m|*,(4)|*
```

结果	具体源码	原因
匹配	`Cm|i;Dm|ii;E|III;`	以 C 为共同偏移时，根音 C、D 和 E 分别处于相对位置 0、2 和 4。 前两个和弦还具有要求的小三和弦性质，E 具有要求的大三和弦性质。
匹配	`Dm|ii;Em|iii;F#|#IV;`	以 D 为共同偏移时，根音 D、E 和 F# 分别处于相对位置 0、2 和 4。 它们的小、小、大三和弦性质也满足三个查询字段。
不匹配	`Cm|i;Dm|ii;F|IV;`	前两个根音确定 C 为共同偏移。 末尾根音 F 位于 C 上方 5 个半音处，而最后的查询要求位置 4。

查询 2

```
(0)/(7)on(2)/(9)|*
```

结果	具体源码	原因
匹配	`C/GonD/A|;`	以 C 为共同偏移时，书写音名 C、G、D 和 A 在要求的根音与低音位置上分别对应 0、7、2 和 9。
不匹配	`C/GonD/F#|;`	C、G 和 D 在偏移 C 下满足位置 0、7 和 2。 末尾 F# 处于位置 6，不满足要求的位置 9。

查询 3

```
(0)m6|*,(2)m6|*
```

结果	具体源码	原因
匹配	`Am7b5/C|;Bm7b5/D|;`	两个音集分别等价于最终低音为 C 和 D 的小六和弦，并使用同一偏移。
不匹配	`Am7b5/C|;Cm7b5/Eb|;`	最终低音 C 与 Eb 相隔三个半音，查询要求两个半音。

8.21 最终音高类集合

语法

```
C[notes:0,4,7]|*
```

[notes:...] 查询比较和弦的完整最终音高类集合。
每个数字表示完成 slash 和 on 运算后，相对于最终根音的模 12 半音音程。
根音可以是具体音名、相对音名或 *。
具体根音保留指定的音名拼写，集合部分忽略构成音的拼写与八度重复。
例如，根音选择器 C# 不选择拼作 Db 的最终根音。

集合至少包含一个数字，由 0 至 11 中互不重复的整数严格递增排列。
匹配要求两个完整集合相等。
例如，C/E 的最终根音为 E，集合为 0、3、8。
[notes:...] 形式结束和弦字段，后面不能再接 slash、add 或 on 语法。
集合内部禁止通配与 OR，但 OR 组可以选择完整的 [notes:...] 形式。
–notallowdifferentnotation 选项保持这种显式集合查询的语义不变。

查询 1

```
C[notes:0,4,7]|*
```

结果	具体源码	原因
匹配	`C|I;`	和弦 C 的最终根音为 C，完整相对音高类集合为 0、4、7，满足和弦查询的两个部分。
不匹配	`C7|IDM7;`	C7 的完整相对音高类集合为 0、4、7、10。多出的 10 使它与要求的完整集合 0、4、7 不同。

查询 2

```
E[notes:0,3,8]|*
```

结果	具体源码	原因
匹配	`C/E|I6;`	slash 运算使 E 成为 C/E 的最终根音。音 E、G 和 C 随后给出要求的完整集合 0、3、8。
不匹配	`C|I;`	和弦 C 的最终根音为 C，相对集合为 0、4、7，而查询要求根音 E 及集合 0、3、8。

查询 3

```
*([notes:0,3,7]||[notes:0,4,7])|*
```

结果	具体源码	原因
匹配	`C|I;Dm|ii;`	根音 * 接受 C 和 D。 C 的完整集合为 0、4、7，Dm 的完整集合为 0、3、7，因此各自满足一个 OR 分支。
不匹配	`G7|V7;`	G7 的完整相对集合为 0、4、7、10，与列出的两个三音集合都不相等。

查询 4

```
C#[notes:0,3,7,9]|*
```

结果	具体源码	原因
匹配	`C#m6|;`	最终根音拼作 C#，完整相对集合为 0、3、7、9。
不匹配	`Bbm7b5/Db|;`	相对集合相等，但最终根音拼作 Db，与要求的 C# 不同。

8.22 有限 OR

语法

```
(A||B)
```

OR 组把用 || 分隔的选项放在括号内。
匹配可以满足其中任意一个选项。
外层语法决定选项是字段片段、完整查询事件、允许的完整设置值还是完整标签参数。
每个事件级选项恰好占一个查询位置。
组内选项必须非空，各组不能嵌套。
含有多个组的查询须在每组中满足一个允许选项。
一个和弦字段仅含一组 OR，且每种结果都是不含 * 或? 的具体和弦时，采用音集等价匹配。
例如，(Cm6||G/C)|* 接受最终低音同为 C 的 Am7b5/C 和 CM9no3。
含通配符或多个独立 OR 组的和弦字段按记法匹配。
Cm(7||9||11||13)*|* 接受所列 tension 拼写及合法的后续后缀，包括 no5。

查询 1

```
(C||G)|*
```

结果	具体源码	原因
匹配	`C|I;G|V;`	C|I 满足精确和弦分支 C，G|V 满足精确和弦分支 G。
不匹配	`Cm|i;`	完整和弦 Cm 既不等于精确分支 C，也不等于精确分支 G。

查询 2

```
(*|I||*|V)
```

结果	具体源码	原因
匹配	`C|I;G|V;`	C|I 满足单事件分支 *|I，G|V 满足单事件分支 *|V。 该查询产生两个独立窗口。
不匹配	`G7|V7;`	精确罗马数字 V7 既不等于要求的分支 I，也不等于 V。

查询 3

```
{TC:(C||"C")}*|I
```

结果	具体源码	原因
匹配	`C|I;{TC:"C"}C|I;`	第一个 C|I 使用普通 TC C，第二个使用模糊 TC ”C”。 设置 OR 接受这两种参照，且两个罗马数字字段都为 I。
不匹配	`{BN:1}{TS:4/4}{D:1}{TC:D} D|I;`	罗马数字 I 匹配，但活动 TC D 既不等于允许的参照 C，也不等于”C”。

查询 4

```
(Cm6||G/C)|*
```

结果	具体源码	原因
匹配	`Am7b5/C|;CM9no3|;`	每个事件的低音为 C，完整音集满足一个具体 OR 分支。
不匹配	`Am7b5|;CM9no3/G|;`	每个事件的最终低音都不是 C。

查询 5

```
Cm(7||9||11||13)*|*
```

结果	具体源码	原因
匹配	`Cm7|;Cm9|;Cm11|;Cm13|;Cm9no5|;`	OR 选择书写的 tension，后缀通配也接受 no5。
不匹配	`Cm6|;CmM7|;`	6 与 M7 均不以所列 tension 拼写之一开头。

8.23 完整字段 NOT

语法

```
P!(N1||N2)
```

NOT 后缀!(N1||N2) 从正向和弦或罗马数字字段 P 中排除完整模式。
每个字段允许一个后缀，写在完整正向模式之后，且与该模式位于 | 的同一侧。
和弦排除项位于 | 前，罗马数字排除项位于 | 后。
每个排除项都是完整的具体或通配模式。
具体和弦排除项默认按最终低音和音集等价，附加 –notallowdifferentnotation 时改按记法；通配排除项按记法匹配。
正向模式必须匹配，每个主解释与附加解释都必须避开全部排除项。
罗马数字排除项检查所有分析层。
正向模式决定共享相对偏移及调性路径的连续性。
Cm(7||9||11||13)*!(*no*)|* 选择这些书写的 tension 家族，并排除包括 Cm9no5 在内的所有带 no 的记法。
NOT 要求实际和声字段，不能否定 X。
排除项不能包含相对音名或 [notes:...] 集合。
NOT 组禁止嵌套，也不能出现在 OR 分支内部。

查询 1

```
Cm*!(*/*||*on*)|*
```

结果	具体源码	原因
匹配	`Cm|i;Cm7|i7;`	Cm 和 Cm7 都满足正向模式 Cm*。 两者都避开了排除的 slash 模式 */* 与复合和弦模式 *on*。
不匹配	`Cm/Eb|i6;CmonAm|;`	两个和弦都满足 Cm*。 Cm/Eb 命中排除项 */*，CmonAm 命中排除项 *on*，因此两者都被排除。

查询 2

```
*|V*!(V7)
```

结果	具体源码	原因
匹配	G|V;G/B|V6;	罗马数字 V 和 V6 都满足 V*。 两者都不等于精确排除项 V7。
不匹配	G7|V7;	罗马数字 V7 满足正向模式 V*，但也等于排除项 V7，因此该字段被排除。

查询 3

```
Cm(7||9||11||13)*!(*no*)|*
```

结果	具体源码	原因
匹配	Cm7|;Cm9|;Cm11|;Cm13|;Cm7/G|;	五种记法均满足正向模式，且都不含 no 关键字。
不匹配	Cm9no5|;Cm11no5|;	和弦侧 NOT 排除两个 no5 后缀。

查询 4

```
*!(Cm6)|*
```

结果	具体源码	原因
匹配	Am7b5|;	最终低音 A 与 C 不同，因此该和弦不作为等价 Cm6 被排除。
不匹配	Am7b5/C|;	具体排除项采用相同最终低音与完整音集，因此等价于 Cm6。

8.24 特殊事件与不支持的控制

语法

```
X / 0 / N.C. / Clus / ClusWK / ClusBK
```

完整事件查询接受 X、0、N.C.、Clus、ClusWK 和 ClusBK。
和声查询字段内部的 X 选择该字段的占位标记。
例如，X|V 要求和弦字段占位，并要求罗马数字精确为 V。
裸写 - 与字段级 - 不能直接查询。
序列匹配把延长作为透明事件跳过。
事件查询不接受 BN、D、T、TEMPO、LB 设置，也不接受 COMMENT 和 F 标注。

查询 1

```
X
```

结果	具体源码	原因
匹配	X;	源码包含完整占位事件 X，满足完整事件查询 X。

查询 2

```
ClusWK
```

结果	具体源码	原因
匹配	`ClusWK;`	完整源码事件 ClusWK 精确匹配请求的白键音块。

查询 3

```
X|V
```

结果	具体源码	原因
匹配	`X|V;`	和弦字段包含要求的占位标记 X，罗马数字字段等于要求的 V。
不匹配	`G|V;`	罗马数字 V 匹配，但实际和弦 G 不满足要求的和弦字段占位 X。

8.25 硬限制

一条查询最多允许 4096 个源码字符、32 个查询事件及 16 个全曲标签选项。
每个和声字段最多允许 256 个字符和 16 个通配符。
OR 每组最多允许 8 个分支，每条查询最多允许 8 组，分支总数最多为 32。
一条查询最多允许 32 个 NOT 组，每个字段最多允许一个。
NOT 内部的 OR 组计入同一 OR 上限。
正向与排除模式共享字段长度及通配符上限。
解析器的确定性工作量分数必须至多为 512。
这些上限约束查询准入，结果窗口数则取决于匹配到的源码。

9 渲染规则

9.1 时间顺序与语义分层

MStruct 将分析呈现为依次排列的谱行（system），每个谱行是渲染谱面中的一整横行。
谱行内部，小节与带时值的槽位从左向右推进。
同一位置的不同类别信息各占指定的纵向区域。
横向组织传达音乐顺序与时值，纵向组织区分和声、结构标注和解释材料。
读者可以横向追踪和声进行，并在同一位置的上下方读取相关信息。
本章记录这些显示约定及其排版关系。
各显示项的语法与音乐含义见第 2 章至第 6 章。

9.2 纵向 Grid 从上到下的组成

每个谱行使用一套共享的纵向 Grid。
以下按视觉上从上到下的顺序列出各区域。
可选区域由整个谱行的可见内容共同决定是否占位。
同一谱行的小节、槽位和省略号共享相应的行位置。

1. **顶部留白**

   最高记号上方保留间距，用于分隔相邻谱行，并容纳上移的小节号。

2. **曲式符号轨**

   `[F:A||]` 等带框曲式符号位于速度标记上方的可选轨道。
   其位置指示曲式边界或段落。

3. **速度标记与上移的小节号**

   可见的 TEMPO、MM、BPM 标记按纵向堆叠。
   因反复结尾括号（volta bracket）而上移的小节号，可在横向错开的位置共用这一高度带。
   只有反复结尾括号的谱行也为上移的小节号预留空间。

4. **小节号与时间标签基准轨**

   普通小节号与 `{T=0:15}` 等书写时间标签共用音乐主体上方的基准。
   反复结尾括号锚定在这一区域，并覆盖其适用的小节或槽位。

5. **音乐主体轨**

   拍号、小节线、和弦记号、Roman、独立事件及反复边界位于中央区域。
   终止小节线 `{||END}` 左细右粗。
   和声区域内部可包含多层对齐的子行。

6. **时值轨**

   时值标记位于音乐主体下方。
   渲染器可将它们显示为分数或音符时值。

7. **COMMENT 注释轨**

   注释标记及注释范围的锚点占用独立的可选轨道。
   整个谱行没有可见注释时，这条轨道及其附加间隙一并收起。

8. **分析标签轨**

   普通标签和语义标签位于注释下方。
   同时存在的标签范围可使用不同的 lane，各 lane 内的标签与持续线保持对齐。
   谱行按实际可见的标签 lane 紧凑排列。

9. **底部留白**

   最后一条占用轨道下方保留间距，将本谱行与下一谱行分开。

9.3 曲式、速度与基准轨

带框曲式符号轨位于速度组上方。
同一槽位内，速度项依次为 TEMPO、按源码顺序排列的 MM 关系、BPM。
整组贴齐速度轨底部，因此单个速度项与多项堆叠时的最下项共用同一底部基准。
`BPMhidden` 参与播放状态，其记号保持隐藏。
书写时间标签 `{T=...}` 占用小节号基准轨。
单独的时间标签使用普通谱行布局，可见的 TEMPO、MM 或 BPM 才启用速度轨。
反复结尾括号以小节号区域为锚点。
只有横向被括号覆盖的小节号上移到其横线上方。
其间距根据括号几何位置确定，因此增添速度项时仍保持括号与这些小节号之间的关系。

9.4 和声事件的内部排列

C、R 两个字段同时存在时，和弦记号字段 C 显示在 Roman 字段 R 上方。
同一谱行内的槽位共享相应的和声子行高度，某一字段为空的位置也沿用该高度。
转调可在音乐主体内部用不同子行展示来源与目标两种 Roman 解释。
主要 Roman 符号提供对齐基准，调性前缀和补充解释围绕其安排。
和弦与 Roman 的字形排版区分根音或级数、性质、扩展音、变化音、低音信息与附加解释。
可用宽度或显示比例改变时，这些组成部分保持内部关系。
拍号在相应音乐边界占位。
小节线与反复边界覆盖音乐主体，时值和分析标注则位于各自的轨道。

9.5 时值、注释与分析标签

时值轨把可见时值标记与其上方的音乐位置对应起来。
分数显示与音符时值显示表达同一组底层时间信息。
注释轨显示紧凑的编号标记，与 COMMENT 分区中的文字关联。
范围注释沿这条轨道连接其起止位置。
标签区域用文字标记分析特征，范围型标签还通过持续线表示延续。
点状标签附着在其自身位置。
带框曲式符号 `[F:...]` 使用上方曲式轨，曲式标签 `<...>` 则与分析标签共同排列。
谱行中任一位置的注释需求决定整行是否需要注释轨，所有槽位使用由此形成的布局。
这条轨道收起时，标签区域整体上移。

9.6 范围标记与跨谱行连续性

反复结尾括号、注释范围与标签范围均关联音乐中的起止位置。
渲染器在横向排版完成后定位这些位置，并在范围经过的谱行上绘制对应线段。
范围遇到换行时延伸到当前谱行末端，再在下一谱行接续。
最后一段到达相应的终点。
页面或窗口宽度改变并重新换行后，范围与音乐事件的对应关系仍然保持。
每个谱行内，标签 lane 为文字与范围线提供一致高度。
这种组织把连续的分析范围与可换行为多行的谱面结合起来。

9.7 横向间距与换行

每个槽位根据自身时值与可见内容所需空间取得自然宽度。
较长时值提供更多节奏空间，记号和标注则确定便于阅读的最小宽度。
调性前缀、拍号与反复记号在相应位置增加必要的避让空间。
需要横向铺满的谱行，按基于时值的伸展权重分配额外空间。
因此，视觉间距同时体现节奏比例和记号宽度需求。
渲染器选择换行位置时，通常保持完整小节。
当一个小节宽于整条谱行时，可在槽位之间拆分，并在后续谱行继续显示。
单个槽位保持完整。
显式换行结束当前谱行，省略号 ... 也结束其所在谱行。
末谱行以及以省略号结束的谱行保留自然间距。
分页导出在完整谱行之间设置页面边界。

9.8 图层显隐与重新排版

渲染器把速度标记、曲式信息、分析标签、注释和新里曼标注分组为可选择的显示图层。
可见内容决定相应轨道和 lane 需要的空间。
隐藏图层会改变呈现内容，并可能缩小谱面的宽度或高度。
底层事件、分析关系及播放时间仍保存在文档中。
窗口宽度、显示比例或图层选择改变时，重新排版沿用同一槽位顺序与语义轨道关系。
共享渲染器将这一组织方式应用于编辑器、查看器和导出谱面。

许可与字体

本文正文、原创示例及规则说明采用 CC BY 4.0 许可。
嵌入字体保留各自许可：Latin Modern 采用 GUST Font License；Fandol 采用 GPL 及其字体例外；Academico、Bravura、Crimson Pro 和源于 Noto Music 的 MStruct Text Accidentals 采用 SIL Open Font License 1.1。
部分插图包含 Windows Arial、Cambria Math 和 Microsoft YaHei 的获准嵌入子集。
这些字体嵌入 PDF 供文档使用，文档许可不重新授权字体软件。

Creative Commons Attribution 4.0 International (CC BY 4.0)

引用

Polymath9. (2026). MStruct：技术手册 (版本 1.0.0).
https://doi.org/10.5281/zenodo.23637500.

参考来源

1. Celeca. Notations of simai. https://w.atwiki.jp/simai/pages/1003.html. 访问日期: 2026-10-25.
2. The LaTeX Project. An introduction to LaTeX. https://www.latex-project.org/about/. 访问日期: 2026-10-25.